



Thakur Educational Trust's (Regd.)
THAKUR COLLEGE OF SCIENCE & COMMERCE
Empowered Autonomous College Permanently Affiliated to University of Mumbai
(NAAC Accredited with Grade "A" (3rd Cycle) & ISO 21001:2018 Certified)
Best College Award by University of Mumbai for the Year 2018-2019



DEPARTMENT OF INFORMATION TECHNOLOGY

A PROJECT REPORT ON

Browser Based Discrete – Event Simulator

“JustCmul8: A No-Code, Browser-Based Discrete Event Simulation Platform”



SUBMITTED BY

Ms. Khushi Singh – 261713

Mr. Mohit Gupta – 261715

UNDER THE GUIDANCE OF

Ms. Jinal Gujar

Submitted in partial fulfilment of the requirements for qualifying

BACHELOR OF SCIENCE (INFORMATION TECHNOLOGY)

Semester – V Examination

THAKUR COLLEGE OF SCIENCE AND COMMERCE

(Empowered Autonomous College, Permanently Affiliated to University of Mumbai)

THAKUR VILLAGE, KANDIVALI (EAST), MUMBAI – 400 101, MAHARASHTRA

ACADEMIC YEAR 2026 – 2027



Thakur Educational Trust's (Regd.)
THAKUR COLLEGE OF SCIENCE & COMMERCE
Empowered Autonomous College Permanently Affiliated to University of Mumbai
(NAAC Accredited with Grade "A" (3rd Cycle) & ISO 21001:2018 Certified)
Best College Award by University of Mumbai for the Year 2018-2019



CERTIFICATE OF APPROVAL

This certifies that the project titled “**Browser Based Discrete – Event Simulator**” was carried out at Thakur College of Science and Commerce by **Ms. Khushi Singh (Roll No. 261713)** and **Mr. Mohit Gupta (Roll No. 261715)**, students of Bachelor of Science (Information Technology). It is submitted in partial fulfilment of the Semester–V examination for the B.Sc. (Information Technology) degree of the University of Mumbai, academic year 2026 – 2027.

The students have finished every phase of the project. The work in this report is theirs, and it has not been offered to any other university or institute for a degree or diploma.

Ms. Jinal Gujar
Project Guide

Prof. Dr. Santosh Kumar Singh
Head of Department

External Examiner

Principal

Date: _____

College Seal / Stamp: _____

CANDIDATES' DECLARATION

We declare that the project described in this report, “**Browser Based Discrete – Event Simulator**”, is our own work. It was done during the academic year 2026 – 2027 under the guidance of Ms. Jinal Gujar and is submitted to the Department of Information Technology, Thakur College of Science and Commerce, in partial fulfilment of the Bachelor of Science in Information Technology (Semester–V) degree.

The analysis, design, source code, integration of the simulation engine, testing and write-up were all done by the two of us. Where we drew on published literature, official technical documentation or open-source libraries, we have said so in the text and listed the source under References. None of this material has been submitted, wholly or partly, to another university or institute for any degree or diploma.

We accept that any lapse in academic integrity, whenever it is found, can lead to disciplinary action under the rules of the college and the University of Mumbai.

Place: Mumbai

Date: _____

Ms. Khushi Singh

Roll No. 261713

Mr. Mohit Gupta

Roll No. 261715

ACKNOWLEDGEMENT

Many people helped this project along, and we would like to thank them here.

Our first thanks go to our guide, **Ms. Jinal Gujar** of the Department of Information Technology, who kept us on course through the whole development of JustCmul8. She insisted that the simulation engine be checked before any feature was stacked on it. We had planned it the other way round, and following her advice spared us a lot of rework. She also read our early drafts, and the shape of this report came out of her comments.

Prof. Dr. Santosh Kumar Singh, Head of the Department of Information Technology, made it possible for students to take on a project of this size, and his suggestions in the review sessions were useful. We are grateful to him.

We thank the Principal and management of Thakur College of Science and Commerce for the laboratory, network and machines we used. Testing a WebAssembly Python runtime, and trying real-time collaboration on several computers at once, needed the department laboratory.

The faculty of the department taught us data structures, database systems, software engineering and web technologies, and we used that material directly: priority queues, normalisation and the life-cycle models all appear in the work below. The laboratory assistants also deserve thanks for sorting out machine access and configuration during long testing sessions.

Ms. Khushi Singh – 261713

Mr. Mohit Gupta – 261715

ABSTRACT

Discrete Event Simulation (DES) can show how a queue, a production line or a delivery network will behave before anyone spends money changing it, but few of the people who could use it ever do. Commercial packages such as AnyLogic, Arena and Simul8 charge licence fees of thousands of dollars a year. The main open-source option, SimPy, is a Python library, so a user must write and debug code before a first model runs. Nearly all of these tools are desktop programs that need installing, which is a problem in shared laboratories and on managed machines. Drawing tools such as Visio and Lucidchart go the other way: they are easy, but their diagrams never execute and cannot say what happens if one more counter is opened.

JustCmul8 is our attempt at filling that gap. Users drag nodes onto a canvas, connect them, fill in each node through ordinary form fields and run the model, all in the browser with nothing to install. The key design choice is that the model is never approximated in JavaScript. The graph is compiled into a real Python SimPy script, which a CPython interpreter built for WebAssembly (Pyodide) runs inside a Web Worker so the interface does not freeze. The numbers on screen therefore come from the same engine a researcher would run in a terminal.

The front end is a client-heavy single-page application written with Next.js 16, React 19 and TypeScript; the canvas uses React Flow and shared state lives in Zustand. Supabase handles sign-in, storage and real-time channels, and PostgreSQL Row-Level Security keeps projects separate at the database instead of relying on client code. There are twelve functional modules and fifteen node types covering six domains: human queues, vehicle traffic, liquid and material flow, manufacturing, logistics and network signalling. After a run, the analytics layer works out utilisation, waiting times, throughput, queue lengths, a Little's Law consistency check and a composite system health score, and it names the bottleneck node. A Monte Carlo facility repeats the run over independent replications and gives 95 per cent confidence intervals rather than a single figure. Around these sit an animated digital-twin playback, a cost and return-on-investment panel, a Python code inspector with Jupyter export, executive PDF reports and read-only share links that can be revoked. A Google Gemini assistant, reached through a model router that watches model health and falls back automatically, turns a plain-English description into a candidate graph and suggests fixes for the bottleneck it finds.

We checked the engine against closed-form queueing theory. The verification suite passed all 28 analytical checks (M/M/1, M/M/c and saturated-queue benchmarks) and all 22 structural

invariants of a multi-stage hospital-flow model, with simulated utilisation, waiting time and queue length within 3.5 per cent of theory. The outcome is a working platform that removes the cost, coding and installation barriers to simulation and keeps the accuracy of the underlying engine.

Keywords: discrete event simulation, SimPy, WebAssembly, Pyodide, no-code modelling, Monte Carlo analysis, Next.js, Supabase, AI-assisted modelling.

INDEX

1. INTRODUCTION.....	1
1.1 Background and Motivation	1
1.2 Objective and Scope of the Project.....	3
1.3 Problem Definition.....	6
1.4 Limitations	7
2. REQUIREMENT ANALYSIS.....	9
2.1 User Requirements / SRS / Functional Requirements	9
2.2 Feasibility Study	13
2.3 Details of Hardware and Software Used.....	15
2.4 Proposed System and Advantages of the Proposed System	17
3. PROJECT PLANNING.....	20
3.1 List of Activities / Tasks	20
3.2 Timeline for Each Task.....	22
3.3 Gantt Chart and Activity Network Diagram	24
3.4 Resource Planning	27
3.5 Proposed Software Development Model	29
4. SOFTWARE / HARDWARE MODULES	32
4.1 Module 1: Platform and Data Layer	33
4.2 Module 2: Visual Graph Editor.....	34
4.3 Module 3: Node Configuration Panel	36
4.4 Module 4: Simulation Engine	37
4.5 Module 5: AI Assistant	38
4.6 Module 6: AI System Optimiser	39
4.7 Module 7: KPI Dashboard and Results.....	39
4.8 Module 8: Monte Carlo and Scenario Comparison	41
4.9 Module 9: Digital Twin Animated Playback	41
4.10 Module 10: Simulation Template Gallery	42
4.11 Module 11: Executive Report and Share	43
4.12 Module 12: Real-Time Collaboration.....	43
4.13 Hardware Modules and Considerations	44
4.14 Individual Work Justification	45
5. SYSTEM ANALYSIS AND DESIGN.....	47
5.1 Context Diagram.....	47
5.2 DFD, ERD, Class Diagram and State Transition Diagram.....	48

5.3 Use Case Diagram.....	54
5.4 Architecture Design	57
5.5 Activity, Component and Collaboration Diagrams	59
5.6 Table Design	62
5.7 Architecture / Deployment Diagram.....	66
6. SYSTEM IMPLEMENTATION AND METHODOLOGY	68
6.1 Development Methodology	68
6.2 Technologies Used.....	68
6.3 Algorithms and Techniques Used.....	69
6.4 Module-wise Implementation	72
6.5 Database Implementation.....	72
6.6 User Interface Design	73
6.7 Important Code Snippets.....	76
7. SYSTEM TESTING	80
7.1 Testing Strategy	80
7.2 Testable Units	81
7.3 Test Case Design with Reference to the Project.....	82
7.4 Test Approach.....	86
7.5 Test Results and Validation	87
8. RESULTS AND DISCUSSION	89
8.1 System Output.....	89
8.2 Result Analysis	97
8.3 Performance Evaluation.....	99
9. COST AND BENEFIT ANALYSIS AND SOFTWARE PARAMETER ESTIMATION.....	102
9.1 Effort Estimation.....	102
9.2 KSLOC Analysis	103
9.3 Function Point Analysis.....	104
9.4 Comparison of Estimates and Cost–Benefit Analysis	106
10. CONCLUSION AND FUTURE SCOPE	108
10.1 Achievements of the Project	108
10.2 Conclusion	109
10.3 Future Scope	109
REFERENCES AND BIBLIOGRAPHY	112

LIST OF FIGURES

Figure 1.1: Functional Modules of JustCmul8 grouped by area	5
Figure 2.1: Execution Pipeline of the Proposed System	18
Figure 3.1: Weekly Gantt Chart of the JustCmul8 Development Schedule (July W1 – December W4, 2026).....	24
Figure 3.2: Activity Network Diagram of the JustCmul8 Development Lifecycle	26
Figure 3.3: Iterative and Incremental Development Model adopted for JustCmul8.....	29
Figure 4.1: Module Structure of JustCmul8.....	32
Figure 5.1: Context Diagram (DFD Level 0) for JustCmul8	47
Figure 5.2: Level-1 Data Flow Diagram for JustCmul8	49
Figure 5.3: Level-2 Data Flow Diagram: Decomposition of Process P4 (Simulation Execution)	50
Figure 5.4: Entity Relationship Diagram for the JustCmul8 Database	51
Figure 5.5: Class Diagram of the JustCmul8 Object Model	52
Figure 5.6: State Transition Diagram for a Node During Simulation Execution.....	53
Figure 5.7: Use Case Diagram for JustCmul8	54
Figure 5.8: Multi-Tier Architecture of JustCmul8.....	58
Figure 5.9: Activity Diagram of the End-to-End User Workflow	60
Figure 5.10: Component Diagram of the JustCmul8 Implementation.....	61
Figure 5.11: Collaboration Diagram for the Simulation Run Interaction	62
Figure 5.12: Deployment Architecture of JustCmul8	67
Figure 6.1: Landing Page.....	74
Figure 6.2: Registration and Login Screens.....	75
Figure 6.3: Project Dashboard	75
Figure 6.4: Workspace with the Visual Graph Editor.....	75
Figure 6.5: Node Configuration Panel	76
Figure 6.6: Template Gallery	76
Figure 7.1: Layered Testing Strategy for JustCmul8.....	80
Figure 8.1: Executive Summary Tab of the Results Dashboard	89
Figure 8.2: Flow and Balance Check (Little’s Law Verification)	90
Figure 8.3: Traffic Heatmap	90
Figure 8.4: Line Buildup Timeline	91
Figure 8.5: Station Counters	91
Figure 8.6: Person / Item Journey (Entity Tracer)	92
Figure 8.7: Cost and Return-on-Investment Panel.....	92
Figure 8.8: Monte Carlo Analysis with 95% Confidence Bands	93
Figure 8.9: Digital Twin Animated Playback	93
Figure 8.10: AI Assistant Generating a Model from a Description	94
Figure 8.11: AI System Optimiser Recommendation	94
Figure 8.12: Python Code Inspector	95
Figure 8.13: Real-Time Presence and Live Cursors	95
Figure 8.14: Export, Share and Executive PDF Report	96

Figure 8.15: Read-Only Shared View	96
Figure 8.16: Profile and Settings.....	97
Figure 8.17: Simulated Results against Queueing Theory (M/M/1 and M/M/3 benchmarks).....	97
Figure 8.18: Deviation of Simulated Metrics from Theory against Accepted Tolerance	98

LIST OF TABLES

Table 1.1: Comparison of Discrete Event and Continuous Simulation	2
Table 1.2: Probability Distributions Implemented in JustCmul8.....	2
Table 2.1: User Classes and Access Levels	9
Table 2.2: Functional Requirements	10
Table 2.3: Non-Functional Requirements.....	12
Table 2.4: Indicative Cost Comparison with Commercial Alternatives	14
Table 2.5: Development Hardware and Software Configuration.....	15
Table 2.6: Minimum and Recommended Client Requirements.....	16
Table 2.7: Technology Stack with Justification.....	17
Table 2.8: Comparison of the Proposed System with Existing Approaches.....	18
Table 3.1: List of Activities and Tasks with Dependencies.....	21
Table 3.2: Planned Timeline of Each Task (July W1 – December W4, 2026).....	22
Table 3.3: Project Milestones and Completion Criteria.....	23
Table 3.4: Human Resources and Responsibilities	27
Table 3.5: Hardware, Software and Service Resources	28
Table 3.6: Planned Effort Allocation by Phase.....	28
Table 3.7: Phases of the Adopted Life-Cycle Model.....	30
Table 3.8: Why the Iterative and Incremental Model Was Preferred	30
Table 4.1: Module Overview and Requirement Coverage.....	32
Table 4.2: The Fifteen Node Types Supported by the Simulation Engine	35
Table 4.3: The Six Simulation Domains and Their Vocabularies	36
Table 4.4: Key Performance Indicators and Their Computation	40
Table 4.5: Hardware Roles in the Deployed System	44
Table 4.6: Division of Work between Team Members.....	45
Table 5.1: Use Case Specification: Run Simulation	55
Table 5.2: Use Case Specification: Generate Model via AI	56
Table 5.3: Use Case Specification: Share Project Read-Only	56
Table 5.4: Use Case Specification: Collaborate in Real Time.....	57
Table 5.5: Table: auth.users (managed by Supabase Auth).....	63
Table 5.6: Table: public.profiles	63
Table 5.7: Table: public.projects.....	63
Table 5.8: Table: public.simulation_runs	64
Table 5.9: Table: public.chat_history	64
Table 5.10: Table: public.project_shares	65
Table 5.11: Row-Level Security Policies	65
Table 6.1: Complexity of Graph Operations on the Adjacency List.....	70
Table 6.2: Module-wise Implementation Summary.....	72
Table 7.1: Testable Units and the Techniques Applied to Them	81
Table 7.2: Engine Verification Test Cases and Results	82
Table 7.3: Invariants Asserted for the Complex Model (Benchmark B6)	83

Table 7.4: Functional Test Case Design	84
Table 7.5: Edge-Case Probes and Observed Behaviour.....	87
Table 7.6: Requirements Traceability	88
Table 8.1: Worked Example: M/M/1 Queue at $\rho = 0.8$ ($\lambda = 0.8$, $\mu = 1.0$, 36,000 s, 5 replications)	98
Table 8.2: Results for the Multi-Stage Hospital-Flow Model (7,200 s, 3 seeds).....	98
Table 8.3: Engine Throughput Measured by the Verification Harness.....	100
Table 8.4: Browser Performance Measurements (to be recorded).....	100
Table 9.1: COCOMO Coefficients by Development Mode.....	102
Table 9.2: COCOMO Estimation Results.....	102
Table 9.3: Source Lines of Code by Layer	103
Table 9.4: Measured Project Size	103
Table 9.5: Standard IFPUG Complexity Weights.....	104
Table 9.6: Function Point Count by Component Type	104
Table 9.7: General System Characteristics and Degrees of Influence	105
Table 9.8: Comparison of Estimates against Planned Effort	106
Table 9.9: Development and Operating Costs	107
Table 9.10: Benefits of the Platform.....	107

LIST OF ABBREVIATIONS

Term	Meaning	Term	Meaning
AI	Artificial Intelligence	NFR	Non-Functional Requirement
API	Application Programming Interface	OAuth	Open Authorization
COCOMO	Constructive Cost Model	PK / FK	Primary Key / Foreign Key
CRUD	Create, Read, Update, Delete	PM	Person-Months
CSV	Comma-Separated Values	PRNG	Pseudo-Random Number Generator
DES	Discrete Event Simulation	Pyodide	CPython distribution for WebAssembly
DFD	Data Flow Diagram	RBAC	Role-Based Access Control
ERD	Entity Relationship Diagram	RLS	Row-Level Security
FIFO / LIFO	First / Last In, First Out	SDLC	Software Development Life Cycle
FP / FPA	Function Point / Analysis	SimPy	Simulation in Python (DES library)
FR	Functional Requirement	SLOC / KSLOC	(Thousand) Source Lines of Code
HCI	Human–Computer Interaction	SPA	Single-Page Application
JSON / JSONB	JavaScript Object Notation / Binary	SRS	Software Requirements Specification
KPI	Key Performance Indicator	TLS	Transport Layer Security
LLM	Large Language Model	UAT	User Acceptance Testing
M/M/1, M/M/c	Markovian queues with 1 or c servers	UFP / VAF	Unadjusted FP / Value Adjustment Factor
ML	Machine Learning	UML	Unified Modeling Language
WASM	WebAssembly	UUID	Universally Unique Identifier

1. INTRODUCTION

A bank branch, a factory floor, a hospital ward and a courier company all ask the same thing sooner or later: what happens if we change this? Trying a new counter, a different workstation order or a re-routed fleet in real life costs money, and a wrong guess costs more. Discrete Event Simulation (DES) lets a computer answer before anything is spent. The catch has always been that it needed an expensive licence, some programming and a local install. **JustCmul8** was started to remove all three at once. The chapter explains our reasons for building it, what we set out to achieve, the problem behind it and where its boundaries lie.

1.1 Background and Motivation

1.1.1 Simulation as a Decision-Making Tool

Real operations are shaped by randomness and by competition for shared resources. Customers do not turn up at even intervals. A machine does not take the same time on every part. Two entities often want the same server at the same moment. Averages hide all of this: a counter that needs four minutes per customer on average can still build a long line if people arrive in clumps. A simulation draws thousands of random arrivals and service times and watches what the system does with them, so the person deciding sees the likely outcome and also how widely it can vary.

The technique is used in manufacturing, healthcare, logistics, transport, telecommunications and public services. Its appeal is that an experiment that would be disruptive, risky or very costly on site becomes cheap and repeatable on screen. A hospital can test a third triage nurse without a single patient being inconvenienced.

1.1.2 Theoretical Background

Several established topics from computer science, operations research and software engineering went into the project. The ones below were applied directly in the design and the code.

Discrete Event Simulation and continuous simulation. A DES models a system as events that happen at separate instants of simulated time. Nothing changes between two events, so the clock leaps from one event to the next rather than ticking in equal steps. An arriving customer, a machine finishing a job and a packet reaching a router are all events. Continuous simulation instead follows variables that change smoothly and integrates them over small fixed steps.

Since a DES skips the stretches where nothing happens, a whole working day can be simulated in a fraction of a second.

Table 1.1: Comparison of Discrete Event and Continuous Simulation

Aspect	Discrete Event Simulation	Continuous Simulation
Time advance	Jumps to the next scheduled event	Advances in small fixed increments
State change	Instantaneous, at event instants only	Smooth and continuous throughout
Mathematical basis	Probability and queueing theory	Differential and difference equations
Typical application	Queues, production lines, supply chains	Thermal, hydraulic and control systems
Computational cost	Proportional to the number of events	Proportional to the simulated duration
Role in JustCmul8	Primary execution paradigm	Approximated by container-level sampling

Probability distributions. Arrival gaps and service times are drawn from probability distributions so that the model varies the way real life does. We implemented five samplers, and each one has its own use.

Table 1.2: Probability Distributions Implemented in JustCmul8

Distribution	Parameters	Characteristics	Typical use
Exponential	mean ($1/\lambda$)	Memoryless; short intervals frequent, long ones rare	Inter-arrival times of independent arrivals
Uniform	min, max (mean $\pm$ 50%)	All values in the range equally likely	A duration known only to lie between two bounds
Normal	mean, std. deviation	Symmetric about the mean; truncated at zero	Machine cycle times with small random error
Poisson	λ (mean rate)	Arrival counts in a fixed interval; exponential gaps	Arrivals per period
Deterministic	fixed value	No variability; repeats exactly	Conveyor transfers and scheduled operations

Queueing theory. Simple queues have exact formulas, which gives a simulator something independent to be checked against. In an M/M/1 queue with arrival rate λ and service rate μ , utilisation is $\rho = \lambda/\mu$, the mean wait in the queue is $W_q = \rho/(\mu - \lambda)$ and the mean number waiting is $L_q = \rho^2/(1 - \rho)$. Multi-server queues use the Erlang-C formula for the same quantities. **Little's Law**, $L = \lambda W$, ties the average number of entities in a system to their arrival rate and their average time inside, and it holds for nearly every stable queue. We use these results twice:

to verify the engine during development, and at run time to test whether each simulation is self-consistent.

Monte Carlo estimation. One stochastic run is a single draw from a spread of possible outcomes. Monte Carlo analysis runs the model many times on independent random streams and treats the results as a sample. The 95 per cent confidence interval of a metric is the sample mean plus or minus 1.96 standard errors, where the standard error is the sample standard deviation divided by the square root of the number of replications.

Supporting concepts. The Software Development Life Cycle (SDLC) split the work into analysis, design, implementation, testing and deployment. A multi-tier architecture keeps presentation, state, domain logic, execution and storage apart so each can be tested alone. A simulation model is a directed graph, so graph traversal is what checks that a model is valid. Primary and foreign keys, referential integrity and row-level security from relational database theory protect user data. Human–computer interaction ideas, namely plain labels, little mental load and familiar drag-and-drop, keep the tool usable for people who have never programmed.

1.1.3 Motivation

Three things pushed us to build this. Commercial simulators cost too much for students, small businesses and most colleges, who are the very people that would gain from a cheap way of testing decisions. The best free engine, SimPy, is a code library, so someone who understands queues but not generator functions cannot use it. And browsers now run WebAssembly at close to native speed, so for the first time a real Python simulation engine can run on the user’s own device with no server and no install.

Together these gave us the brief: a free platform that opens in any browser, uses the words of the user’s own field rather than simulation jargon, and has a verifiable engine underneath. The AI assistant and live collaboration follow from how people now expect to work, which is to describe what they want in ordinary language and share a workspace with colleagues.

1.2 Objective and Scope of the Project

1.2.1 Objective

The main aim of JustCmul8 is to build and publish a free, no-code DES platform on which students, researchers, start-ups and companies can create, run, analyse and share simulation models inside a web browser, without installing software, learning a programming language or buying a licence.

Users get a drag-and-drop modelling environment that sits on top of a real Python SimPy engine running through WebAssembly, so the results are exact while the experience is that of an ordinary web application. An AI assistant turns a description written in English into an executable model, interactive dashboards present the results, and a digital-twin animation shows the system at work.

The specific objectives are these.

- **Remove cost and installation barriers:** no licence fee, and nothing to install apart from a modern browser.
- **Keep the simulation accurate:** run a real Python SimPy engine instead of a visual imitation, and verify it against analytical queueing theory.
- **Remove the coding prerequisite:** offer a visual interface that a person who has never programmed can use.
- **Quantify uncertainty:** give confidence intervals from Monte Carlo replications, not one deterministic number.
- **Point to what to do next:** find the bottleneck automatically, check results with Little's Law, score overall system health and suggest specific changes together with their estimated financial effect.
- **Support collaboration and sharing:** live presence and cursors in a shared workspace, plus read-only share links that can be revoked.
- **Allow for growth:** organise the system so new domains, node types and engines can be added through a registry without redesigning the core.

1.2.2 Scope of the Project

JustCmul8 covers the design and development of one browser-based application in which users build, run, analyse and share DES models. The finished system has twelve functional modules in four groups, which between them span the whole workflow from creating an account to exporting results.

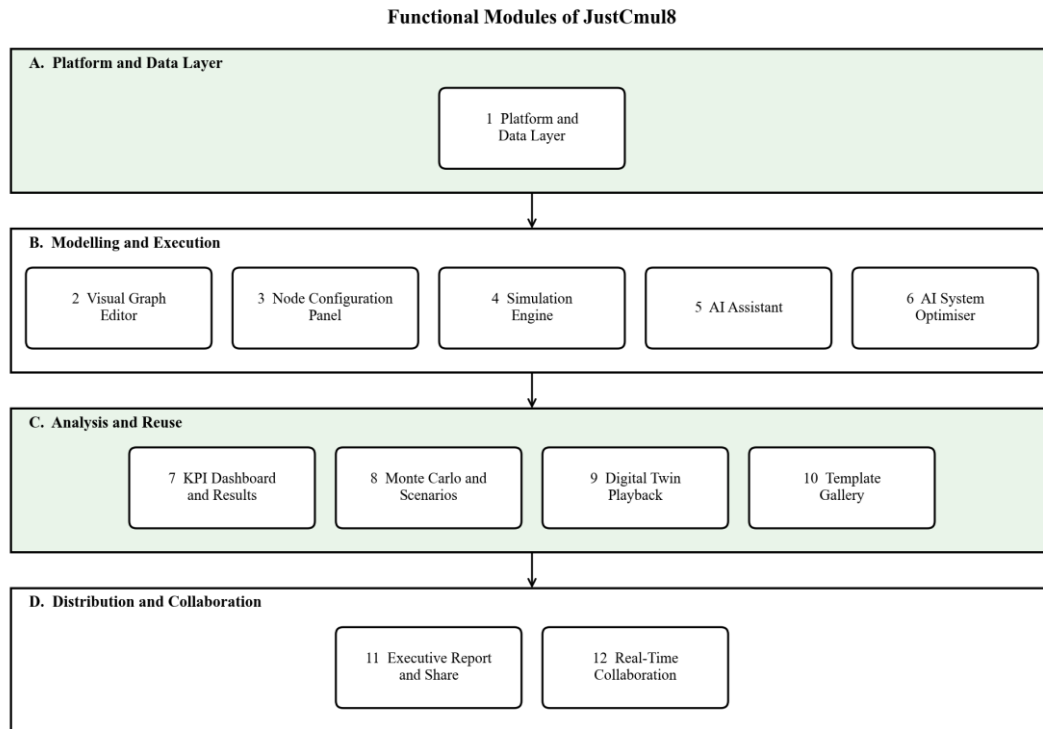


Figure 1.1: Functional Modules of JustCmul8 grouped by area

- **Platform and data layer:** registration and login by email or Google/GitHub OAuth, session handling, creating, renaming and deleting projects, automatic cloud saving, and profile and preference management (Module 1).
- **Modelling and execution:** the visual graph editor, the node configuration panel, the SimPy engine hosted in Pyodide, the AI assistant that builds models from natural language, and the AI optimiser that diagnoses a finished run and proposes a fix (Modules 2 to 6).
- **Analysis and reuse:** the KPI dashboard with bottleneck detection and cost analysis, the Monte Carlo and scenario comparison facility, the animated digital-twin playback and the template gallery (Modules 7 to 10).
- **Distribution and collaboration:** graph export as JSON, results as CSV, executive PDF reports, Jupyter notebook export, read-only public share links, and real-time presence with collaborative editing (Modules 11 and 12).

Six application domains are supported: human queue systems, vehicle traffic, liquid and material flow, manufacturing processes, logistics and warehousing, and network and signal flow. One simulation engine drives all six, and they share a library of fifteen configurable node types that handles a broad range of real scenarios.

1.2.3 Out of Scope

A few capabilities were left out of this release on purpose, to keep the project achievable within the academic programme. We list them so that a decision is not mistaken for an oversight. Several return in the roadmap of Chapter 10.

- **Native mobile applications:** the interface is responsive and works in mobile browsers, but there is no separate Android or iOS app.
- **Enterprise services:** single sign-on, LDAP integration, organisation-level administration and subscription billing.
- **Learning-management features:** the platform is a simulation tool, not courseware.
- **Offline and self-hosted deployment:** the application is delivered as a cloud-hosted web application.
- **Industrial hardware integration:** no direct link to IoT devices, PLCs, sensors or robots.
- **Automated optimisation:** the AI optimiser suggests a change for a person to approve; it does not search the parameter space by itself.
- **Desktop distribution:** there is no packaged Windows, macOS or Linux build, in keeping with the browser-first approach.

1.3 Problem Definition

DES is well known for improving manufacturing, healthcare, logistics, transport and service operations, yet outside large organisations few people use it. The causes are practical, not theoretical, and fall into four groups.

1.3.1 The Cost of Commercial Simulation Software

AnyLogic, Arena and Simul8 are licensed at thousands of dollars per seat per year. A department teaching operations research to sixty students, or a restaurant owner asking whether a second service counter would pay for itself, may find that the licence costs more than the decision is worth. The result is that simulation is available to organisations that could easily afford a wrong guess and out of reach for those that could not.

1.3.2 The Technical Complexity of Free Alternatives

Free frameworks exist, and SimPy is the leading one, but they are programming libraries and not finished applications. A user has to install Python, manage a virtual environment, learn generator functions and the yield statement, work out how resource requests are written and

debug the errors that follow. Simulation ideas and Python idioms must be picked up at the same time, which is a lot to ask of someone whose real question is whether to open another counter.

1.3.3 The Limits of Desktop Distribution

Most simulation software must be installed, configured and maintained. In a shared college laboratory, on a company-managed laptop or on a Chromebook, people often have no right to install anything. A tool that cannot be opened is useless however good its analysis, and this problem exists regardless of price or skill.

1.3.4 The Inadequacy of Static Diagrams and Spreadsheets

People often try two substitutes, and both fail for the same reason. Visio and Lucidchart draw clear process maps, but the maps are inert. A box labelled “Inspection: 5 min” has no behaviour attached, so the diagram cannot show that inspection will have thirty items waiting by mid-afternoon. A spreadsheet does calculate, but it has no natural way to represent an event queue, entities competing for a resource or a server that is still busy when the next arrival appears. Neither captures contention, and contention is what makes queues form.

1.3.5 Statement of the Problem

Put together, these barriers keep a technique that could cheaply answer everyday operational questions within organisations that have both a licence budget and trained staff. What is needed is a platform that is free, needs no installation, asks for no programming, opens from any modern browser and, above all, does not give up the accuracy of its engine for the sake of being accessible. JustCmul8 was built to meet that need.

1.4 Limitations

Every design involves trade-offs. The limits of the current release are listed below. Each follows from an architectural choice we made deliberately, and each has a matching entry in the roadmap of Section 10.3.

- **Client-side resources bound model size:** the simulation runs on the user’s own device, so the memory the browser can spare caps the size of a model and the length of a run. It is a memory limit, not a speed limit.
- **Monte Carlo replications run sequentially:** they execute one after another in a single worker, so elapsed time grows in proportion to the replication count even though the runs are independent.

- **First-run runtime download:** the Pyodide runtime is fetched on first use and cached afterwards, so the first simulation in a session starts more slowly.
- **Connectivity is needed for cloud features:** saving, collaboration and AI help all need the network. A model already loaded can still run offline, but nothing is stored.
- **Collaboration is project-scoped:** presence and graph operations travel over a private channel authorised by the project's row-level security. Role-based invitations with editor and viewer permissions are on the roadmap.
- **AI output needs human judgement:** generated graphs and recommendations are validated and shown for approval, but there is no guarantee that they model the described process faithfully.
- **One simulation paradigm:** the engine is SimPy-based, and continuous or agent-based behaviour is approximated (the container node, for example, samples its level at event boundaries).
- **No native mobile client:** the interface is responsive, but placing nodes precisely on a small touchscreen is still awkward.

2. REQUIREMENT ANALYSIS

This chapter sets down what the system has to do and for whom. It begins with the Software Requirements Specification, names the kinds of user, and lists the functional and non-functional requirements one by one, each with an identifier that later chapters refer back to. It then looks at feasibility, describes the hardware and software used, and presents the proposed system with its advantages.

2.1 User Requirements / SRS / Functional Requirements

2.1.1 Purpose and Overall Description

JustCmul8 is a simulation platform reached through an ordinary browser on a desktop, laptop, tablet or phone. It has four tiers: a user-interface layer, an application and domain-logic layer, a client-side execution layer and a managed database layer. Nothing has to be installed beyond the browser. Most of the weight sits on the client, which holds the interface, the application state and all of the simulation work. The server only stores data, handles authentication, relays real-time messages and forwards requests to the AI service.

2.1.2 User Classes and Characteristics

Table 2.1: User Classes and Access Levels

User class	Responsibilities	Access level	Skill required
Registered User (Project Owner)	Creates projects, builds and configures models, runs simulations, uses AI, exports and shares results	Full control of owned projects	None (no-code)
Collaborator	Joins a project's live workspace to view presence, cursors and shared edits	Project-scoped, authorised by row-level security	None
Guest / Public Viewer	Opens a public share link to view a model read-only	Read only; no account required	None

None of the classes needs programming skill, which follows from the no-code goal. The guest role is read-only so that a model can be passed around for comments without any risk of accidental edits.

2.1.3 Operating Environment

- **Client:** a modern browser (Google Chrome, Mozilla Firefox, Microsoft Edge or Safari) with WebAssembly support, on Windows, macOS, Linux, Android or iOS. The interface adapts to desktops, tablets and phones.
- **Server:** Supabase supplies PostgreSQL, authentication, real-time channels and storage, and a Next.js server runtime hosts the pages and a small set of API routes.
- **Simulation:** SimPy runs inside the browser through Pyodide in a Web Worker, so the server has very little computing to do.

2.1.4 Design and Implementation Constraints

- Everything had to run in the browser, so that no client-side installation is needed.
- Row-level security in the database had to decide who sees what, so a user reaches only their own projects or those explicitly shared with them.
- The interface had to stay simple and quick for people with little or no programming background.
- The architecture had to be modular and registry-driven, letting new domains and node types be added without redesigning the core.
- Secrets, above all the AI service key, had to stay on the server and out of the client bundle.

2.1.5 Functional Requirements

The functional requirements are grouped by module. Their identifiers are reused in the test cases of Chapter 7.

Table 2.2: Functional Requirements

ID	Module	Requirement
FR-01	Platform	Users register and log in with email and password or through Google or GitHub OAuth, and can log out securely.
FR-02	Platform	A signed-in session carries over between visits, and protected routes are checked before any project data is fetched.
FR-03	Platform	A user who forgets a password receives an emailed link and can choose a new one.
FR-04	Platform	Signed-in users can create, open, rename and delete simulation projects, picking one of six domains for each.
FR-05	Platform	Project graph state is saved to cloud storage automatically, with no save button to press.

ID	Module	Requirement
FR-06	Platform	A dashboard lists the user's projects with their domain and last-modified time; profile, preferences and account management, including deleting the account, are also available.
FR-07	Graph Editor	The canvas supports drag-and-drop of fifteen node types, including placing, moving, connecting and deleting them.
FR-08	Graph Editor	Edges follow nodes as they are moved, and the canvas can be zoomed, panned and fitted to view.
FR-09	Graph Editor	Before a run, the graph structure is validated; if it cannot run, the message names the offending node.
FR-10	Node Configuration	A side panel edits the selected node's parameters (arrival rate, capacity, service-time distribution, routing and costs), showing only the fields that suit that node type.
FR-11	Node Configuration	Parameter values are checked against the node type before the engine uses them.
FR-12	Simulation Engine	The graph is compiled to a Python SimPy script and executed on the client through Pyodide; the server does no simulation work.
FR-13	Simulation Engine	Before a run the user sets the simulated duration and its unit: seconds, minutes, hours or days.
FR-14	Simulation Engine	A run executes inside a Web Worker, which keeps the interface responsive.
FR-15	Simulation Engine	While a run is in progress the user sees its progress and status and a stream of log events, can stop it, and can change playback speed between 0.5× and 10×.
FR-16	AI Assistant	A scenario described in natural language is converted into a candidate graph of nodes and edges.
FR-17	AI Assistant	A generated graph is validated first; the user can preview it and apply it. Chat history is kept for each project, and if one AI model is unavailable another is tried automatically.
FR-18	AI Optimiser	After a run finishes, the optimiser diagnoses the bottleneck and suggests a specific correction that can be applied in one action.
FR-19	KPI Dashboard	When a run completes, each node gets its utilisation, average and percentile waiting time, throughput and queue length.
FR-20	KPI Dashboard	The dashboard marks the bottleneck node, checks results with Little's Law and shows a composite system health score from 0 to 100.
FR-21	KPI Dashboard	A Cost and ROI analysis is built from resource, holding and waiting-penalty cost parameters.
FR-22	Monte Carlo	A model can be run over 10, 20, 50 or 100 replications, with 95% confidence intervals reported and a baseline compared against a modified scenario.
FR-23	Digital Twin	Entities are animated in a two-dimensional playback with play, pause, step and speed controls.
FR-24	Template Gallery	Starter templates cover all six domains, and each one creates a separate project owned by whoever requested it.

ID	Module	Requirement
FR-25	Export and Share	The graph exports as JSON and results as CSV, the generated Python script as a Jupyter notebook and an executive report as PDF.
FR-26	Export and Share	A read-only public share link can be created and revoked. It never gives edit access, and a revoked or invalid link shows a clear notice.
FR-27	Collaboration	Users in the same project see each other's presence and cursors, and graph operations are broadcast over an authenticated private channel.

2.1.6 Non-Functional Requirements

Table 2.3: Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	Dragging and connecting nodes on the canvas should feel immediate, and the dashboard should show a completed run without a noticeable delay.
NFR-02	Responsiveness	A running simulation must never freeze the interface, so runs go to a background worker thread.
NFR-03	Correctness	For reference queues, simulated utilisation, waiting time and queue length should match analytical queueing theory within a stated tolerance.
NFR-04	Security	The authentication provider keeps passwords only as salted hashes, all traffic uses TLS, and secret keys never reach the client.
NFR-05	Access Control	Row-Level Security applies to every table, so a modified client cannot get around authorisation.
NFR-06	Usability	A first-time, non-technical user should be able to operate the interface, which prefers domain vocabulary to simulation jargon.
NFR-07	Reliability	Project state persists automatically between sessions and across devices.
NFR-08	Portability	The application runs unmodified on current WebAssembly-capable browsers on Windows, macOS, Linux, Android and iOS.
NFR-09	Maintainability	Simulation logic stays behind the worker boundary, and domain and node definitions live in one registry.
NFR-10	Data Integrity	The database enforces foreign-key constraints with cascading deletion.
NFR-11	Availability	The hosted application aims for 99 per cent availability across the front end and the managed backend.

2.1.7 User Personas and Operational Workflow

We designed the interface around three personas. Each takes a different route through the system.

- **The student** meets queueing theory in a lecture and wants to watch M/M/1 behaviour instead of deriving it. This person opens a template, changes the arrival rate, runs again

and compares dashboards. What matters here is that a first useful result appears within a minute of arriving.

- **The lecturer** demonstrates congestion live in class. The animated playback helps the explanation, and the share link lets students open the same model later on their own devices without accounts.
- **The small-business owner** is weighing up a second service counter. This person describes the shop to the AI assistant in plain English, accepts the generated graph, runs a Monte Carlo analysis and reads the confidence intervals and the cost panel before spending anything.

The workflow common to all three is: sign in, create or open a project, build the graph by hand or with the assistant, set node parameters, validate the structure, run, read the analytics, optionally run again under a changed scenario, and finally export or share the result. Chapter 5 draws this path as an activity diagram.

2.2 Feasibility Study

Before the design was fixed, we examined the project along five dimensions. Technical feasibility mattered most, because the whole architecture hinged on it, so it comes first.

2.2.1 Technical Feasibility

Next.js, React, TypeScript, Supabase, React Flow, Pyodide and SimPy are widely used, well documented and backed by large communities. That lowers development risk and eases long-term maintenance.

The big question was whether a Python discrete event simulation could run properly inside a browser. Pyodide answers it by compiling the CPython interpreter to WebAssembly, so SimPy runs on the client with nothing installed. We proved this early with a prototype spike, since everything else depended on it. Two constraints came out of that spike, and we designed around them. The Pyodide runtime is a download of several megabytes, so it is fetched once and cached. WebAssembly also works inside the browser's memory allowance, which sets a practical ceiling on model size; running the engine in a Web Worker keeps the interface usable while a large model executes.

Canvas rendering was a smaller but real concern, because hundreds of interactive nodes in the DOM can slow down sharply. We used React Flow, which is made for node-based interfaces,

for the editor, and Pixi.js, which draws through WebGL, for the animated playback where many entities move together.

2.2.2 Economic Feasibility

The project is affordable because almost everything in it is open source or sits on cloud services with generous free tiers. React, Next.js, TypeScript, SimPy, Pyodide, React Flow, Zustand, Recharts and Pixi.js all carry no licence fee, unlike the commercial packages of Section 1.3.

Running simulations on the client has a direct financial effect. The user's browser does the computing, so the server cost does not rise with the number or size of simulations. A thousand users running hour-long simulations put no more load on the server than a thousand people reading a static page. The only recurring costs are the managed database tier and the metered AI quota, and the latter stays small because an AI request is sent only when the user asks for generation or diagnosis.

Table 2.4: Indicative Cost Comparison with Commercial Alternatives

Item	Commercial DES tool	JustCmul8
Software licence	Typically thousands of USD per seat per year	None; open-source stack
Installation and IT effort	Required on every machine	None; runs in the browser
Simulation compute	Local workstation or licensed server	User's browser; no server cost
Backend hosting	Not applicable or self-managed	Managed free tier during the academic phase
AI assistance	Generally unavailable	Gemini API, metered per explicit request
Training	Formal instruction usually necessary	Not required; no-code interface

The commercial figures are only indicative, since vendor price lists change often. The point of the table is the qualitative difference.

2.2.3 Operational Feasibility

Adopting the platform is easy. With the drag-and-drop interface and the AI assistant, someone with no programming or simulation background can build a model, and because it all happens in a browser there is nothing to install or maintain. Our intended users (students, teachers, researchers and small-business owners) already know modern web applications, so formal training and a technical support desk are not needed. Some interface choices help further.

Palettes use the vocabulary of the chosen domain, so someone modelling a bank sees “Teller” instead of “Resource”, and result tabs have plain names such as “Line Buildup Timeline” and “Station Counters”.

2.2.4 Schedule Feasibility

The development plan fits the academic timeframe. The work is cut into modules ordered by their dependencies, with core features first and extras such as AI assistance and collaboration after. Testing, debugging and integration go on beside development, not after it. The scheduling decision with the biggest effect was to start the simulation engine in the second week of July, alongside the backend work, so that the hardest technical uncertainty was settled while there was still time to react to it. Chapter 3 gives the full twenty-four-week schedule.

2.2.5 Legal and Ethical Feasibility

All dependencies are used under open-source licences or within the terms of free service tiers, so no licensing obligation is outstanding. PostgreSQL Row-Level Security confines each user to their own projects, and passwords are never kept in a form that can be recovered. Deleting an account really does cascade through and remove the user’s data. A public share link can be revoked whenever the owner likes, after which the address shows an explanation instead of the project. AI-generated graphs are always put in front of a person before they change a project, so the assistant advises and never acts on its own.

2.3 Details of Hardware and Software Used

Two configurations matter here, and they are easy to mix up. One is the machine on which we developed the platform, which is fairly demanding because an editor, a development server and the browser runtime all run together. The other is what an end user needs, which is much less, and that second one decides how widely the platform can be adopted.

2.3.1 Development Environment

Table 2.5: Development Hardware and Software Configuration

Component	Specification	Purpose
Processor	Intel Core i5 / AMD Ryzen 5 or higher	Runs the development server, browser and Pyodide runtime concurrently
Memory	8 – 16 GB RAM	Editor, browser with several tabs and local build processes together

Component	Specification	Purpose
Storage	256 GB SSD or larger	Sources, Node.js dependency tree and build artefacts
Display	1920 × 1080 or higher	Canvas, configuration panel and dashboard side by side
Operating system	Windows 11	Primary development platform
Network	Stable broadband connection	Package installation, Supabase and AI API access
Editor	Visual Studio Code with TypeScript and ESLint	Primary development environment
Version control	Git with a GitHub remote	Source history, branching and collaborative development
API testing	Postman	Verification of API routes
Runtime	Node.js LTS with npm	Dependency management and the Next.js build pipeline
Verification	Python 3 with SimPy	Native harness used to verify the engine against queueing theory
Browser tooling	Chrome DevTools	Profiling, memory inspection and performance measurement

2.3.2 Target Client Requirements

A web application needs no special hardware on the user's side. A reasonably recent desktop, laptop, tablet or smartphone with a modern browser will do. One caveat comes from the architecture: because the engine runs on the client, a device with little memory will handle very large models more slowly, and the Web Worker path is there to keep the interface usable when that happens.

Table 2.6: Minimum and Recommended Client Requirements

Requirement	Minimum	Recommended
Browser	Current Chrome, Edge, Firefox or Safari with WebAssembly	Latest evergreen release of Chrome or Edge
Memory	4 GB RAM	8 GB RAM or more for large models
Processor	Dual-core	Quad-core or better
Display	1366 × 768	1920 × 1080 or higher
Network	Stable connection for loading and persistence	Broadband for real-time collaboration
Local installation	None	None

2.3.3 Software and Technology Stack

Table 2.7: Technology Stack with Justification

Layer	Technology	Reason for selection
Frontend framework	Next.js 16 (App Router)	File-based routing, server rendering and API routes in one framework
UI library	React 19	Component model suited to a complex, highly interactive editor
Language	TypeScript	Static typing across the code base, catching parameter mismatches at compile time
Styling	Tailwind CSS v4	Utility-first styling giving consistent spacing and a coherent theme
Graph editor	React Flow (@xyflow/react)	Production-grade node-based canvas with handles and edge routing
Graph layout	Dagre	Automatic layered layout for generated and template graphs
Simulation engine	Python SimPy via Pyodide (WebAssembly)	A real DES engine executing client-side
State management	Zustand	Lightweight store with slice-based separation
Charts	Recharts	Declarative, composable charts for the KPI dashboard
2D animation	Pixi.js	WebGL rendering for the digital-twin playback
AI service	Google Gemini (via a model router)	Structured generation of graphs and diagnoses from natural language
Database and auth	Supabase (PostgreSQL, Auth, Realtime)	Managed Postgres with Row-Level Security, OAuth and WebSocket channels
Reporting	jsPDF, html-to-image	Client-side generation of executive PDF reports
Animation and scroll	Framer Motion, Lenis	Transition choreography and smooth scrolling on the landing page
Icons	Lucide React	Consistent icon set covering all node types

2.4 Proposed System and Advantages of the Proposed System

2.4.1 The Proposed System

The system we propose is JustCmul8, a discrete event simulation platform that lives in the browser. The user drags nodes onto a canvas, fills in their parameters in forms, presses **Run** and reads the answer in plain language. Behind the button, the graph is translated into a genuine SimPy program that CPython, compiled to WebAssembly, executes in a Web Worker. Its output passes through an analytics layer that computes performance indicators, verifies them

and ranks the constraint, and then produces recommendations, cost figures and an animated replay.

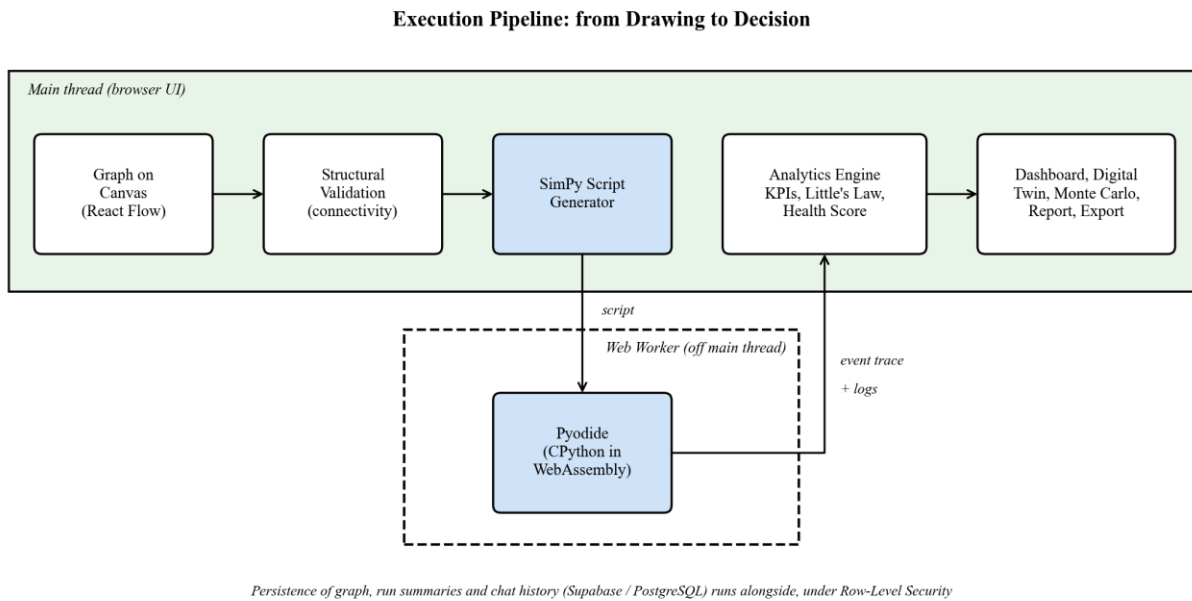


Figure 2.1: Execution Pipeline of the Proposed System

2.4.2 Comparison with Existing Approaches

Table 2.8: Comparison of the Proposed System with Existing Approaches

Capability	Commercial DES tools	SimPy library	Diagram tools / spreadsheets	JustCmul8
Free to use	No	Yes	Partly	Yes
No installation	No	No	Yes	Yes
No programming needed	Partly	No	Yes	Yes
Executes real DES	Yes	Yes	No	Yes (SimPy)
Runs in the browser	Rarely	No	Yes	Yes
AI-assisted modelling	Rarely	No	No	Yes
Confidence intervals (Monte Carlo)	Yes	Manual coding	No	Yes, one click
Live collaboration and sharing	Limited	No	Yes	Yes
Readable generated code	No	Yes	No	Yes (inspector and notebook)

2.4.3 Advantages of the Proposed System

- **No cost, no installation:** it is free, and a URL is all that is needed from any laptop, lab machine or Chromebook.
- **Trustworthy results:** the numbers come from SimPy and not from an imitation, and the engine has been checked against queueing theory.
- **No-code modelling:** domain vocabulary, form-based configuration and a template gallery get a first-time user to a result within a minute.
- **AI-assisted workflow:** a plain-English description becomes a runnable model, and a finished run becomes a specific, costed recommendation.
- **Uncertainty shown with the answer:** Monte Carlo confidence intervals reveal whether an apparent improvement is bigger than the noise.
- **Transparent and repeatable:** the generated Python can be read in the code inspector and exported as a Jupyter notebook, so anyone can re-run a result independently.
- **Output that is easy to read:** bottleneck flags, a system health score, a flow-balance check, a traffic heatmap, an entity tracer and an animated digital twin turn statistics into pictures.
- **Security built in:** Row-Level Security in the database decides access, and share links are opaque, read-only and revocable.
- **Cheap to scale:** computation happens on the client, so hosting cost hardly depends on how heavily the platform is used.
- **Easy to extend:** one registry defines each domain's vocabulary, palette, layout and templates, so new domains and node types can be added without touching the engine.

3. PROJECT PLANNING

A big idea only gets built if it is cut into steps that can actually be finished. This chapter shows how we organised the work on JustCmul8: the activities, the time given to each, the schedule in graphical form, the resources we needed and the development model we followed.

3.1 List of Activities / Tasks

We arranged the work along the usual phases of the Software Development Life Cycle: initiation and analysis, design, development, testing, documentation and deployment. It was split into twenty-seven activities, each with an estimated duration and a recorded dependency, so nothing could start before the work it relied on was done. The full schedule covers twenty-four working weeks, from the first week of July 2026 to the last week of December 2026.

Mapping dependencies did two jobs. It stopped us from building a module before its prerequisites existed, and it showed which activities were independent and could run side by side, which shortened the total time without cutting quality. The most important call concerned the **Simulation Engine**, which starts in the second week of July and runs alongside backend development. Running Python SimPy in a browser through Pyodide and WebAssembly was the biggest technical risk in the project, and the AI assistant, KPI dashboard, Monte Carlo analysis and digital twin all assume it works. An early start left time to fix compatibility problems, check accuracy against known analytical results and settle the interface that the other modules would call.

A word on module numbering is needed. The schedule below follows the eleven-module breakdown from the project synopsis, which was the plan when we began: Authentication, Project Management, Visual Graph Editor, Node Configuration Panel, Simulation Engine, AI Assistant, KPI Dashboard, Template Gallery, Export and Share, User Profile and Settings, and Real-Time Collaboration. While building, we reorganised the delivered system into **twelve modules** (Chapter 4). Authentication, Project Management and Profile were merged into one Platform and Data Layer, and three features that outgrew their first scope, namely the AI System Optimiser, Monte Carlo and Scenario Comparison, and the Digital Twin Playback, became modules of their own. The activity list therefore records the plan as made, and Chapter 4 records the system as delivered.

Table 3.1: List of Activities and Tasks with Dependencies

Sr.	Task / Activity	Phase	Duration	Depends on
1	Project Initiation	Initiation / Analysis	2 weeks	None
2	Requirement Gathering	Initiation / Analysis	3 weeks	None
3	Literature Survey	Initiation / Analysis	2 weeks	Project Initiation
4	System Analysis	Initiation / Analysis	2 weeks	Requirement Gathering
5	UI/UX Design	Design	3 weeks	System Analysis
6	Database Design	Design	3 weeks	System Analysis
7	Module: Authentication	Development	3 weeks	Database Design
8	Module: Project Management	Development	3 weeks	Authentication
9	API Development	Development	5 weeks	Database Design
10	Backend Development	Development	8 weeks	API Development
11	Module: Visual Graph Editor	Development	4 weeks	UI/UX Design, Project Management
12	Module: Node Configuration Panel	Development	3 weeks	Visual Graph Editor
13	Frontend Development	Development	9 weeks	UI/UX Design
14	Module: Simulation Engine	Development	8 weeks	Early parallel track; runs ahead of backend
15	Module: AI Assistant	Development	4 weeks	Simulation Engine
16	Integration	Development	5 weeks	Modules 1 – 6 of the plan
17	Module: KPI Dashboard	Development	3 weeks	Simulation Engine
18	Module: Template Gallery	Development	3 weeks	Visual Graph Editor
19	Module: Export and Share	Development	3 weeks	KPI Dashboard
20	Module: User Profile and Settings	Development	3 weeks	Authentication
21	Module: Real-Time Collaboration	Development	4 weeks	Visual Graph Editor
22	Unit Testing	Testing	5 weeks	Modules in progress
23	Integration Testing	Testing	3 weeks	Integration complete
24	Bug Fixing	Testing	3 weeks	Integration Testing
25	User Acceptance Testing (UAT)	Testing	2 weeks	Bug Fixing
26	Documentation	Documentation	23 weeks	Continuous, alongside every module
27	Final Review and Deployment	Deployment	4 weeks	User Acceptance Testing

The phases work out as follows.

- **Initiation and Analysis:** we fixed the objectives, the intended users and the requirements, surveyed the commercial and open-source tools discussed in Section 1.3, and gathered the findings into a specification before writing any code.
- **Design:** interface design gave us the workspace layout, the way the node palette is organised and how the configuration panel behaves; database design gave the schema described in Chapter 5.
- **Development:** this took the biggest share of the schedule and ran as three parallel streams. The backend and API stream covered authentication, project management and the data layer. The simulation stream covered the engine, AI assistant, KPI dashboard and templates. The frontend stream covered the graph editor, configuration panel, export, profile and collaboration.
- **Testing:** unit testing starts while modules are still being built, so defects in early modules get fixed as later ones are written. Integration testing, bug fixing and user acceptance testing follow one after another.
- **Documentation:** written throughout twenty-three of the twenty-four weeks so that the record matches the work.
- **Deployment:** a final review, then release to the hosting platform.

3.2 Timeline for Each Task

The table gives the planned start and finish week of every activity. Weeks are counted within the calendar month, so W1 is the first week of that month.

Table 3.2: Planned Timeline of Each Task (July W1 – December W4, 2026)

Sr.	Task / Activity	Duration	Start	End
1	Project Initiation	2 weeks	Jul W1	Jul W2
2	Requirement Gathering	3 weeks	Jul W1	Jul W3
3	Literature Survey	2 weeks	Jul W2	Jul W3
4	System Analysis	2 weeks	Jul W3	Jul W4
5	UI/UX Design	3 weeks	Jul W4	Aug W2
6	Database Design	3 weeks	Jul W4	Aug W2
7	Module: Authentication	3 weeks	Aug W1	Aug W3
8	Module: Project Management	3 weeks	Aug W2	Aug W4
9	API Development	5 weeks	Aug W1	Sep W1
10	Backend Development	8 weeks	Aug W1	Sep W4
11	Module: Visual Graph Editor	4 weeks	Aug W2	Sep W1

Sr.	Task / Activity	Duration	Start	End
12	Module: Node Configuration Panel	3 weeks	Aug W3	Sep W1
13	Frontend Development	9 weeks	Aug W2	Oct W2
14	Module: Simulation Engine	8 weeks	Jul W2	Sep W1
15	Module: AI Assistant	4 weeks	Sep W2	Oct W1
16	Integration	5 weeks	Sep W3	Oct W3
17	Module: KPI Dashboard	3 weeks	Sep W3	Oct W1
18	Module: Template Gallery	3 weeks	Sep W4	Oct W2
19	Module: Export and Share	3 weeks	Oct W1	Oct W3
20	Module: User Profile and Settings	3 weeks	Oct W1	Oct W3
21	Module: Real-Time Collaboration	4 weeks	Oct W2	Nov W1
22	Unit Testing	5 weeks	Sep W4	Oct W4
23	Integration Testing	3 weeks	Oct W4	Nov W2
24	Bug Fixing	3 weeks	Nov W1	Nov W3
25	User Acceptance Testing (UAT)	2 weeks	Nov W3	Nov W4
26	Documentation	23 weeks	Jul W1	Dec W3
27a	Final Review	2 weeks	Dec W1	Dec W2
27b	Deployment	2 weeks	Dec W3	Dec W4

Project Milestones

Table 3.3: Project Milestones and Completion Criteria

Milestone	Target	Completion criterion
M1: Requirements baselined	Jul W3	Functional and non-functional requirements approved by the guide
M2: Literature survey complete	Jul W3	Comparable tools evaluated and the technology stack fixed
M3: Design frozen	Aug W2	Interface layout and database schema approved
M4: Engine proof of concept	Sep W1	A SimPy model executing in the browser with output verified against theory
M5: Core modelling path working	Sep W1	A graph can be built, configured and executed end to end
M6: Analytics operational	Oct W1	KPI dashboard rendering utilisation, wait time and throughput
M7: Feature complete	Nov W1	All scheduled modules implemented and integrated
M8: Testing signed off	Nov W4	Unit, integration and acceptance testing passed
M9: Deployed	Dec W4	Application live on the hosting platform and report submitted

Milestone M4 matters most. We placed it early because it removes the project’s biggest technical risk, and every later milestone takes it for granted that Python can run in the browser.

3.3 Gantt Chart and Activity Network Diagram

3.3.1 Gantt Chart

A Gantt chart draws each activity as a horizontal bar against a calendar, so its length and its overlap with other activities can be read straight off the page. Software teams use it for scheduling tasks, allocating resources and tracking progress. Its real value is that it breaks a project into manageable pieces, gives each a realistic time span, and exposes scheduling conflicts early enough to fix them.

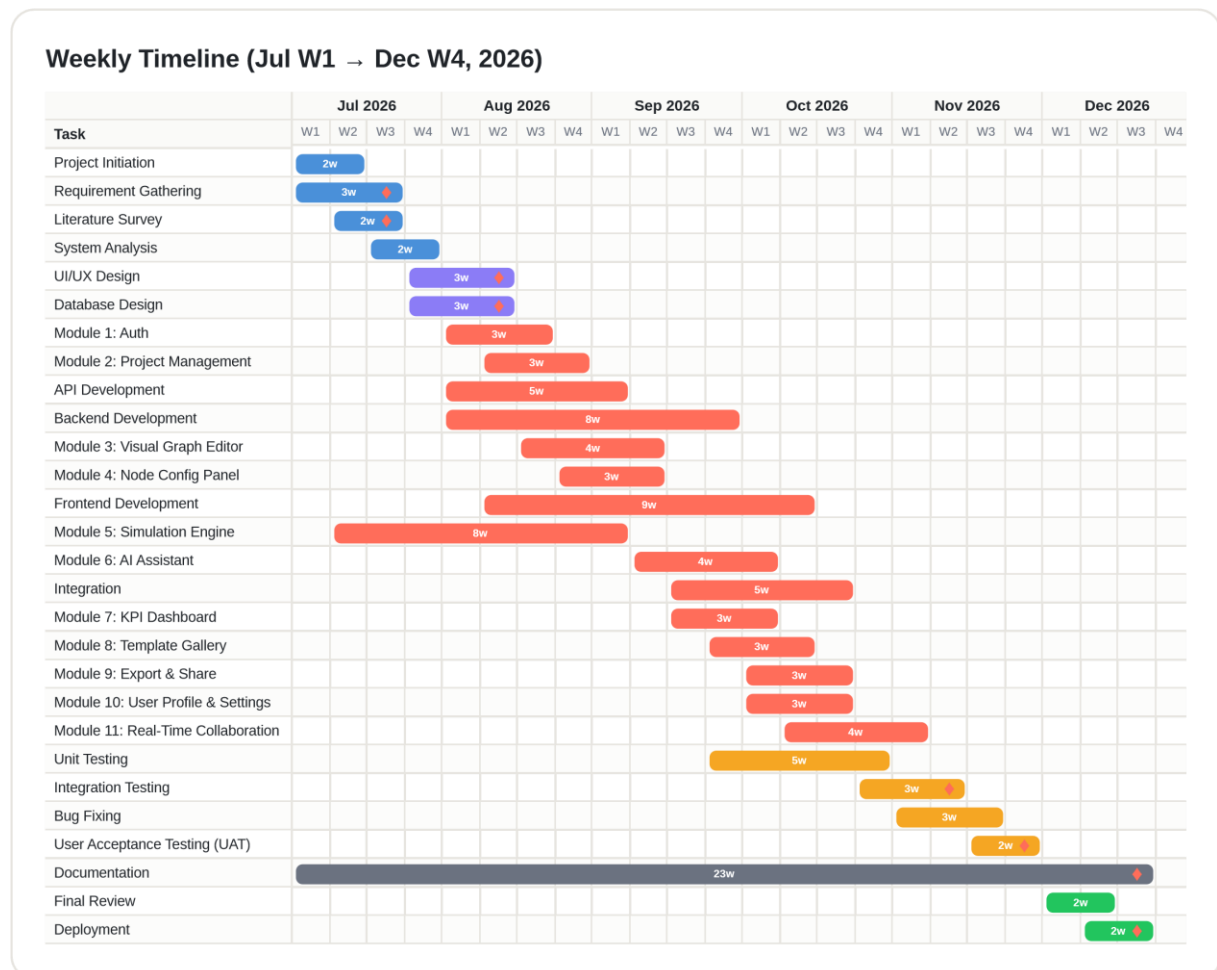


Figure 3.1: Weekly Gantt Chart of the JustCmul8 Development Schedule (July W1 – December W4, 2026)

Bars are grouped by phase, with one shade per stage of the life cycle. The first four bars are initiation and analysis, which fill July and settle the objectives, requirements and technology

choices that the design rests on. The next two bars are interface and database design, spread over about three weeks from late July. Development takes up most of the chart. API and backend work sits beside the two foundation modules, authentication and project management, and the visual graph editor and node configuration panel follow, since together they form the surface the user actually works on.

The simulation engine bar needs the closest look. It starts in the second week of July and overlaps backend development, for the reason given in Section 3.1. The AI assistant comes once the engine has settled, and the other modules are delivered through September and October, each after its prerequisite. Testing bars overlap development bars on purpose, because testing starts before every module is finished. The one long bar is documentation, which covers nearly the whole period, and the last two bars are the final review and deployment in December.

3.3.2 Activity Network Diagram

An activity network diagram is a behavioural model showing how activities flow through a project, with their order, decision points and parallel paths. A Gantt chart tells us *when* work happens; the network tells us *in what logical order*, under which decisions and along which concurrent paths. We drew it in UML notation, using activity nodes, control flows, decision nodes, fork and join bars, and start and end markers.

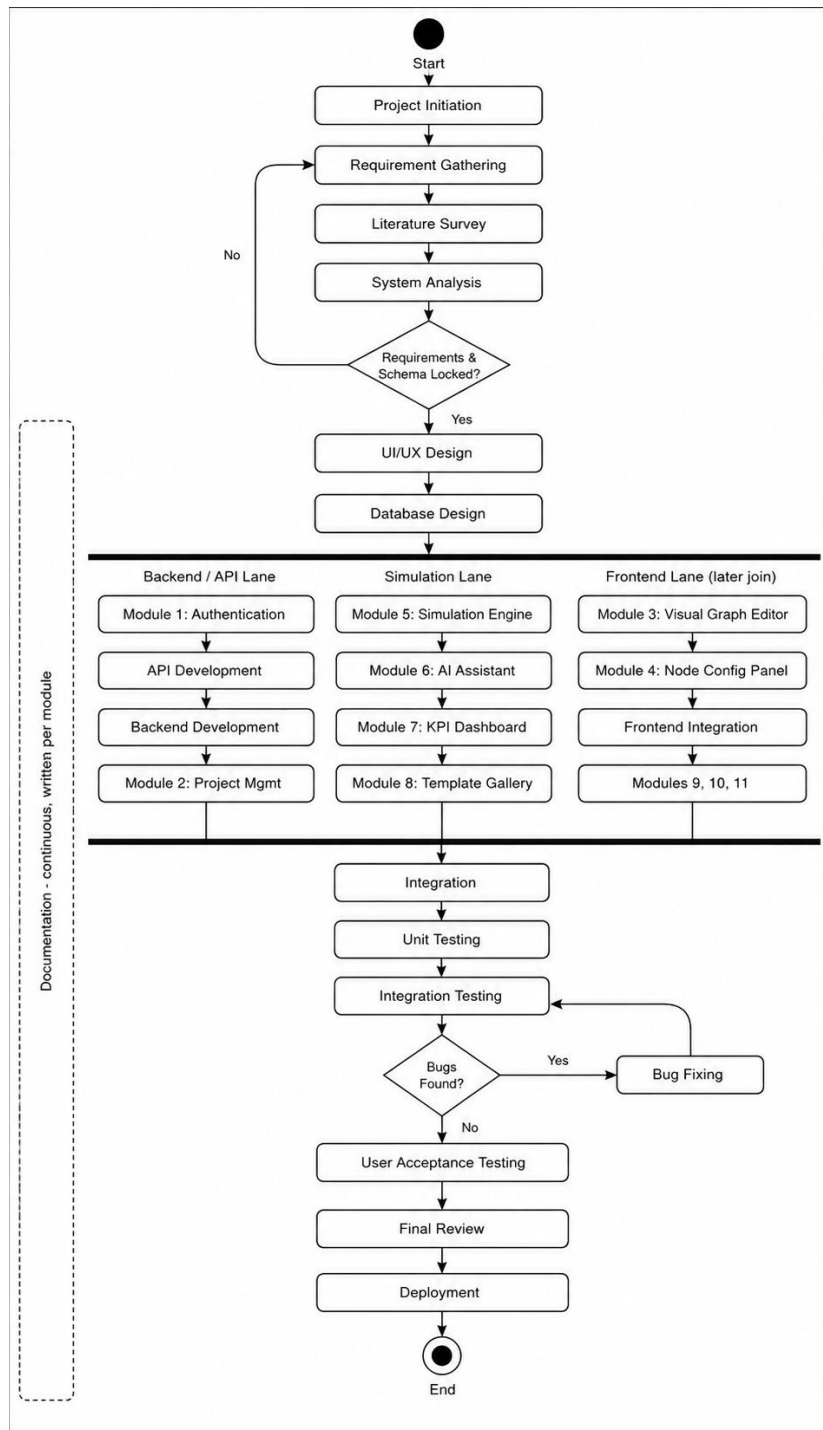


Figure 3.2: Activity Network Diagram of the JustCmul8 Development Lifecycle

The flow opens with project initiation, requirement gathering, literature survey and system analysis. A decision node then asks whether the requirements and the database schema are locked. If not, control goes back to requirement gathering, and design starts only when both are settled. Development is a fork into three parallel lanes. The backend and API lane holds authentication, API development, backend development and project management. The simulation lane holds the simulation engine followed by the AI assistant, KPI dashboard and

template gallery. The frontend lane holds the graph editor, configuration panel, frontend integration, export, profile and collaboration. A join bar brings the three lanes back together for system integration.

Testing starts with unit tests and moves to integration testing, where a second decision node asks whether defects turned up. If they did, control goes to bug fixing and back to integration testing, and this repeats until the build is clean. Only then does the flow reach user acceptance testing, the final review and deployment. Documentation runs beside every phase as a supporting activity.

3.3.3 Critical Path

The critical path is the chain of dependent activities that fixes the earliest possible finish date. Here it goes through Requirement Gathering, System Analysis, Database Design, API and Backend Development, Integration, Integration Testing, Bug Fixing, User Acceptance Testing, Final Review and Deployment. A delay anywhere on the chain pushes back delivery, whereas activities off the chain, such as the Template Gallery or Profile and Settings, have float and can absorb short delays. Had the Simulation Engine been scheduled after the backend it would have been critical too; running it in parallel takes it off the path and protects the final date.

The two diagrams complement each other. Together they show the calendar and the control flow of the development approach we took for JustCmul8.

3.4 Resource Planning

Resource planning lists the people, equipment, software and services needed, and when they are needed. JustCmul8 uses an open-source stack and free-tier cloud services, so most of the planning was about human effort and making sure the time of the two team members went where it counted most.

3.4.1 Human Resources

Table 3.4: Human Resources and Responsibilities

Resource	Role	Responsibilities
Ms. Jinal Gujar	Project Guide	Review of requirements, design and drafts; validation of milestones; academic guidance
Khushi Singh (261713)	Developer, Analyst and Documenter	Requirements and design documents; Platform and Data Layer; configuration panel; AI assistant; KPI dashboard; digital twin; report and share; testing support

Resource	Role	Responsibilities
Mohit Gupta (261715)	Developer, Architect and Tester	System architecture; graph editor; simulation engine; AI optimiser; Monte Carlo; templates; real-time collaboration; verification harness; deployment
Both members	Team	Requirement analysis, database design, integration, testing, user acceptance and the project report

3.4.2 Hardware, Software and Service Resources

Table 3.5: Hardware, Software and Service Resources

Category	Resource	Cost basis
Hardware	Two development laptops (8–16 GB RAM, SSD); college laboratory machines for cross-machine collaboration testing	Owned / provided by the college
Development software	VS Code, Git, GitHub, Node.js, Postman, Python and SimPy, Chrome DevTools	Free
Frameworks and libraries	Next.js, React, TypeScript, Tailwind CSS, React Flow, Zustand, Recharts, Pixi.js, Pyodide	Open source
Backend services	Supabase (database, authentication, realtime)	Free tier
AI service	Google Gemini API	Free tier / metered usage
Hosting	Vercel (Next.js hosting and preview deployments)	Free tier
Project management	GitHub issues and project board	Free tier

3.4.3 Effort Allocation

Two people working for six months give a nominal twelve person-months. The table divides that effort across phases in line with the schedule in Section 3.2.

Table 3.6: Planned Effort Allocation by Phase

Phase	Weeks	Share of effort	Principal activity
Initiation and Analysis	4	10%	Requirements, literature survey, technology assessment
Design	3	10%	Interface design, database design, architecture
Development	18	45%	Twelve modules, API and backend, integration
Testing	9	20%	Unit, integration, security, acceptance testing
Documentation	continuous	10%	Module documents and this report
Deployment	4	5%	Final review, configuration, release

3.5 Proposed Software Development Model

We followed an **iterative and incremental model**, which borrows the discipline of waterfall and the flexibility of agile practice. Two features of this project led us there.

First, the requirements were fairly stable from the start. The problem area is well known, and the tools we surveyed in Section 1.3 already set out the expected features. Pure agile earns its keep when requirements keep emerging during development, so it would have added little, and the formal documentation that an academic project must produce suits the stage discipline of waterfall. Second, the technical risk sat in one component. Whether SimPy could run in a browser through Pyodide was an open question at the start, and nearly everything else relied on the answer. Strict waterfall would have put off finding out until implementation, by which time the design would have been built around an assumption that might be false. So we added an early iterative prototyping loop and treated the engine as a spike to be proved before the modules depending on it were designed in detail.

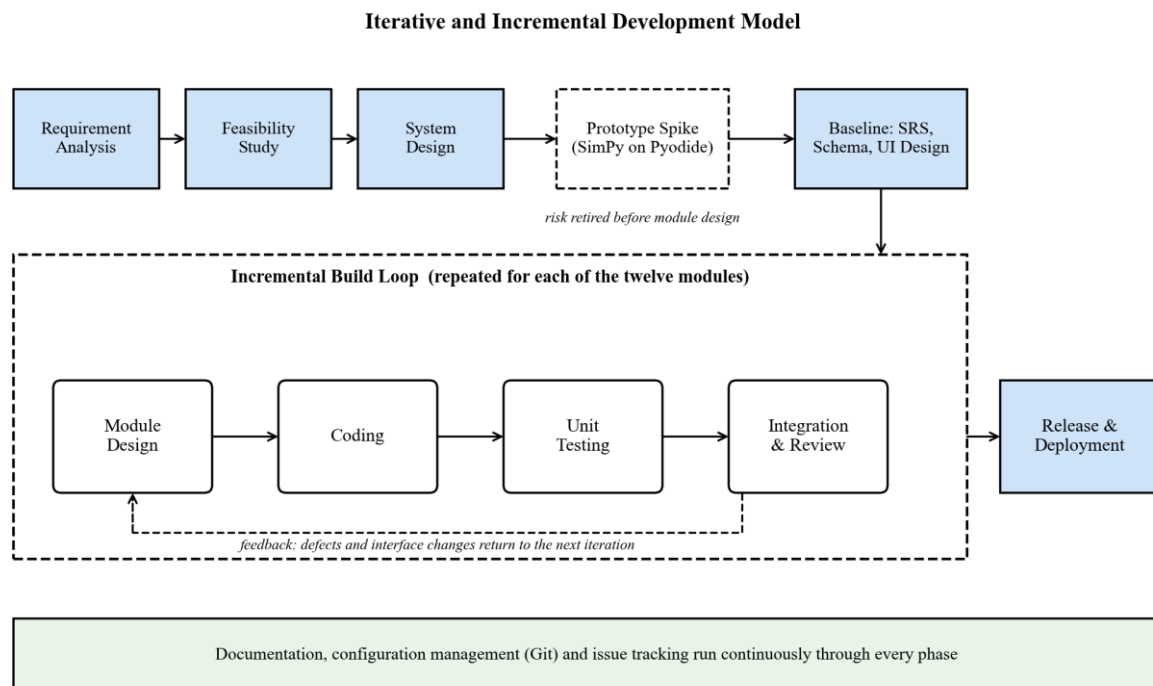


Figure 3.3: Iterative and Incremental Development Model adopted for JustCmul8

In practice, analysis and design went ahead in waterfall fashion, giving us the requirement specification and the schema before implementation. Implementation then went module by module, and each module passed through its own design, coding, testing and integration cycle

before the next began. Testing ran throughout, and documentation was written with each module.

Table 3.7: Phases of the Adopted Life-Cycle Model

Phase	Activities performed	Outcome
Requirement analysis	Elicitation of needs, study of comparable tools, identification of functional and non-functional requirements	Approved requirement specification (Section 2.1)
Feasibility study	Assessment of technical, economic, operational, schedule, legal and ethical viability	Confirmation that the browser-hosted approach was achievable
System design	Architecture, database schema, interface layout and UML modelling	Design documents and diagrams (Chapter 5)
Prototyping (iterative)	Spike implementation of the Pyodide and SimPy execution path, verified against analytical results	Retirement of the dominant technical risk at milestone M4
Implementation (incremental)	Module-by-module construction across three parallel streams, each with its own design-code-test cycle	Twelve delivered modules (Chapter 4)
Testing	Unit, structural, analytical, integration, security and acceptance testing alongside development	Verified system (Chapter 7)
Deployment	Final review, environment configuration, database policies, release	Live application on the hosting platform
Documentation	Continuous recording of analysis, design, implementation and test outcomes	This report and module documents

Table 3.8: Why the Iterative and Incremental Model Was Preferred

Model	Strength	Weakness for this project
Waterfall	Clear stages and documentation	Defers discovery of the engine risk until late implementation
Pure Agile / Scrum	Adapts to changing requirements	Requirements were stable; formal documentation is required
Spiral	Explicit risk analysis in each loop	Heavyweight for a two-person team
Iterative and Incremental (adopted)	Early risk retirement, testable increments, disciplined documentation	Requires interface stability between modules, managed by the worker boundary

One side effect of this model is worth recording. The engine was built before several modules that consume its output, so its result format grew as those modules arrived. Monte Carlo analysis wanted per-run statistics, and the digital twin wanted a per-entity journey trace. Both

additions were cheap because the engine sat behind a worker boundary, which is the very benefit the layered architecture was meant to give.

4. SOFTWARE / HARDWARE MODULES

JustCmul8 is made up of twelve software modules. Each owns one responsibility, offers a defined interface to the others, and was built and tested on its own before integration. The chapter goes through the modules one at a time: what each does, how we implemented it, which requirements of Section 2.1 it meets, and who on the team wrote it. It ends with the hardware points that follow from running simulations on the client.

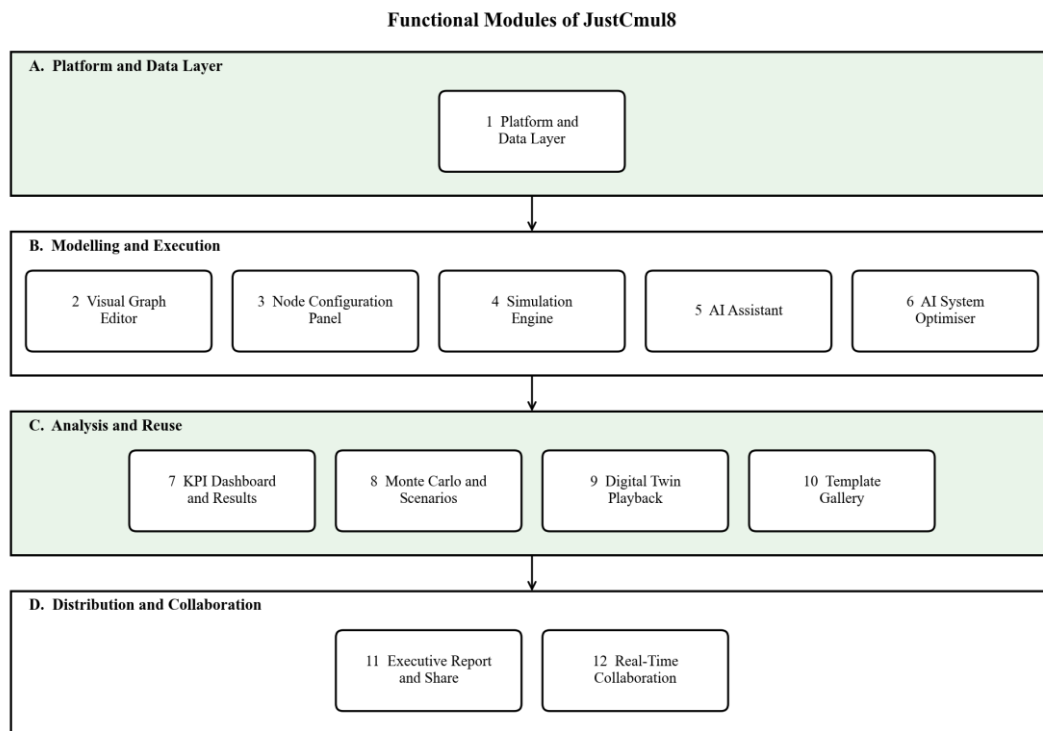


Figure 4.1: Module Structure of JustCmul8

Table 4.1: Module Overview and Requirement Coverage

No.	Module	Primary responsibility	Requirements
1	Platform and Data Layer	Authentication, project lifecycle, cloud persistence, profile	FR-01 to FR-06
2	Visual Graph Editor	Canvas-based construction of the simulation graph	FR-07 to FR-09
3	Node Configuration Panel	Parameter and cost entry with per-type validation	FR-10, FR-11
4	Simulation Engine	Code generation and in-browser SimPy execution	FR-12 to FR-15
5	AI Assistant	Natural-language generation of candidate models	FR-16, FR-17

No.	Module	Primary responsibility	Requirements
6	AI System Optimiser	Bottleneck diagnosis and one-click corrective fixes	FR-18
7	KPI Dashboard and Results	Metric computation, verification, visualisation, cost and ROI	FR-19 to FR-21
8	Monte Carlo and Scenario Comparison	Replicated runs, confidence intervals, baseline comparison	FR-22
9	Digital Twin Animated Playback	Animated 2D entity flow with transport controls	FR-23
10	Simulation Template Gallery	Reusable starter models across six domains	FR-24
11	Executive Report and Share	PDF, CSV, JSON, notebook export and share links	FR-25, FR-26
12	Real-Time Collaboration	Live presence, cursors and graph-operation broadcast	FR-27

4.1 Module 1: Platform and Data Layer

Purpose. This module decides who a user is, what they own and where their work is stored. Every other module depends on it, because a project cannot be created, shared or saved until someone exists to own it. It folds authentication, project management, cloud storage and account settings into one base layer on top of Supabase.

Functional responsibilities.

- Registration and login with email and password, plus one-click Google or GitHub OAuth.
- Session protection: a signed-in session carries over between visits, and middleware checks protected routes before any project data is fetched.
- Password reset by emailed link, password change while signed in, secure logout, and account deletion that cascades through everything the user owns.
- Creating a project in one of the six domains, optionally from a template, and renaming or deleting projects.
- Automatic saving of the graph to cloud storage as the user edits, with no save action.
- A dashboard listing every project with its domain and last-modified time, and a settings page for display name, avatar and default simulation domain.

Implementation. Sign-in is handled by Supabase Auth, so the application never sees a plaintext password beyond the form that submits it. Next.js middleware refreshes sessions and guards protected routes. The important design decision is that authorisation does not live in the

client. PostgreSQL Row-Level Security policies on every table check the requesting user's identity when a query runs, so a modified client cannot read another user's rows even if it builds the request by hand. A database trigger makes a profile row whenever a new user registers, and every child table is declared with cascading deletion so that removing an account removes its data.

Requirements covered	Principal source files	Developed by
FR-01 to FR-06	app/login, signup, forgot-password, update-password, auth/callback, dashboard, settings; middleware.ts; lib/supabase; api/account/delete; supabase/schema.sql	Khushi Singh (261713)

4.2 Module 2: Visual Graph Editor

Purpose. The graph editor is where modelling happens, and it carries most of the platform's no-code promise. A process can be written down as a directed graph by direct manipulation alone.

Functional responsibilities.

- A drag-and-drop canvas where nodes are picked from a palette and freely moved around.
- Fifteen node types across the six simulation domains, each with its own icon, colour and connection handles.
- Creating and deleting directed edges between node handles, with edges re-routed live as nodes move.
- Zoom, pan and fit-to-view, plus selecting and deleting nodes and edges.
- Structural validation before a run, which rejects graphs that cannot run and names the offending node.
- Automatic layout of generated and template graphs, using a strategy chosen for each domain.

Implementation. The canvas uses React Flow, which supplies the viewport transform, the handle-based connection model and the edge routing. Writing those ourselves would have eaten a large part of the schedule. Custom node components draw each of the fifteen types. Labels come from a domain registry, so a bank modeller sees “Teller” where a factory modeller sees “Machine”, although both are the same resource node underneath. Automatic layout uses Dagre

with a strategy picked per domain, since one generic layout gave diagrams that were correct but unintuitive.

Requirements covered	Principal source files	Developed by
FR-07 to FR-09	components/workspace/NodeCanvas.tsx, NodePalette.tsx; lib/simulation/layoutEngine.ts, simTypeRegistry.ts	Mohit Gupta (261715)

Table 4.2: The Fifteen Node Types Supported by the Simulation Engine

Node type	Behaviour	Typical domains
source	Generates entities at intervals drawn from a chosen distribution	All six
queue	A waiting line of finite or infinite capacity under FIFO, LIFO or priority discipline	All six
resource	A capacity-limited server that entities seize, hold for a sampled service time, then release	All six
service	A single processing step with its own duration	Manufacturing, logistics
decision	Routes entities down one of several outbound edges by assigned probabilities	All six
sink	Terminates an entity's journey and records its completion statistics	All six
container	Holds a level of bulk material or liquid, sampled at event boundaries	Liquid / material flow
store	A buffer of distinguishable items (SimPy Store, priority and filter variants)	Manufacturing, logistics
event_trigger	Monitors a condition and fires an event when it is met	Network, manufacturing
priority_resource	A preemptive resource: a higher-priority arrival displaces a lower one	Human queues, healthcare
channel	A transmission medium with propagation delay	Network and signal flow
broadcaster	Duplicates an entity to every outbound edge (one-to-many fan-out)	Network and signal flow
any_of	Waits on several concurrent events and proceeds when the first completes (SimPy AnyOf)	Network, manufacturing
all_of	Waits until every concurrent event has completed (SimPy AllOf)	Manufacturing, logistics
interrupter	Interrupts an in-progress process, modelling breakdowns and abandonment	Manufacturing, human queues

The first six node types are the classic discrete-event vocabulary and are enough for most models. The other nine cover what that set cannot express: continuous levels, typed buffers, preemption, propagation delay, fan-out and event synchronisation. The two synchronisers map directly onto SimPy's `AnyOf` and `AllOf`, so a generated script can wait on several events at once without custom scheduling code.

Table 4.3: The Six Simulation Domains and Their Vocabularies

Domain	Typical scenarios	Example node labels
Human Queues	Banks, clinics, ticket counters, reception desks	Teller, Counter, Waiting Area, Customer Arrival
Vehicle Traffic	Junctions, toll plazas, fuel stations, car washes	Approach, Signal, Toll Booth, Pump
Liquid / Material Flow	Tanks, pipelines, mixing and bottling plants	Reservoir, Pipe, Valve, Mixer
Manufacturing Lines	Assembly lines, machining cells, inspection bays	Machine, Buffer, Inspection, Assembly
Logistics / Warehousing	Depots, sorting hubs, order fulfilment	Inbound Dock, Picking, Packing, Dispatch
Network / Signal	Packet routing, message buses, sensor networks	Transmitter, Router, Link, Receiver

4.3 Module 3: Node Configuration Panel

Purpose. Placing a node says what it is; configuring it says how it behaves. This module collects the parameters that turn a bare topology into a runnable model, using ordinary form controls and no scripting field.

Functional responsibilities.

- A side panel that opens for the selected node and shows only the fields that fit its type.
- Entry of arrival rates, service-time distributions and their parameters, capacities, queue disciplines, routing probabilities and propagation delays.
- A choice among the five probability distributions, with the parameter fields changing to suit.
- Entry of cost parameters (resource hourly rate, holding cost and waiting-penalty cost) that the Cost and ROI analysis uses.
- Checking every value against the node type, with each change passed straight to the underlying model.

Implementation. A node's configuration is stored as a JSON object shaped by that node type's schema. This keeps the database schema stable when node types are added and still allows type-specific validation. Validation runs when a value is entered and again before a run. Messages name the offending node and say what to do about it, because showing a raw engine exception to a non-technical user would defeat the point of a no-code interface.

Requirements covered	Principal source files	Developed by
FR-10, FR-11	components/workspace/NodePropertiesPanel.tsx; lib/simulation/types.ts	Khushi Singh (261713)

4.4 Module 4: Simulation Engine

Purpose. The engine is the technical heart of the project. It turns the visual graph into an executable discrete event simulation and runs it wholly inside the browser on a genuine Python engine.

Functional responsibilities.

- Compiling the node-and-edge graph into a Python SimPy script that reflects the configured parameters.
- Running that script on the client through the Pyodide WebAssembly runtime, with no simulation computation on the server.
- A Web Worker execution path that keeps the interface responsive during a run.
- Taking the simulated duration and time unit (seconds, minutes, hours, days) before a run, and streaming periodic state snapshots and log events to the interface as it proceeds.
- A live Python code inspector that shows the generated script and exports it as a Jupyter notebook.

Implementation. A run has four stages. The graph is validated structurally. The code generator walks it and writes a SimPy script in which each node becomes a resource, store or process and each edge becomes a routing decision. The script goes to the worker, where Pyodide executes it. The resulting event trace returns to the main thread for analysis. Each resource uses the matching SimPy primitive (`Resource`, `PriorityResource`, `Store`, `PriorityStore`, `FilterStore` or `Container`). We kept the generated script readable on purpose. It shows in the code inspector and exports as a notebook, so a student can look at the Python that their

diagram produced and run it again independently. That makes the platform a teaching aid for SimPy and lets anyone verify a published result.

Requirements covered	Principal source files	Developed by
FR-12 to FR-15	lib/simulation/codeGenerator.ts, pyodideEngine.ts, pyodideWorker.ts, clientEngine.ts, jupyterExport.ts; components/workspace/CodeInspectorPanel.tsx	Mohit Gupta (261715)

4.5 Module 5: AI Assistant

Purpose. This module lowers a conceptual barrier, not a technical one. Someone who does not know how to draw a process as a graph can describe it in plain English and get a candidate model back.

Functional responsibilities.

- Turning a natural-language scenario into a candidate graph of nodes and edges with plausible parameters.
- Validating the generated graph (valid node types, no orphaned edges) before it is offered.
- A preview and an explicit accept step, so the live project never changes without consent.
- Help with simulation concepts and with the user's own model in non-technical language, with chat history kept for each project.
- Automatic fall-back to another AI model when the preferred one is down.

Implementation. Requests pass through Next.js API routes to Google Gemini, which keeps the API key on the server and out of the client bundle. The system prompt lists the available node types with their parameter schemas and includes the current graph. Putting the node registry in the prompt is what limits the model to graphs the engine can run. A model router holds a priority list of models, sorts failures into unavailable, rate-limited and empty response, and marks an unhealthy model for a while so later requests skip it. Generated graphs are validated by graph operations before display and arranged by the layout engine.

Requirements covered	Principal source files	Developed by
FR-16, FR-17	components/workspace/AIChatPanel.tsx; app/api/ai/chat and generate; lib/ai/systemPrompt.ts, graphOps.ts, modelRouter.ts, textFormat.ts	Khushi Singh (261713)

4.6 Module 6: AI System Optimiser

Purpose. Module 5 helps a user build a model; this one helps them act on it. After a run it reads the computed metrics, explains which node is holding the system back and why, and proposes a specific change that a single action can apply.

Functional responsibilities.

- Post-run inspection of utilisation, waiting times and flow through each node.
- A diagnosis of the constraining node, worded so that a non-specialist can act on it.
- A concrete corrective change, such as more capacity at a resource, rerouting at a decision node or a larger buffer.
- A one-click Apply Fix action, carried out as an ordinary graph edit.
- Keeping the graph from before the change so its effect can be compared in Module 8.

Implementation. The optimiser works from analytics output and not from the raw event trace, so its advice cites computed utilisation and queue statistics instead of impressions. A flow-analysis step first works out which node it makes structural sense to change, since adding capacity to a node that is not the constraint achieves nothing. Only that candidate is sent to the language model for explanation. The proposal is always shown for approval, so the optimiser advises and never acts on its own.

Requirements covered	Principal source files	Developed by
FR-18	app/api/ai/optimize/route.ts; lib/ai/flowAnalysis.ts, graphOps.ts	Mohit Gupta (261715)

4.7 Module 7: KPI Dashboard and Results

Purpose. An event trace is not an answer. This module turns the raw output of a run into the performance indicators a decision needs, and lays them out so the constraining node stands out without further analysis.

Functional responsibilities.

- Computing and showing utilisation, average and percentile waiting times, throughput and queue length for each node.
- Finding the bottleneck node automatically and flagging it clearly.
- A Flow and Balance Check that applies Little's Law to confirm the run is consistent, and a composite 0–100 system health score.

- Interactive tabs: Executive Summary, Flow and Balance Check, Traffic Heatmap, Line Buildup Timeline, Station Counters, Person / Item Journey, Live Activity Log and Cost and ROI.
- A Cost and ROI panel that puts the run in money terms, so a proposed change can be judged by whether it pays for itself.

Implementation. The analytics are TypeScript modules with no React dependency, so they can be unit-tested alone. The system health score is a weighted composite: completion rate (35%), utilisation balance (30%), queueing stability judged by Little’s Law (20%) and tail latency (15%). The bottleneck is the resource or queue node with the highest average waiting time in the run. Percentiles come from the real per-entity samples and are not estimated, and a conservation check confirms that work-in-progress equals arrivals minus completions.

Requirements covered	Principal source files	Developed by
FR-19 to FR-21	lib/simulation/analyticsEngine.ts, resolveKpiMetrics.ts; components/workspace/SimResultsPanel.tsx, AdvancedResultsDashboard.tsx, results-dashboard/*	Khushi Singh (261713)

Table 4.4: Key Performance Indicators and Their Computation

Metric	Definition	Input data
Utilisation	Fraction of run time for which a resource was occupied	Accumulated busy-seconds per resource
Average waiting time	Mean interval between joining a queue and being served	Queue entry and service-start times per entity
Percentile waiting time	50th, 90th, 95th and 99th percentiles of the waiting distribution	Real per-entity waiting samples
Throughput	Entities completing the model per unit of simulated time	Sink completion counts and run duration
Average queue length	Time-weighted mean number of entities waiting	Periodic depth snapshots
Little’s Law check	Compares L with λW and reports the discrepancy and stability verdict	Arrivals, completions, work-in-progress, mean time in system
System health score	0–100 composite: completion 35, utilisation balance 30, stability 20, tail latency 15	Outputs of the above
Total system cost	Resource, holding and waiting-penalty costs over the run	Cost parameters from Module 3

4.8 Module 8: Monte Carlo and Scenario Comparison

Purpose. One stochastic run is a single observation, not an estimate. This module repeats the run and reports the range in which the true behaviour lies, and then lets two configurations be compared against that measure of uncertainty.

Functional responsibilities.

- Running a model over 10, 20, 50 or 100 replications, each an independent run.
- Computing the mean, standard deviation and 95% confidence interval of every metric across replications.
- Drawing the intervals as bands, so that significance can be seen and not merely inferred.
- Placing a baseline scenario beside a modified one and reporting the difference for each metric.
- Progress reporting during a long analysis, so the user always has feedback.

Implementation. The runner pushes replications through the same engine as a single run and aggregates them with the standard-error formula of Section 1.1.2. It also exposes a scenario-comparison function that pairs the baseline kept before an applied fix with the current configuration. The aggregation is plain TypeScript, so it is easy to test and does not depend on the interface.

Requirements covered	Principal source files	Developed by
FR-22	lib/simulation/monteCarlo.ts, monteCarloRunner.ts; components/workspace/MonteCarloPanel.tsx	Mohit Gupta (261715)

4.9 Module 9: Digital Twin Animated Playback

Purpose. Numbers describe congestion, and animation shows it. This module replays a finished run as a moving two-dimensional picture of entities passing through the model, which shows where a system is failing much faster than a chart does.

Functional responsibilities.

- Animated playback of entities flowing through the nodes and edges of the model.
- Highlighting of queue congestion, so lines that are building up can be seen as they grow.
- Video-style transport controls: play, pause, step and speed selection from 0.5× to 10×.
- Scrubbing to any point in the run without executing the model again.

Implementation. Playback is drawn with Pixi.js on WebGL and not with DOM elements, because animating several hundred moving entities through the DOM slows down unacceptably. The animation is driven by the recorded entity journeys and not by re-running the model. Scrubbing and speed changes are therefore immediate, and what is on screen matches exactly the run whose metrics are displayed.

Requirements covered	Principal source files	Developed by
FR-23	components/workspace/DigitalTwinCanvas.tsx, PlaybackControls.tsx; lib/simulation/timelineSelectors.ts	Khushi Singh (261713)

4.10 Module 10: Simulation Template Gallery

Purpose. A blank canvas puts off a first-time user. This module offers working models in all six domains so that a user can start from something that already runs and change it, which is a gentler introduction than building from nothing.

Functional responsibilities.

- A gallery of starter graphs grouped by domain, each with a description.
- Instantiating a template, which fills a project with its nodes, edges and default parameters.
- Creating a fully independent copy, so edits can never flow back into the source template.
- Free editing of every parameter after instantiation.

Implementation. Templates are declared in the simulation type registry next to each domain's node palette and layout strategy. A domain is therefore defined in one place, and adding one does not mean touching many components. Instantiation copies the template's nodes and edges into the new project instead of referring to them, so independence holds by construction.

Requirements covered	Principal source files	Developed by
FR-24	components/workspace/TemplateGallery.tsx; lib/simulation/simTypeRegistry.ts	Mohit Gupta (261715)

4.11 Module 11: Executive Report and Share

Purpose. Work that cannot leave the platform is worth little. This module carries models and results outward, whether for archiving, for analysis elsewhere or for showing to someone who will never log in.

Functional responsibilities.

- One-click generation of a PDF executive report with a written summary, a canvas snapshot and KPI tables.
- Export of the project graph as JSON and of run results as CSV.
- Export of the generated Python script as a Jupyter notebook, for independent reproduction.
- Creation of a read-only public share link that never gives edit access.
- Revocation of a share link, after which the URL shows an explanatory notice.

Implementation. PDF generation runs on the client with jsPDF, and html-to-image captures the canvas. No document-rendering service is needed, so the architecture keeps its property of zero server compute. Share links are opaque random tokens. Public access goes only through two narrow database functions: one returns the name, domain and graph of a single shared project, and the other reports whether a token is active, revoked or invalid. Anonymous visitors therefore never get direct table access, and the read-only guarantee is enforced by the database and not by hiding controls in the interface.

Requirements covered	Principal source files	Developed by
FR-25, FR-26	lib/simulation/pdfReportGenerator.ts, reportGenerator.ts, jupyterExport.ts; components/workspace/ShareExportModal.tsx, ShareViewer.tsx, ShareLinkUnavailable.tsx; app/api/projects/[id]/share and export; app/share/[token]	Khushi Singh (261713)

4.12 Module 12: Real-Time Collaboration

Purpose. Teams now expect any modern design tool to work with several people at once, and this module supplies that: you see who else is in the workspace and what they are doing, much as in Canva or Figma.

Functional responsibilities.

- Live presence: a row of avatars showing who is in the project right now.

- Live cursors: each collaborator's pointer drawn on the canvas in their own colour, labelled with their name.
- Broadcasting graph operations (node and edge changes and node-data edits) to the other connected clients.
- Awareness of selection and editing, meaning which node a collaborator has selected or is editing.
- Database-level authorisation of the project channel, so that only entitled users can join.

Implementation. Collaboration uses Supabase Realtime channels over WebSockets. Each project has a private channel named after its identifier. Before joining, the client confirms it can read the project through Row-Level Security, and a realtime policy separately authorises the channel at the database. Presence uses the channel's presence state, while cursor positions and graph operations travel as broadcast messages. Cursor positions are temporary and never stored, since they mean nothing once a session ends.

Requirements covered	Principal source files	Developed by
FR-27	lib/realtime/usePresence.ts; components/workspace/LiveCursor.tsx	Mohit Gupta (261715)

4.13 Hardware Modules and Considerations

JustCmul8 needs no custom hardware. Even so, running simulations on the client and not the server gives the user's own machine a job it would not otherwise have, and we want to state that clearly.

Table 4.5: Hardware Roles in the Deployed System

Element	Role in the system	Consequence of the design
Client CPU	Executes the WebAssembly runtime and the entire simulation	Determines run duration; large models are slower on weaker machines
Client memory	Holds the Pyodide runtime, the graph, the event trace and the rendered canvas	Sets the practical ceiling on model size and replication count
Client GPU	Renders the digital-twin playback through WebGL	Enables smooth animation of many simultaneous entities
Network link	Carries persistence, collaboration and AI requests only	Simulation continues to run on an intermittent connection
Application server	Serves the application and a small number of API routes	Carries no simulation load, so hosting cost is independent of usage intensity

Element	Role in the system	Consequence of the design
Database server	Managed PostgreSQL storing projects, runs, chats and shares	Sized by data volume rather than by computational demand

The last two rows restate the economic argument of Section 2.2.2 in hardware terms. Neither server takes part in the computation, so the infrastructure for a thousand users running simulations is about what a thousand users viewing a static page would need. That is what keeps the platform sustainable without a licence fee.

4.14 Individual Work Justification

The project was done by two people. Requirements analysis, architecture, database design, integration, testing and this report were shared work. The twelve modules were split equally, six each. Every module was designed, implemented, unit-tested and documented by its owner and reviewed by the other member. The split is summarised below.

Table 4.6: Division of Work between Team Members

Module	Owner	Justification of contribution
1 Platform and Data Layer	Khushi Singh (261713)	Authentication flows, session middleware, dashboard, settings, database schema and Row-Level Security policies
2 Visual Graph Editor	Mohit Gupta (261715)	React Flow canvas, custom node components, palette, layout engine and structural validation
3 Node Configuration Panel	Khushi Singh (261713)	Type-specific forms, distribution controls, cost parameters and value validation
4 Simulation Engine	Mohit Gupta (261715)	SimPy code generator, Pyodide worker, execution pipeline and the verification harness
5 AI Assistant	Khushi Singh (261713)	Chat interface, prompt design, graph generation, model router and history
6 AI System Optimiser	Mohit Gupta (261715)	Flow analysis, bottleneck diagnosis, one-click fix and cost-aware recommendation
7 KPI Dashboard and Results	Khushi Singh (261713)	Analytics engine, Little's Law check, health score, dashboard tabs and Cost and ROI
8 Monte Carlo and Scenario Comparison	Mohit Gupta (261715)	Replication runner, confidence intervals and scenario comparison
9 Digital Twin Playback	Khushi Singh (261713)	Pixi.js renderer, entity journeys, transport controls and timeline
10 Template Gallery	Mohit Gupta (261715)	Domain registry, six-domain template library and instantiation

Module	Owner	Justification of contribution
11 Executive Report and Share	Khushi Singh (261713)	PDF report generation, JSON/CSV/notebook export, share links and read-only viewer
12 Real-Time Collaboration	Mohit Gupta (261715)	Presence channel, live cursors and graph-operation broadcast

Both of us contributed to the specification, the design diagrams, the test cases and the report. Reviewing each other's modules is what let the pieces be integrated with few defects.

5. SYSTEM ANALYSIS AND DESIGN

This chapter covers the analysis and design of JustCmul8, moving from the outside of the system to the inside. The context diagram draws the boundary. Data flow diagrams break down the processes within it. Entity relationship and class diagrams describe the data and the objects, and behavioural diagrams show how those objects interact while the system runs. Two physical topics close the chapter: the database tables and the deployment topology.

5.1 Context Diagram

The context diagram (data flow diagram level 0) fixes the system boundary. JustCmul8 appears as one process, surrounded by the external entities that exchange data with it, and nothing inside is broken down. Its job is to settle what belongs to the system and what does not before internal structure is discussed.

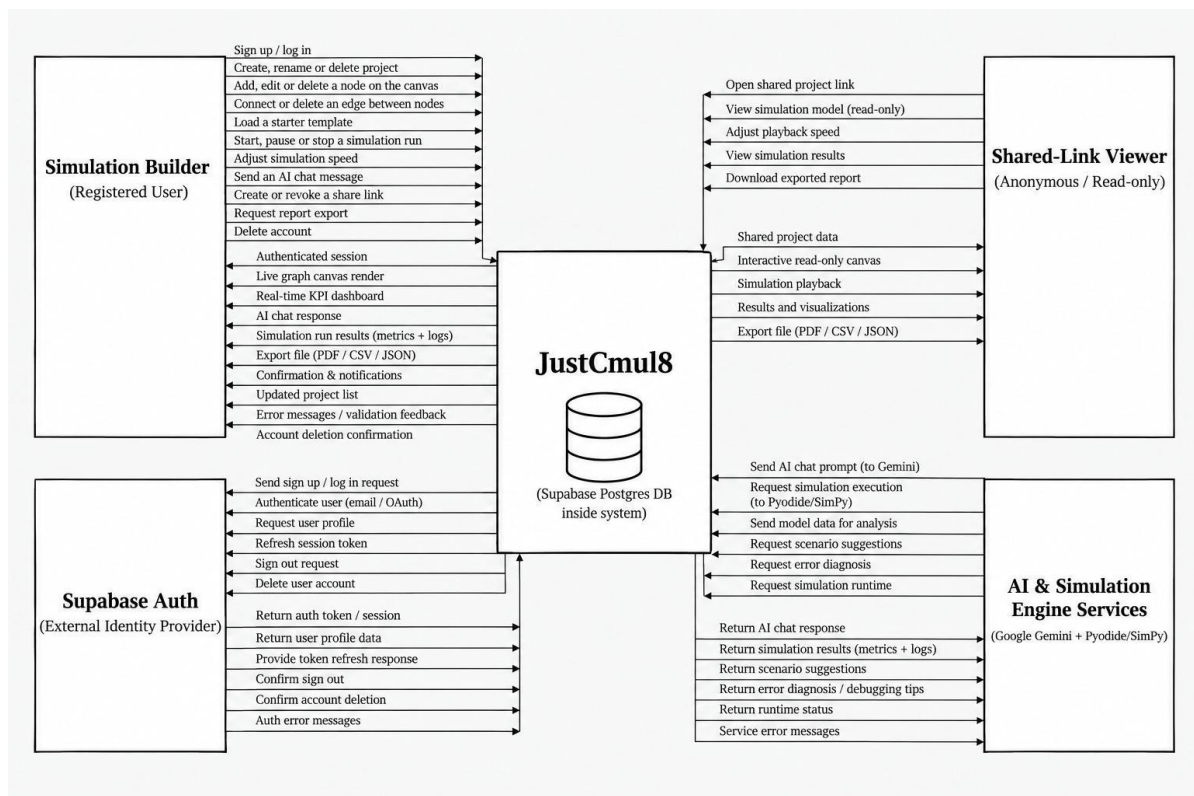


Figure 5.1: Context Diagram (DFD Level 0) for JustCmul8

Four external entities touch the system. The **Simulation Builder** is the registered user who creates and runs models. This entity sends in flows for signing up and logging in, creating, renaming and deleting projects, adding, editing and deleting nodes and edges, loading starter templates, starting and stopping runs, changing simulation speed, sending AI chat messages,

creating and revoking share links, requesting report exports and deleting the account. Flows going back carry the authenticated session, the live canvas rendering, the real-time KPI dashboard, AI responses, run results with metrics and logs, export files, confirmations, the updated project list and validation feedback.

The **Shared-Link Viewer** is an anonymous visitor who holds a public share URL. This entity opens a shared project, views the model read-only, changes playback speed, views results and downloads an exported report. The two user entities are deliberately unequal: no flow from the Shared-Link Viewer changes stored state.

Supabase Auth is the external identity provider. The system sends it sign-up and login requests, profile queries, session refreshes, sign-out requests and account-deletion instructions, and gets back tokens, sessions, profile data and confirmations. Keeping authentication outside the boundary shows that JustCmul8 hands credential handling to someone else and does not build it. The **AI and Simulation Engine Services** entity stands for the Google Gemini API and for the Pyodide and SimPy runtime delivered over a CDN. The PostgreSQL database sits inside the boundary, because the application provisions and governs it and does not simply consume it as an independent third party.

5.2 DFD, ERD, Class Diagram and State Transition Diagram

5.2.1 Data Flow Diagram, Level 1

The level-1 diagram splits the single process of the context diagram into seven functional processes and adds the data stores that they read and write.

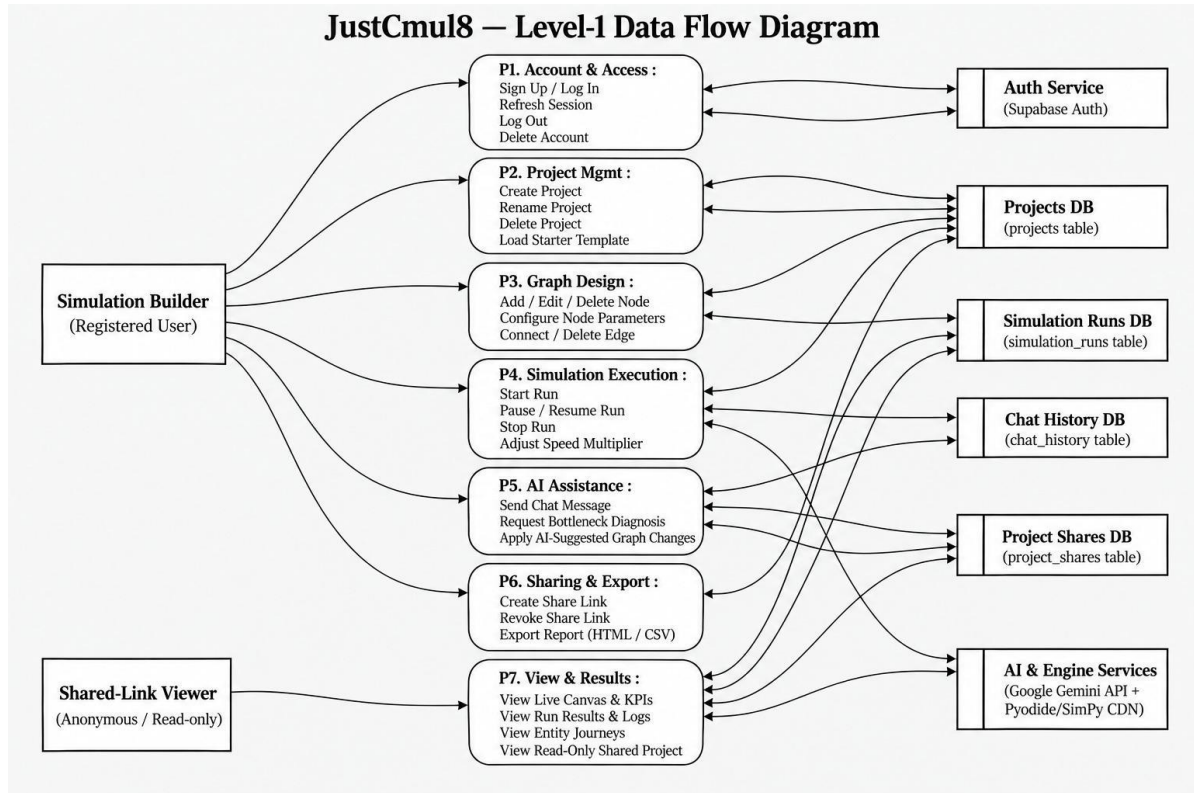


Figure 5.2: Level-1 Data Flow Diagram for JustCmul8

Process **P1, Account and Access**, deals with sign-up, log-in, session refresh, log-out and account deletion, and talks to the external authentication service. **P2, Project Management**, covers creating, renaming and deleting projects and loading starter templates, and reads and writes the Projects store. **P3, Graph Design**, adds, edits and deletes nodes, sets node parameters and connects or removes edges, saving every change to the same store.

P4, Simulation Execution, starts and stops runs and changes the speed multiplier. It reads the graph from the Projects store, drives the external engine services and writes run records to the Simulation Runs store. **P5, AI Assistance**, sends chat messages, asks for bottleneck diagnosis and applies AI-suggested graph changes; it exchanges data with the Gemini service and logs the exchanges in the Chat History store. **P6, Sharing and Export**, creates and revokes share links and produces exports, writing to the Project Shares store. **P7, View and Results**, serves the live canvas and KPIs, run results and logs, entity journeys and the read-only shared view. It is the only process that the anonymous Shared-Link Viewer can reach, so the read-only guarantee comes from the structure of the design and not from convention.

5.2.2 Data Flow Diagram, Level 2

Process P4 is the most involved in the system, so level 2 opens it up. This is the path that turns a drawing into numbers, and it is where the architectural commitment of Chapter 4 shows up as data flow.

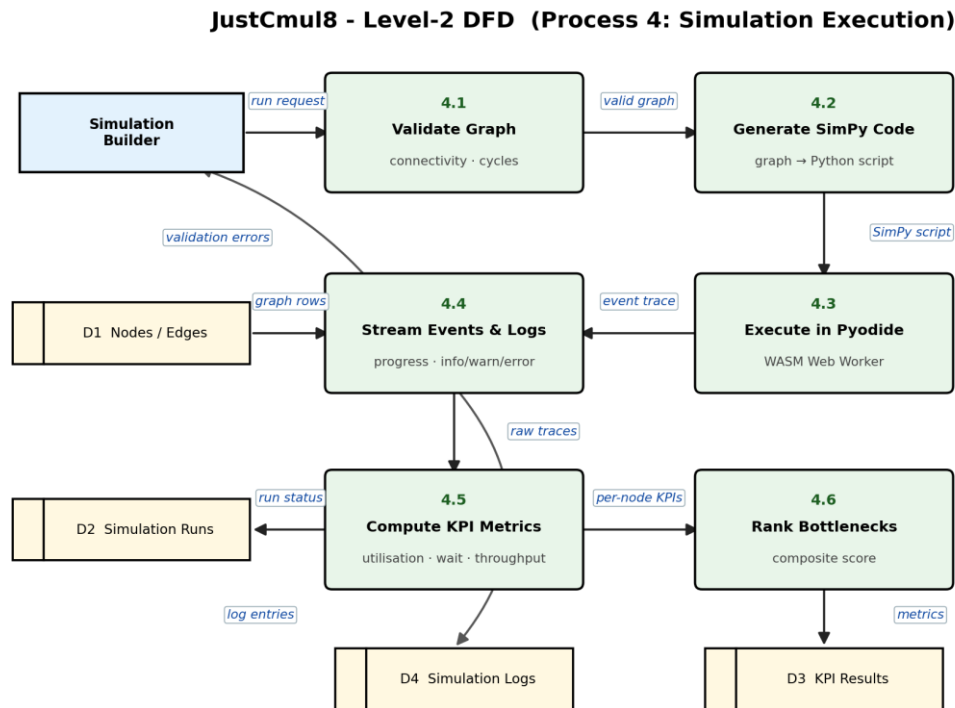


Figure 5.3: Level-2 Data Flow Diagram: Decomposition of Process P4 (Simulation Execution)

A run request from the Simulation Builder enters process 4.1, **Validate Graph**, which reads node and edge data from the Projects store and checks connectivity. If validation fails, errors go back to the user and the run stops there, so an invalid model never reaches the engine. A valid graph moves to process 4.2, **Generate SimPy Code**, whose output goes to process 4.3, **Execute in Pyodide**, running inside a Web Worker. The event trace from there flows to process 4.4, **Stream Events and Logs**, which passes progress and log entries to the interface during the run.

Raw results then go to process 4.5, **Compute KPI Metrics**, which works out utilisation, waiting times and throughput and updates the run record. Per-node metrics continue to process 4.6, **Rank Bottlenecks**. The order is intentional. Bottleneck ranking uses computed metrics and not the raw trace, so 4.6 must follow 4.5, and the optimiser of Module 6 rests on the same dependency.

5.2.3 Entity Relationship Diagram

The entity relationship diagram shows how persistent data is organised and how the entities relate. It uses Chen notation: rectangles for entities, ellipses for attributes and diamonds for relationships, with cardinality marked.

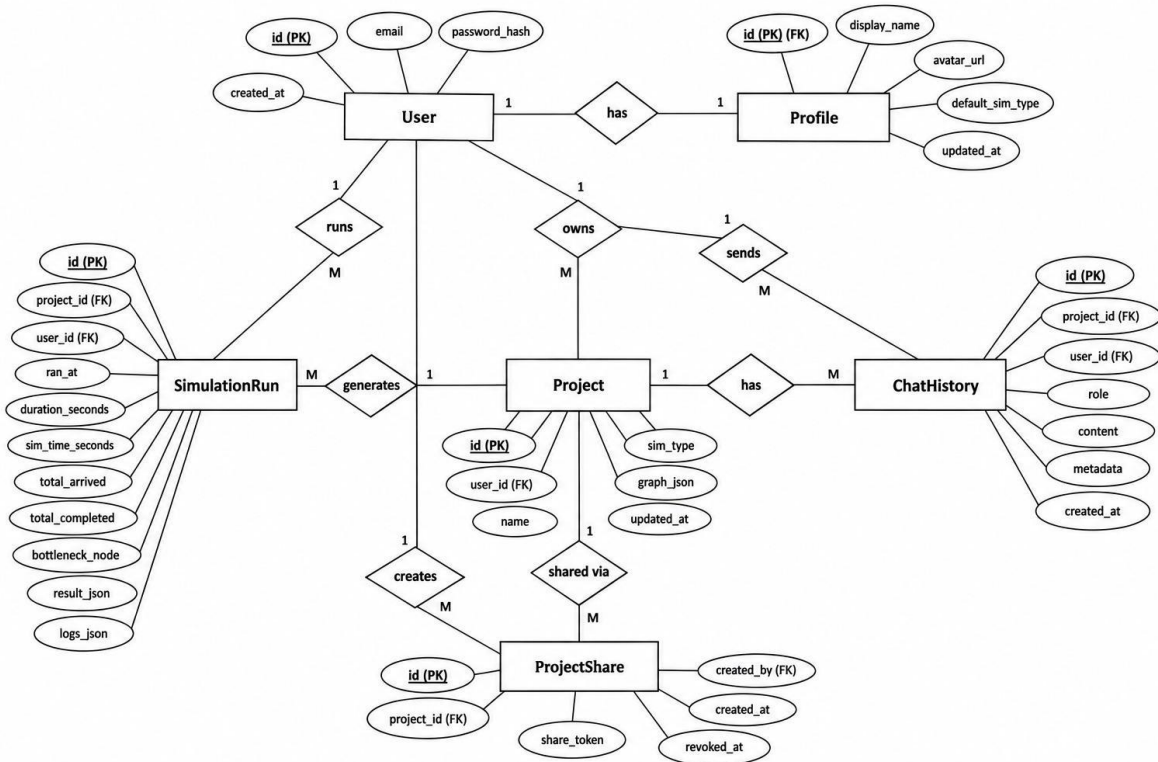


Figure 5.4: Entity Relationship Diagram for the JustCmul8 Database

The **User** entity sits at the centre and has a one-to-one relationship with **Profile**, which stores the display name, avatar URL, default simulation type and update timestamp. Keeping the profile apart from the credential record separates presentation data from authentication data. A User owns many **Projects**, and each project has an identifier, an owner, a name, a simulation type, the graph (kept as a JSON document) and an update timestamp. A User also runs many **SimulationRuns**, sends many **ChatHistory** messages and creates many **ProjectShares**. Each Project produces many runs, has many chat entries and is shared through many share records.

The SimulationRun entity keeps the run timestamp, wall-clock duration, simulated duration, totals arrived and completed, the identified bottleneck node and the result and log payloads. The bottleneck is stored as an attribute of the run instead of being recomputed on request, so an old run keeps the diagnosis made at the time. **ProjectShare** holds an opaque share token with creation and revocation timestamps. A revoked share stays as a row with a revocation

timestamp and is not deleted. That is how the system can tell a withdrawn link from one that never existed, and show the viewer an explanation in place of a bare error.

5.2.4 Class Diagram

The class diagram shows the object model as implemented in TypeScript, with each class, its typed attributes, its operations and its associations.

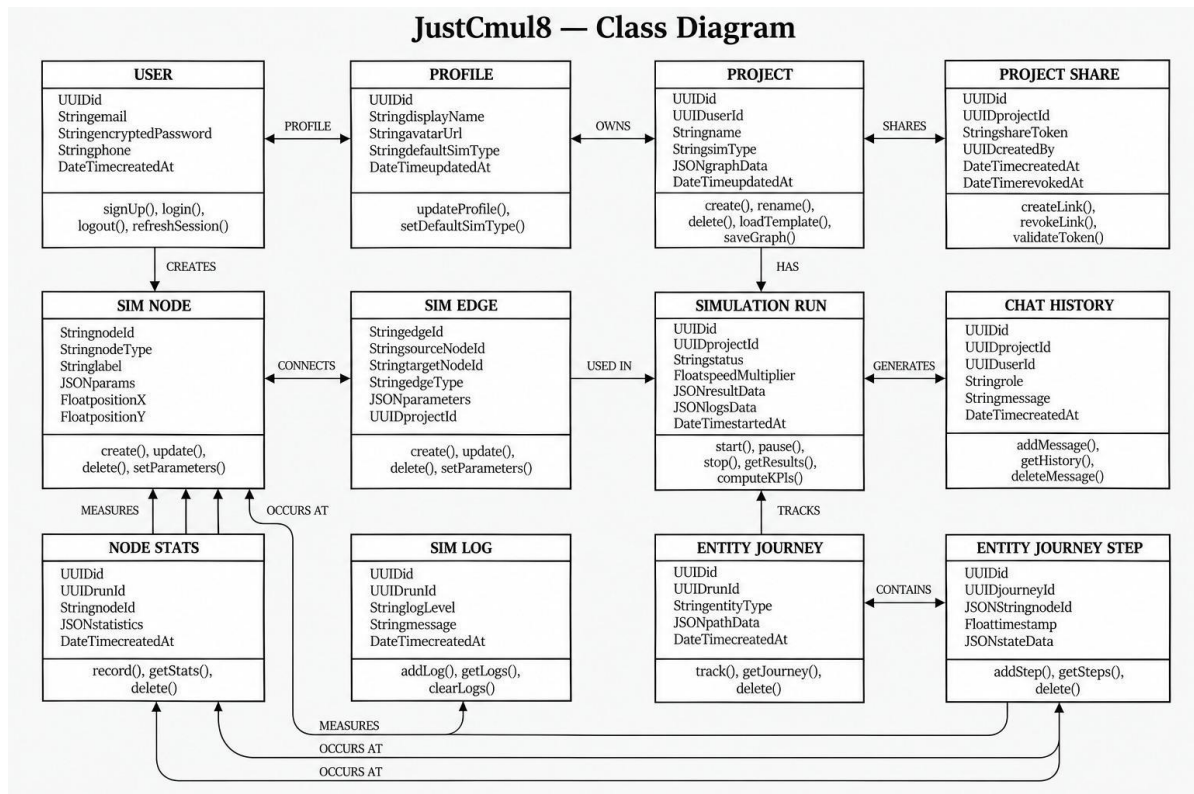


Figure 5.5: Class Diagram of the JustCmul8 Object Model

The top row holds the account and project classes: **User**, **Profile**, **Project** and **ProjectShare**. The associations read “User has a Profile”, “User owns Projects” and “Project is shared through ProjectShare”. The middle row holds the graph and execution classes. **SimNode** has the node identifier, type, label, a JSON parameter object and canvas coordinates. **SimEdge** joins two nodes. **SimulationRun** has status, speed multiplier, and result and log payloads. **ChatHistory** records each AI exchange. Keeping parameters as JSON, not as fixed columns, lets fifteen node types with quite different parameters share one class and avoids a sparse schema.

The bottom row holds the measurement classes: **NodeStats**, **SimLog**, **EntityJourney** and **EntityJourneyStep**. The journey and step classes exist for the digital-twin playback. Each step carries a timestamp and a node reference, so the animation can be rebuilt and scrubbed without running the simulation again.

5.2.5 State Transition Diagram

The state transition diagram follows the life of a node while a model runs. At any instant every node is in exactly one of these states, and the event scheduler causes the transitions.

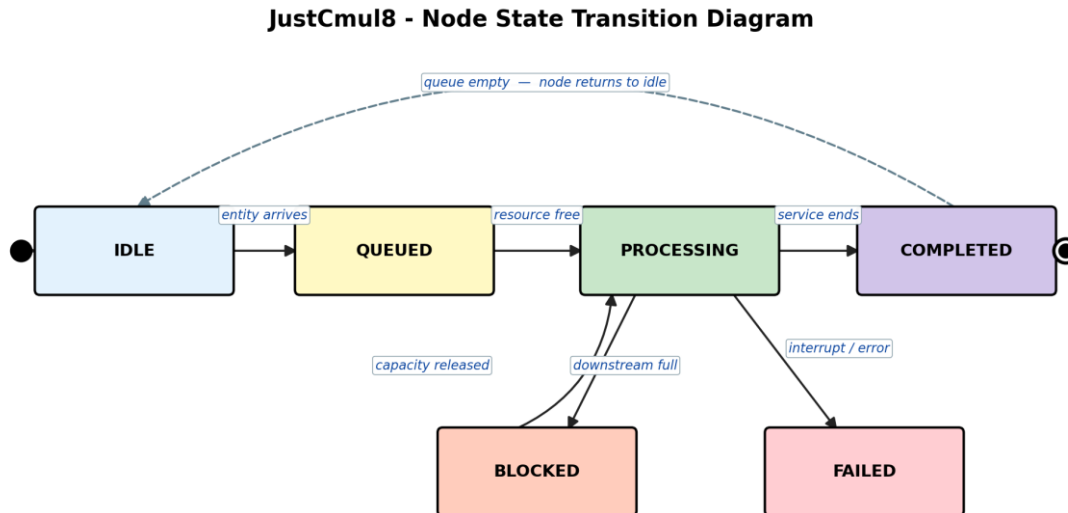


Figure 5.6: State Transition Diagram for a Node During Simulation Execution

A node starts in **IDLE** with no entity to process. When an entity arrives, the node moves to **QUEUED**, where the entity waits for capacity. Once the resource is free, the node enters **PROCESSING** and holds the entity for a service time sampled from the configured distribution. When that ends the node goes to **COMPLETED** and the entity is routed onward; if nobody else is waiting, the node drops back to **IDLE**.

Two unusual transitions leave **PROCESSING**. If the downstream node cannot take the entity, the node goes to **BLOCKED** and stays there until capacity is released. Blocking is how congestion travels backwards through a model, and having it as a separate state lets the analytics layer tell a node that is slow from one that is merely held up by the node after it. The other transition leads to **FAILED**, which happens when a process is interrupted by an interrupter node, by preemption at a priority resource or by a runtime error. The state model maps straight onto SimPy semantics: **QUEUED** is an outstanding resource request, **PROCESSING** is a timeout while the resource is held, **BLOCKED** is a pending put on a downstream store, and **FAILED** is a raised `Interrupt`.

5.3 Use Case Diagram

The use case diagram names the actors and the services the system offers each of them. It describes functionality from outside, as what an actor can accomplish, and not from inside, as how it is done.

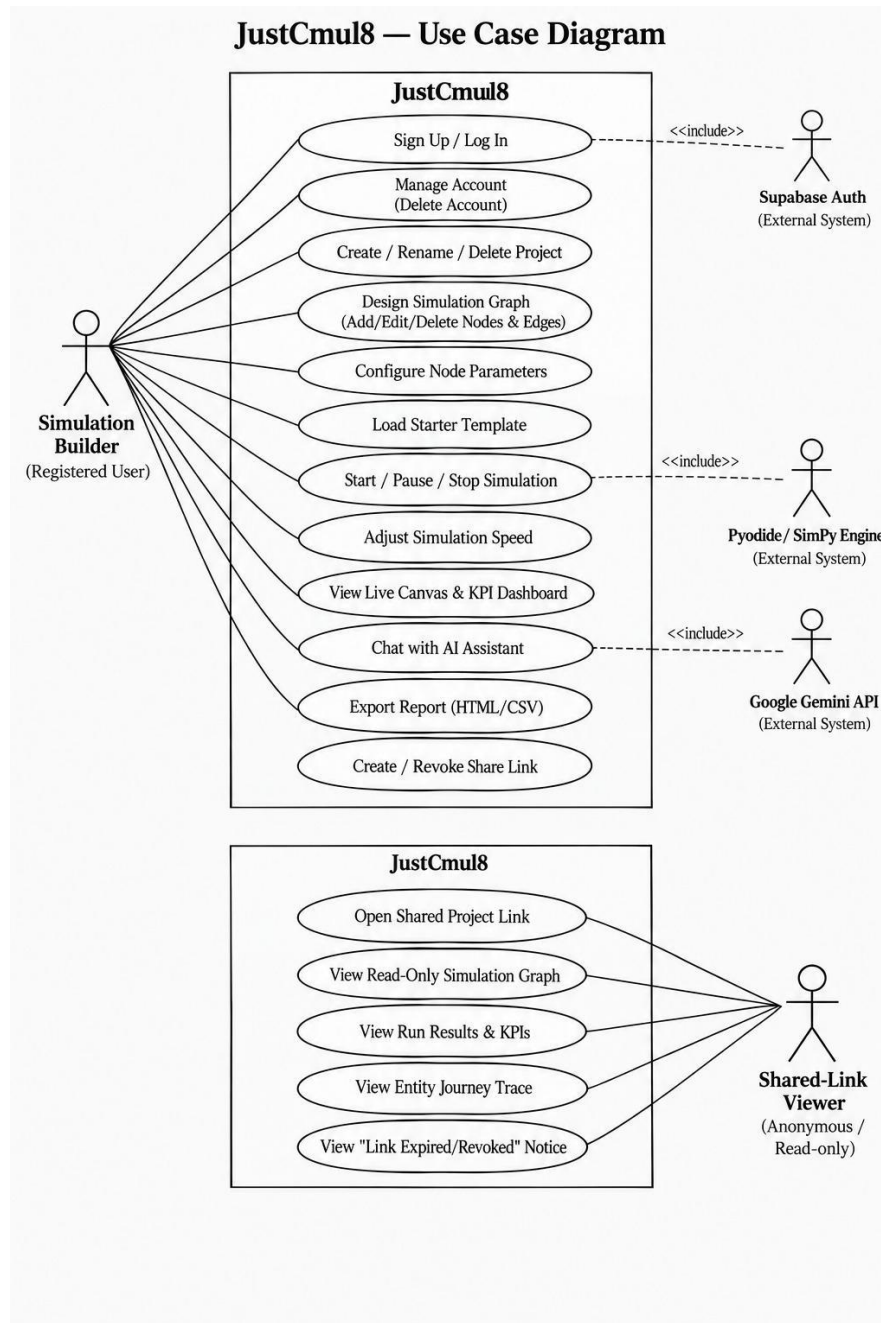


Figure 5.7: Use Case Diagram for JustCmul8

There are two human actors. The **Simulation Builder**, a registered user, can sign up or log in, manage the account (including deleting it), create, rename and delete projects, design the simulation graph, set node parameters, load a starter template, start and stop a simulation,

change the speed, watch the live canvas and KPI dashboard, chat with the AI assistant, export a report and create or revoke a share link. The **Shared-Link Viewer** is an anonymous actor whose use cases sit in a separate boundary: opening a shared project link, viewing the read-only graph, viewing results and KPIs, viewing the entity journey trace, and viewing the notice shown when a link has expired or been revoked. That last use case is easy to forget, and leaving it out makes for a poor experience, so we modelled it explicitly and did not treat it as an error path.

Three external systems take part through include relationships. Supabase Auth is included by sign-up and log-in, the Pyodide and SimPy engine by starting and stopping a simulation, and the Google Gemini API by chatting with the assistant. We used include and not extend because in each case the external interaction has to happen for the use case to finish.

5.3.1 Use Case Specifications

Table 5.1: Use Case Specification: Run Simulation

Field	Description
Use case ID	UC-01
Use case name	Run Simulation
Primary actor	Simulation Builder
Secondary actor	Pyodide / SimPy engine
Precondition	The user is authenticated, a project is open and the graph contains at least one source and one sink
Main flow	1. The user sets the duration and time unit. 2. The user starts the run. 3. The system validates the graph structurally. 4. The system generates the SimPy script. 5. The worker executes the script through Pyodide. 6. Progress and log events stream to the interface. 7. The system computes KPIs and ranks the bottleneck. 8. The dashboard renders the results and the run is saved.
Alternative flow	3a. Validation fails: the system names the disconnected node, explains the fault and aborts before execution. 5a. The runtime raises an error: the error is shown with the relevant log entries. 5b. The user stops the run: execution halts.
Postcondition	On success a run record with KPI results and logs is persisted
Requirements	FR-12, FR-13, FR-14, FR-15, FR-19, FR-20

Table 5.2: Use Case Specification: Generate Model via AI

Field	Description
Use case ID	UC-02
Use case name	Generate Simulation Model from Natural Language
Primary actor	Simulation Builder
Secondary actor	Google Gemini API
Precondition	The user is authenticated and a project is open
Main flow	1. The user describes the scenario in plain English. 2. The system composes a prompt from the description, the node registry and the current graph. 3. The request is proxied through the server to Gemini. 4. The response is parsed into candidate nodes and edges. 5. The candidate graph is validated. 6. The graph is laid out and shown as a preview. 7. The user accepts and the graph is written to the project.
Alternative flow	5a. Validation fails on an unknown node type or orphaned edge: the candidate is rejected and the user is invited to rephrase. 7a. The user discards the preview: the project is unchanged. 3a. The preferred model is unavailable: the router falls back to the next model; if all fail, an explanatory message is shown and manual construction remains available.
Postcondition	Either the project contains the accepted graph or it is entirely unmodified
Requirements	FR-16, FR-17

Table 5.3: Use Case Specification: Share Project Read-Only

Field	Description
Use case ID	UC-03
Use case name	Share Project through a Public Link
Primary actor	Project Owner
Secondary actor	Shared-Link Viewer (anonymous)
Precondition	The user owns the project
Main flow	1. The owner requests a share link. 2. The system generates an opaque token and stores it against the project. 3. The link is presented for copying. 4. A viewer opens the link without authenticating. 5. The system validates the token and renders the project read-only.
Alternative flow	5a. The token has been revoked: the viewer sees an explanatory notice. 5b. The token is unrecognised: access is refused. 2a. The owner later revokes the link: the revocation timestamp is set and later requests fall to 5a.

Field	Description
Postcondition	The viewer has read access to that project alone, with no ability to modify anything
Requirements	FR-26

Table 5.4: Use Case Specification: Collaborate in Real Time

Field	Description
Use case ID	UC-04
Use case name	Live Collaboration in a Project
Primary actor	Collaborator
Secondary actor	Supabase Realtime, other collaborators
Precondition	Two or more authenticated users with access have the same project open
Main flow	1. Each client verifies its access to the project and joins the project's private channel. 2. Presence state and cursor positions are broadcast continuously. 3. A user modifies a node or edge. 4. The change is broadcast as a graph operation. 5. Every other client applies the operation and re-renders.
Alternative flow	1a. The user has no access to the project: the channel is not joined. 5a. A client disconnects: its presence indicator is removed.
Postcondition	Connected clients display the same graph and can see one another's presence
Requirements	FR-27

5.4 Architecture Design

JustCmul8 uses a layered architecture in which each layer depends only on the one below it. What is unusual is the execution layer. In a conventional web application the computation happens on the server, but here it happens on the client, and nearly every consequence discussed in this report comes from that reversal.

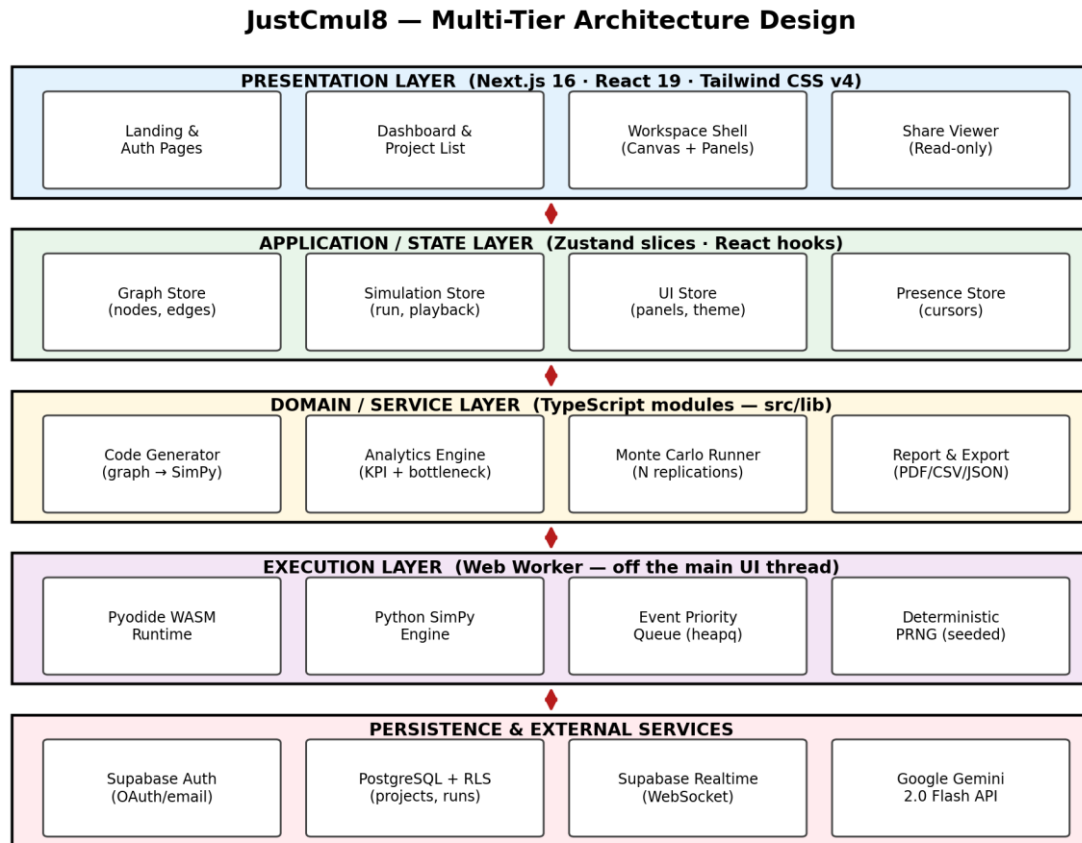


Figure 5.8: Multi-Tier Architecture of JustCmul8

The **presentation layer** uses Next.js 16, React 19 and Tailwind CSS. It contains the landing and authentication pages, the dashboard, the workspace shell that hosts the canvas and panels, and the read-only share viewer. It has no business logic: it displays state and passes on what the user wants to do.

The **application and state layer** holds the Zustand slices: the graph store, the simulation store, the interface store and the presence store. Because they are separate slices, a change in playback position does not make components that only watch the graph render again, which matters when the canvas has hundreds of nodes.

The **domain and service layer** contains the TypeScript modules that do the real work: the code generator that turns a graph into SimPy, the analytics engine that computes KPIs, the Monte Carlo runner and the report and export modules. They are plain modules with no React dependency, so each can be unit-tested alone.

The **execution layer** runs in a Web Worker, off the main thread. It holds the Pyodide WebAssembly runtime and the SimPy engine, whose event priority queue orders the scheduled events. Its isolation is what satisfies the requirement that the interface never freezes.

The **persistence and external services layer** consists of Supabase Auth, PostgreSQL with Row-Level Security, Supabase Realtime and the Gemini API. We put authorisation at this lowest layer, and not in the presentation layer where it would be most visible, because that makes it impossible to bypass. A request that reaches the database carries the user's identity and is filtered there, whatever the client believes it may do.

5.5 Activity, Component and Collaboration Diagrams

5.5.1 Activity Diagram: End-to-End User Workflow

Chapter 3 gave an activity network of the development lifecycle. The diagram here shows a different flow: the path a user takes through the running system, from authentication to export. Swimlanes separate the Simulation Builder, the system, Supabase Auth, the Pyodide and SimPy engine and the Gemini API, so it is clear who is responsible for each activity.

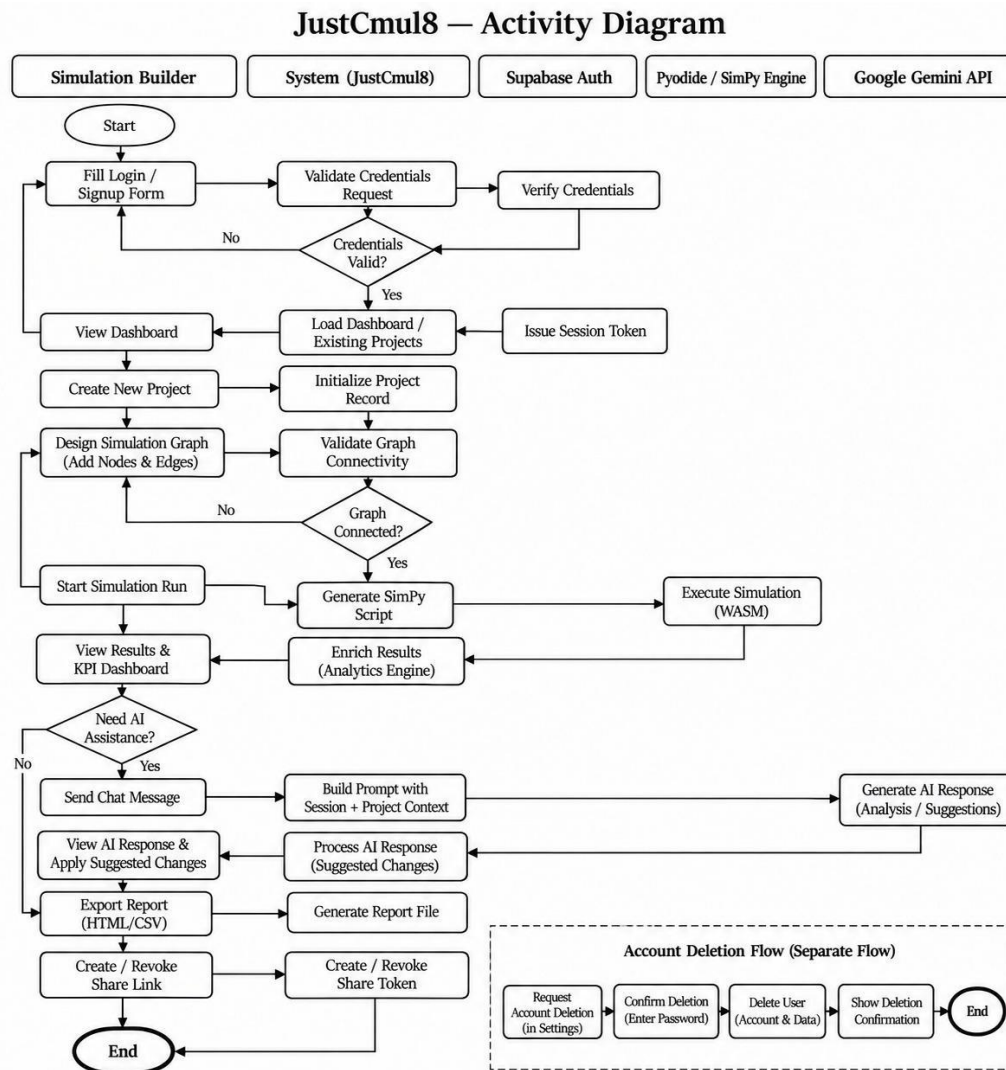


Figure 5.9: Activity Diagram of the End-to-End User Workflow

The flow begins with the user filling in the login or sign-up form. The system asks Supabase Auth to verify the credentials, and a decision node checks whether they are valid; if not, control goes back to the form. On success a session is issued and the dashboard loads with the user's projects. The user creates a project and designs the graph, and the system validates connectivity. A second decision node sends control back to the design activity if the graph is not connected, and that loop keeps an unrunnable model away from the engine.

Once the graph is valid, the user starts the run. The system generates the SimPy script and passes it to the engine lane, where it runs as WebAssembly. Results come back to the system lane, where the analytics engine enriches them, and the user looks at the KPI dashboard. A third decision node asks whether AI assistance is wanted. If it is, the system builds a prompt with the project context, Gemini replies with analysis or suggestions, and the user may apply

them, which returns control to the design activity. If not, the flow goes on to exporting a report and creating or revoking a share link before it reaches the end node. Account deletion is drawn as its own flow, because it is not part of modelling and must not be reachable by accident from inside it.

5.5.2 Component Diagram

The component diagram shows the implementation units of the system and how they depend on one another, grouped by the environment in which each one runs.

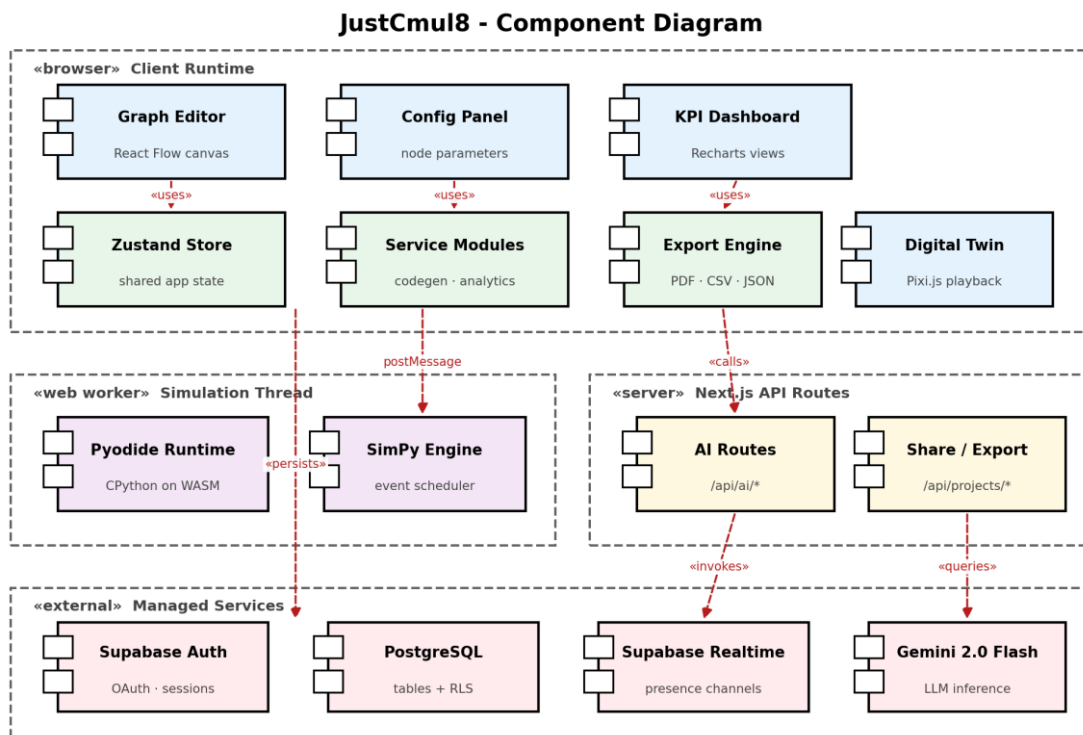


Figure 5.10: Component Diagram of the JustCmul8 Implementation

The **browser client runtime** has the interface components (the graph editor on React Flow, the configuration panel, the KPI dashboard on Recharts and the Pixi.js digital twin), the Zustand store, the service modules for code generation and analytics, and the export engine. Interface components depend on the store and not on each other, so a panel can be replaced or moved without touching the rest. The **simulation thread** is a separate group because it runs in a Web Worker. It holds the Pyodide runtime and the SimPy engine and talks to the client only by message passing, so a long simulation cannot block rendering. The **server** group contains the Next.js API routes (the AI routes and the share and export routes). They exist for the few operations that must not run on the client, mainly holding the Gemini API key. The **external** group contains the managed services.

5.5.3 Collaboration Diagram

The collaboration diagram shows the objects involved in one simulation run and the messages they pass, numbered in the order they occur.

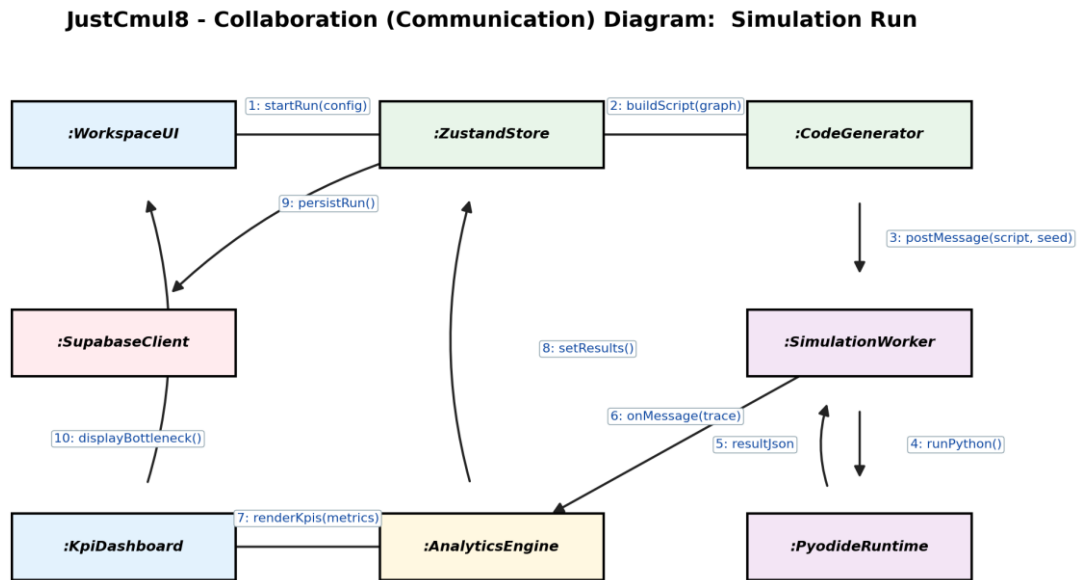


Figure 5.11: Collaboration Diagram for the Simulation Run Interaction

The interaction starts when the workspace interface calls `startRun` on the store with the run configuration (message 1). The store calls `buildScript` on the code generator and passes the current graph (2). The generated script is posted to the simulation worker (3), a message that crosses the thread boundary. Inside the worker, `runPython` is called on the Pyodide runtime (4), which returns the result payload (5). The trace is forwarded to the main thread, where the analytics engine receives it (6), renders KPIs on the dashboard (7) and writes the computed results into the store (8). The store saves the run through the Supabase client (9), and the dashboard shows the identified bottleneck (10). Two properties stand out. The store sits between the interface and the services, so any component that watches it sees a consistent view of the run. And saving happens after analysis, so a run is stored with its computed metrics in a single write.

5.6 Table Design

The database is a managed PostgreSQL instance. Every primary key is a UUID generated by the database. Foreign keys cascade on deletion wherever the child cannot outlive the parent, and every application table has a Row-Level Security policy. Structures whose shape depends

on type (the graph, result payloads and log collections) are stored as JSONB, which keeps the relational schema small and stable and still queryable. The graph, meaning nodes with their parameters and positions plus edges, is kept as a single JSONB document on the project, which makes loading and export fast.

5.6.1 Tables

Table 5.5: Table: auth.users (managed by Supabase Auth)

Column	Data type	Constraints	Description
id	uuid	PRIMARY KEY	Unique user identifier, referenced by every application table
email	text	UNIQUE	Login email address
encrypted_password	text	None	Salted hash of the password; never stored in plain text
raw_user_meta_data	jsonb	None	Registration metadata, including the full name
created_at	timestamptz	DEFAULT now()	Account creation timestamp

Table 5.6: Table: public.profiles

Column	Data type	Constraints	Description
id	uuid	PRIMARY KEY, FK → auth.users(id) ON DELETE CASCADE	Shares the user's identifier (one-to-one)
display_name	text	None	Name shown in the interface and on collaborator cursors
avatar_url	text	None	Profile image location
default_sim_type	text	DEFAULT 'human_queue'	Domain preselected for new projects
updated_at	timestamptz	DEFAULT now()	Last modification timestamp

Table 5.7: Table: public.projects

Column	Data type	Constraints	Description
id	uuid	PRIMARY KEY, DEFAULT gen_random_uuid()	Unique project identifier
user_id	uuid	NOT NULL, FK → auth.users(id) ON DELETE CASCADE	Owning user
name	text	NOT NULL	Display name

Column	Data type	Constraints	Description
sim_type	text	NOT NULL, DEFAULT 'human_queue'	One of the six simulation domains
graph_json	jsonb	DEFAULT {nodes: [], edges: []}	The graph: nodes, parameters, positions and edges
updated_at	timestampz	DEFAULT now()	Last modification timestamp

Table 5.8: Table: public.simulation_runs

Column	Data type	Constraints	Description
id	uuid	PRIMARY KEY	Unique run identifier
project_id	uuid	NOT NULL, FK → projects(id) ON DELETE CASCADE	Project that was simulated
user_id	uuid	NOT NULL, FK → auth.users(id) ON DELETE CASCADE	User who initiated the run
ran_at	timestampz	DEFAULT now()	Run timestamp
duration_seconds	integer	None	Wall-clock execution time
sim_time_seconds	numeric	None	Simulated duration modelled
total_arrived	integer	None	Entities generated
total_completed	integer	None	Entities that reached a sink
bottleneck_node	text	None	Node identified as the constraint in this run
result_json	jsonb	None	Per-node metrics and aggregate KPIs
logs_json	jsonb	None	Sampled log events

Table 5.9: Table: public.chat_history

Column	Data type	Constraints	Description
id	uuid	PRIMARY KEY	Unique message identifier
project_id	uuid	NOT NULL, FK → projects(id) ON DELETE CASCADE	Project the exchange concerns
user_id	uuid	FK → auth.users(id) ON DELETE CASCADE	User who sent the message
role	text	NOT NULL, CHECK IN ('user', 'assistant')	Originator of the message
content	text	NOT NULL	Message text
metadata	jsonb	None	Model identifier and any graph payload the reply produced

Column	Data type	Constraints	Description
created_at	timestampz	DEFAULT now()	Message timestamp

Table 5.10: Table: public.project_shares

Column	Data type	Constraints	Description
id	uuid	PRIMARY KEY	Unique share record
project_id	uuid	NOT NULL, FK → projects(id) ON DELETE CASCADE	Project being shared
share_token	text	NOT NULL, UNIQUE, random 128-bit hex default	Opaque token embedded in the public URL
created_by	uuid	NOT NULL, FK → auth.users(id) ON DELETE CASCADE	Owner who created the link
created_at	timestampz	DEFAULT now()	Creation timestamp
revoked_at	timestampz	NULL while active	Revocation timestamp

5.6.2 Access Policies, Functions and Indexes

Table 5.11: Row-Level Security Policies

Table	Operations	Policy condition
projects	SELECT, INSERT, UPDATE, DELETE	auth.uid() = user_id
simulation_runs	SELECT, INSERT, DELETE	auth.uid() = user_id
chat_history	SELECT, INSERT, DELETE	auth.uid() = user_id
project_shares	ALL	auth.uid() = created_by
profiles	SELECT, INSERT, UPDATE	auth.uid() = id
realtime.messages	SELECT	Channel topic must be “project:” followed by a project the user owns

The schema holds some objects that carry behaviour as well as the tables and policies. `get_shared_project(token)` is a security-definer function that returns the name, domain and graph of one project when the given token is active. `get_share_status(token)` returns *invalid*, *revoked* or *active*. Anonymous visitors may execute these two functions and nothing else; they never get direct access to any table. `handle_new_user()` is a trigger function that inserts a profile row each time a new authentication user is created. Indexes on `projects(user_id)`, `projects(updated_at DESC)`, `chat_history(project_id)`,

`chat_history(user_id)` and `project_shares(share_token)` keep dashboard and share lookups fast.

5.6.3 Referential Integrity

- `profiles.id` → `auth.users.id` (one-to-one, cascading)
- `projects.user_id` → `auth.users.id` (cascading)
- `simulation_runs.project_id` → `projects.id` (cascading); `simulation_runs.user_id` → `auth.users.id` (cascading)
- `chat_history.project_id` → `projects.id` (cascading); `chat_history.user_id` → `auth.users.id` (cascading)
- `project_shares.project_id` → `projects.id` (cascading); `project_shares.created_by` → `auth.users.id` (cascading)

We applied cascading deletion wherever a child row means nothing without its parent. Because of that, deleting an account is a single operation that really removes the user's data and does not leave orphans behind.

5.7 Architecture / Deployment Diagram

The deployment diagram shows the physical nodes on which the system runs and the protocols that link them.

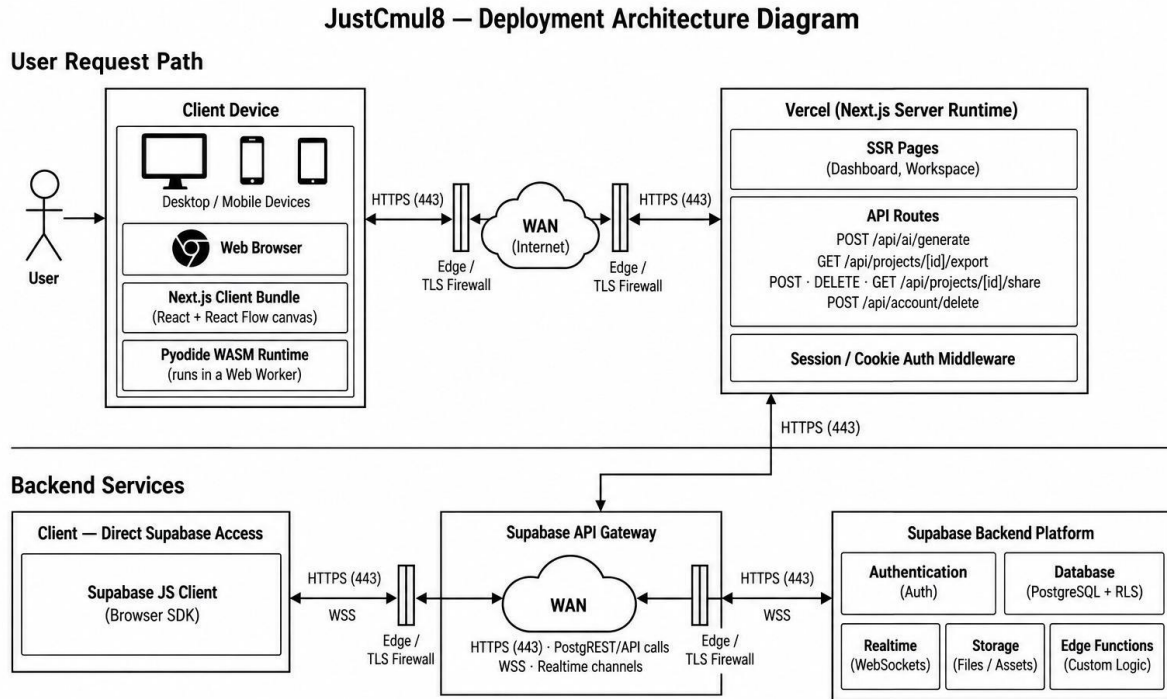


Figure 5.12: Deployment Architecture of JustCmul8

The **client device** hosts a web browser that runs the Next.js client bundle, containing the React application and the React Flow canvas, and beside it the Pyodide WebAssembly runtime executing in a Web Worker. The diagram makes the main architectural point plain: the simulation runtime is deployed on the user’s own device and not in a data centre.

Requests go over HTTPS, through edge TLS protection and the public internet, to the **Vercel** platform, which runs the Next.js server runtime. That runtime serves the dashboard and workspace pages and exposes the API routes (AI generation, chat and optimisation, project export, sharing and account deletion). It also applies session middleware. The **backend services tier** contains the Supabase API gateway, which is reached both from the Next.js server and directly from the browser through the Supabase client. Behind the gateway are authentication, PostgreSQL with Row-Level Security and the realtime WebSocket service.

Letting the browser reach the database directly, and not proxying every read through the application server, is safe only because Row-Level Security is enforced inside PostgreSQL. The server is kept for operations that truly need a secret, chiefly holding the Gemini API key. That keeps the deployment small and, together with client-side simulation, makes the running cost of the platform nearly independent of how heavily it is used.

6. SYSTEM IMPLEMENTATION AND METHODOLOGY

This chapter explains how the design of Chapter 5 became working software. It covers the methodology we followed, the technologies and the key algorithms, how each module and the database were implemented, how the user interface was designed, and the code fragments that show the system best.

6.1 Development Methodology

Implementation followed the iterative and incremental model of Section 3.5. We proved the engine first as a prototype spike and checked it against analytical queueing results, and only then built the modules that consume it. Each module then went through its own cycle of design, coding, unit testing and integration.

- **Version control:** Git with a GitHub remote. We committed in small, described steps, so the history reads as a record of the project.
- **Code review:** whoever did not write a module reviewed it before it was merged.
- **Type safety:** TypeScript throughout, so a mismatch between a node's parameters and the code that uses them is caught at compile time.
- **Separation of concerns:** analytics, code generation, Monte Carlo and export are plain TypeScript modules with no interface dependency, which makes them testable on their own.
- **Verification first:** a separate native harness checks the simulation engine against closed-form queueing results before any interface work relies on its output.
- **Continuous documentation:** module documents were written beside the code and later merged into this report.

6.2 Technologies Used

Section 2.3.3 gave the technology stack with reasons. Here we describe how each technology is used in the implementation.

- **Next.js 16 and React 19** handle routing, server-side rendering of the landing and authentication pages, and the API routes that proxy the AI service and issue share tokens. The workspace itself is a client-rendered single-page application.

- **TypeScript** covers all application and service code. The SimPy program is produced as Python text by a TypeScript module.
- **React Flow** implements the canvas, node handles, edges, viewport and selection, and **Dagre** arranges generated and template graphs.
- **Pyodide** loads CPython compiled to WebAssembly inside a Web Worker. The generated script runs with `runPythonAsync`, and results come back through callback functions registered in the Python global namespace.
- **SimPy** provides the event scheduler, resources, stores, containers and interrupts that the generated model is built on.
- **Zustand** keeps application state in slices, **Recharts** draws the dashboard charts and **Pixi.js** renders the digital twin. **Framer Motion** and **Lenis** give transitions and smooth scrolling on the landing page.
- **Supabase** supplies authentication (email and OAuth), PostgreSQL with Row-Level Security, and Realtime channels for presence and broadcast.
- **Google Gemini** is reached through a model router that keeps a priority list of models and a short-lived health record for each.
- **jsPDF** and **html-to-image** produce the executive report and capture the canvas in the browser.
- **Tailwind CSS v4** styles the interface consistently through utility classes and design tokens.

6.3 Algorithms and Techniques Used

6.3.1 Graph Representation and Validation

The simulation model is a directed graph, held in memory as an **adjacency list**, which maps each node identifier to the list of its outbound neighbours. Simulation graphs are sparse, since a node usually has one or two successors. An adjacency matrix grows with the square of the node count no matter how few edges exist, so it would waste space. The list also suits how the engine works: routing an entity needs the successors of one node, and the list hands them over directly. Before a run, the interface walks the graph from its nodes using a visited set and reports any node that cannot be reached. A disconnected model is then refused with a message that names the node, and does not fail inside the engine.

Table 6.1: Complexity of Graph Operations on the Adjacency List

Operation	Complexity	Where it is used
Add node / add edge	$O(1)$	Placing and connecting nodes on the canvas
Retrieve successors	$O(\deg v)$	Routing an entity onward
Graph traversal	$O(V + E)$	Connectivity validation before a run
Space occupied	$O(V + E)$	Whole-graph representation

6.3.2 Event Scheduling

The event queue is the central structure of any discrete event simulator. It holds events scheduled for future simulated times, and the engine keeps taking out the earliest one and processing it. SimPy builds it as a **binary min-heap** keyed on event time, so inserting and extracting each take logarithmic time in the number of queued events. A sorted list would make extraction constant but insertion linear, and an unsorted list does the reverse. The heap balances the two. Ties in event time are broken by a counter that only increases, so events due at the same instant are handled in the order they were created. JustCmul8 gets this behaviour from SimPy and does not reimplement it, which is exactly the reuse the architecture is meant to allow.

6.3.3 Random Variate Generation

Variability comes from sampling inter-arrival and service times from the five distributions in Table 1.2. In the generated script the exponential distribution uses `random.expovariate(1/mean)`. The normal distribution uses `random.gauss` truncated at zero, since a negative service time has no physical meaning. The uniform distribution spans $\text{mean} \pm 50$ per cent, and the deterministic distribution returns the mean unchanged. Routing at decision nodes uses cumulative-probability sampling: a uniform variate is drawn and compared with the running sum of the configured route probabilities. Source nodes can spread arrivals over several successors in round-robin order.

6.3.4 Metric Computation

Utilisation is found by accumulating busy-seconds. Whenever the number of busy servers changes, the time since the last change is multiplied by the previous busy count and added to a running total. Utilisation is that total divided by capacity times run duration. **Waiting time** is the gap between an entity joining a queue and starting service, and percentiles come from the real per-entity samples. To keep memory bounded on long runs, entity logs are **reservoir-**

sampled: we keep a uniform random sample of whole entity journeys and not just the first part of the run, which would bias the tail percentiles.

The **Little's Law check** compares the time-averaged number of entities in the system with the arrival rate multiplied by the mean time in the system. It reports the percentage discrepancy and classifies the run as steady-state, transient or accumulating backlog. The **bottleneck** is the resource or queue node with the highest average waiting time. The **system health score** is a weighted composite of four components, as set out in Table Table 4.4:

$$\text{Health} = \text{Completion (35\%)} + \text{Utilisation balance (30\%)} + \text{Stability (20\%)} + \text{Tail latency (15\%)}$$

A maximum utilisation above 88 per cent, and again above 95 per cent, reduces the utilisation component step by step. An unstable or inconsistent Little's Law result lowers the stability component, and a 99th-percentile wait more than five times the median lowers the tail component.

6.3.5 Monte Carlo Confidence Intervals

For each metric we compute the mean and the sample standard deviation across replications, and the 95 per cent confidence interval is:

$$CI = \bar{x} \pm 1.96 \times (s / \sqrt{n})$$

where $\bar{x}$ is the sample mean, s the sample standard deviation and n the number of replications. The half-width shrinks with the square root of n , so four times as many replications halve the interval. That is why the interface offers presets of 10, 20, 50 and 100 and not a free-entry field.

6.3.6 AI Model Routing

The AI features go through a **model router** and not a hard-coded model. The router keeps a priority list of Gemini models and sorts every failure into unavailable, rate-limited or empty response. A failing model is marked unhealthy for a short time-to-live, so later requests skip it and go straight to the next candidate. The assistant thus fails over automatically without the user doing anything, and a model that the provider retires does not take the feature down. Generated graphs are validated before display, and every recommendation is put forward for approval.

6.3.7 Presence and Broadcast

Collaboration relies on two channel mechanisms. **Presence** tracks who is in the project: each client publishes a small payload (identifier, name, colour, selected and edited node) and

receives the merged state of everyone else. **Broadcast** carries short-lived messages, namely cursor positions and graph operations, to the other clients, which ignore messages that they sent themselves. Cursor colours are derived from the user identifier, so the same person always shows up in the same colour.

6.4 Module-wise Implementation

The table relates each module to the size of its implementation. Line counts cover the module's source files and come from the delivered code base.

Table 6.2: Module-wise Implementation Summary

Module	Principal implementation	Approx. lines
1 Platform and Data Layer	Auth pages, dashboard, settings, middleware, Supabase clients, account deletion route	$\approx 3,850$ (app/)
2 Visual Graph Editor	NodeCanvas (951), NodePalette, layoutEngine, simTypeRegistry (715)	$\approx 1,900$
3 Node Configuration Panel	NodePropertiesPanel (1,109), shared types (514)	$\approx 1,600$
4 Simulation Engine	codeGenerator (783), pyodideEngine/Worker, clientEngine, CodeInspectorPanel	$\approx 1,300$
5 AI Assistant	AIChatPanel (962), chat and generate routes, systemPrompt, graphOps (739), modelRouter (370)	$\approx 2,750$
6 AI System Optimiser	optimize route (182), flowAnalysis (268)	≈ 450
7 KPI Dashboard and Results	analyticsEngine (806), resolveKpiMetrics, SimResultsPanel (1,047), AdvancedResultsDashboard, eight result tabs	$\approx 4,400$
8 Monte Carlo and Scenarios	monteCarlo (132), monteCarloRunner, MonteCarloPanel (778)	≈ 970
9 Digital Twin Playback	DigitalTwinCanvas (366), PlaybackControls, timelineSelectors	≈ 510
10 Template Gallery	TemplateGallery (135), templates in simTypeRegistry	≈ 850
11 Executive Report and Share	pdfReportGenerator (235), reportGenerator, jupyterExport, share modal, viewer, share/export routes	≈ 700 (excl. API routes)
12 Real-Time Collaboration	usePresence (143), LiveCursor	≈ 160

6.5 Database Implementation

The schema lives in one SQL script that can be run repeatedly, because every statement uses `IF NOT EXISTS` or a guarded block. Tables come first, then Row-Level Security is enabled

on each, policies are added in guarded blocks, and indexes, functions and the profile trigger finish the script. The excerpt shows the projects table and its policies; the other tables follow the same pattern.

Listing 6.1: Projects table and Row-Level Security policies

```
CREATE TABLE IF NOT EXISTS public.projects (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references auth.users(id) on delete cascade,
  name text not null,
  sim_type text not null default 'human_queue',
  graph_json jsonb default '{"nodes": [], "edges": []}':jsonb,
  updated_at timestamptz default now()
);

ALTER TABLE public.projects ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Users can view their own projects"
  ON public.projects FOR SELECT USING (auth.uid() = user_id);
CREATE POLICY "Users can update their own projects"
  ON public.projects FOR UPDATE
  USING (auth.uid() = user_id) WITH CHECK (auth.uid() = user_id);
```

Public sharing never gives anonymous users access to a table. A share token is looked up inside a security-definer function, which returns only the fields needed to draw the project:

Listing 6.2: Read-only public access through a security-definer function

```
CREATE OR REPLACE FUNCTION public.get_shared_project(token text)
RETURNS TABLE (name text, sim_type text, graph_json jsonb) AS $$
  SELECT p.name, p.sim_type, p.graph_json
  FROM public.projects p
  JOIN public.project_shares s ON s.project_id = p.id
  WHERE s.share_token = token AND s.revoked_at IS NULL
  LIMIT 1;
$$ LANGUAGE sql SECURITY DEFINER STABLE;

GRANT EXECUTE ON FUNCTION public.get_shared_project(text)
  TO anon, authenticated;
```

6.6 User Interface Design

The interface rests on three ideas: **plain language**, **low cognitive load** and **familiar interaction**. Words come from the user’s field and not from simulation theory. A bank model offers a “Teller” and a “Waiting Line”, a factory model a “Machine” and a “Buffer”, and the result tabs are named after what they show: *Executive Summary*, *Traffic Heatmap*, *Line Buildup Timeline*, *Station Counters*, *Person / Item Journey*, *Live Activity Log* and *Cost and ROI*. The workspace has three regions. A node palette sits on the left, the canvas in the middle, and on

the right a contextual panel that switches between node configuration, AI chat, results and Monte Carlo analysis depending on what the user is doing.

The landing page states the offer, which is simulation with no installation, no licence and no code, and leads to registration. The visual identity uses a restrained neutral palette with one accent, smooth scrolling and some motion for polish, and a responsive layout keeps the interface usable on tablets and phones. Error messages are written as instructions that name the offending node and say how to fix it.

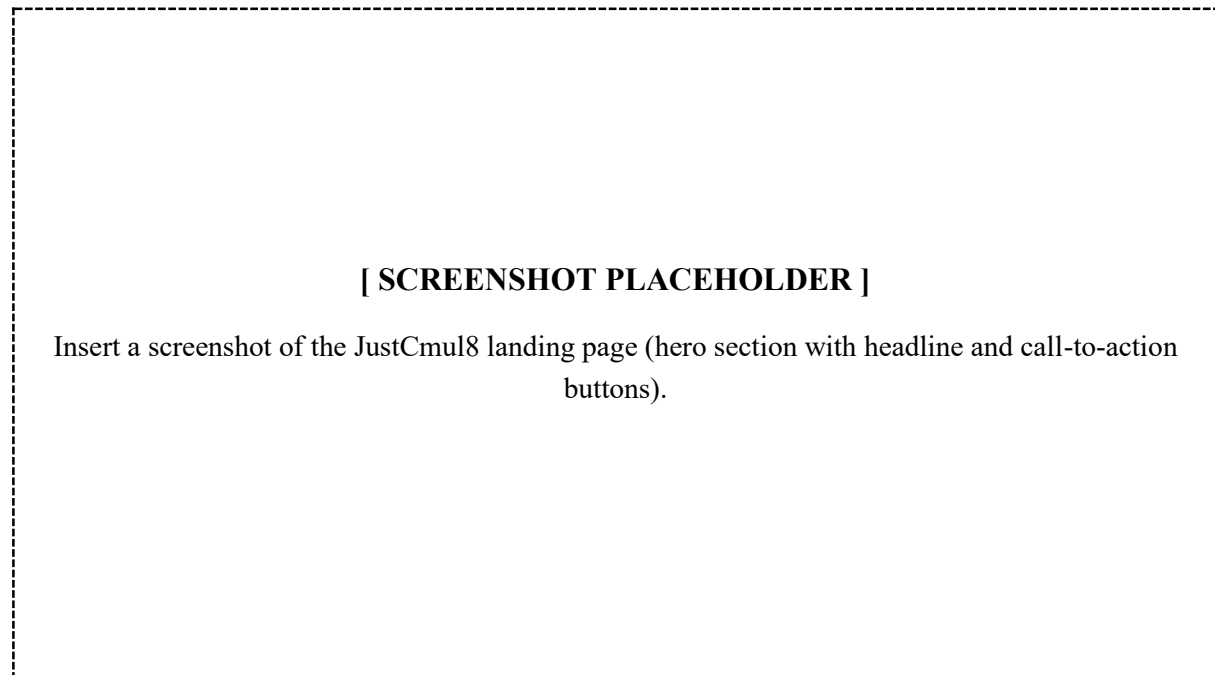


Figure 6.1: Landing Page

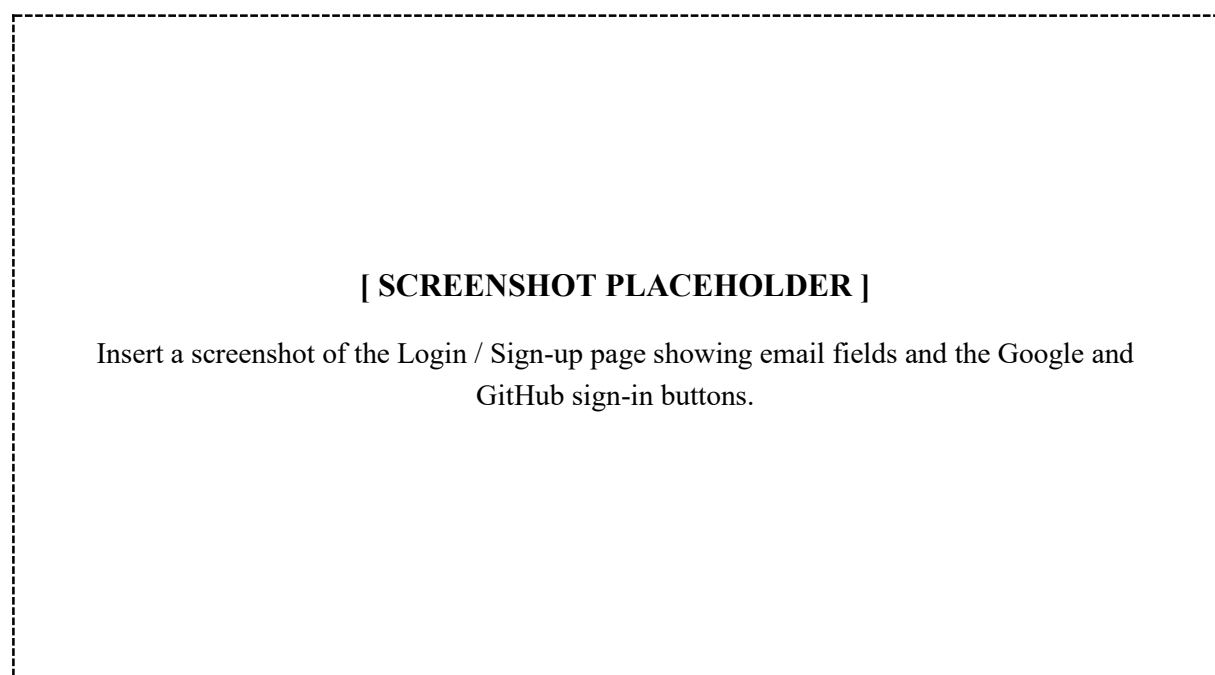


Figure 6.2: Registration and Login Screens

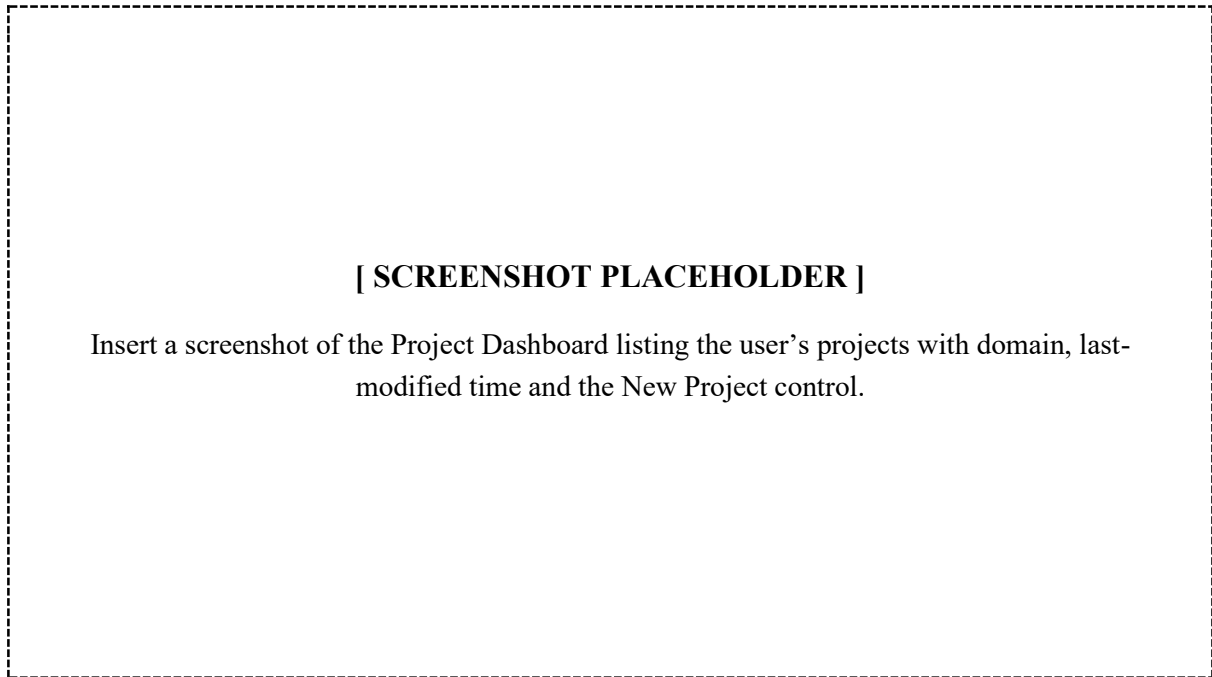


Figure 6.3: Project Dashboard

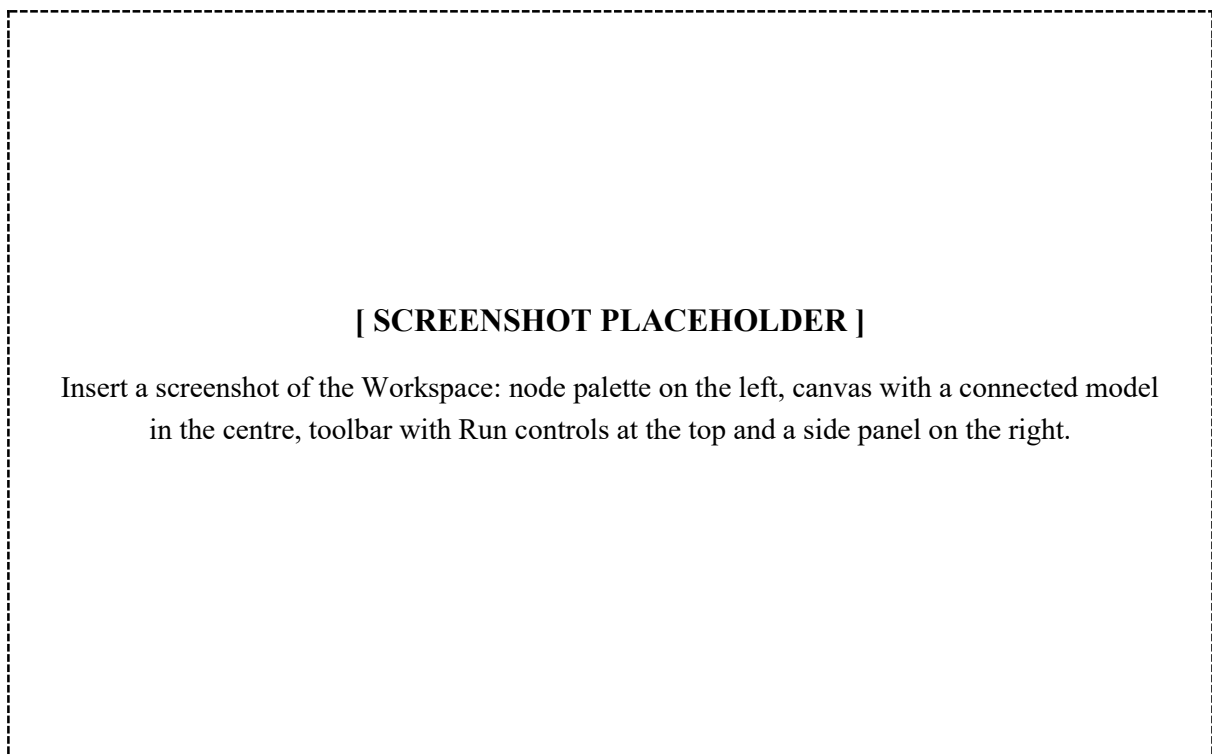


Figure 6.4: Workspace with the Visual Graph Editor

[SCREENSHOT PLACEHOLDER]

Insert a screenshot of the Node Configuration Panel for a Resource node (capacity, service-time distribution, cost parameters).

Figure 6.5: Node Configuration Panel

[SCREENSHOT PLACEHOLDER]

Insert a screenshot of the Template Gallery showing starter models grouped by the six domains.

Figure 6.6: Template Gallery

6.7 Important Code Snippets

Here we reproduce the pieces of code that best show how the system works. Each comes straight from the delivered source and has a short explanation.

6.7.1 Distribution Sampling in the Generated SimPy Script

The generated program has one sampling function that every source and resource calls. It shows how the five distributions of Table 1.2 are realised, including the truncation of the normal distribution at zero.

Listing 6.3: Distribution sampler in the generated script (codeGenerator.ts)

```
def sample(dist_type, mean, std=None):
    if dist_type == "exponential":
        return random.expovariate(1.0 / mean) if mean > 0 else 0
    elif dist_type == "uniform":
        return random.uniform(mean * 0.5, mean * 1.5)
    elif dist_type == "normal":
        s = std if std is not None else mean * 0.3
        return max(0, random.gauss(mean, s))
    elif dist_type == "deterministic":
        return max(0, mean)
    elif dist_type == "poisson":
        return random.expovariate(1.0 / mean) if mean > 0 else 0
```

6.7.2 Resource Service with Busy-Time Accounting

This fragment is the heart of the model: an entity waits, seizes a server, is served for a sampled time and releases the server. Busy time is added up whenever the busy count changes, which makes the reported utilisation exact and not sampled. The comment records a correction we made to the design. The next hop is returned to the caller so that the server is released before the entity moves on, which stops the server being held for the rest of the entity's journey.

Listing 6.4: Resource service and utilisation accounting (excerpt)

```
def do_resource_service(env, eid, node_id, cfg, s, resources,
                       queued_at, queue_stats=None):
    params = cfg["params"]
    wait_time = env.now - queued_at
    s["waits"].append(wait_time)
    if s["busyCount"] > 0:
        s["busy_seconds"] += (env.now - s["last_busy_change"]) * s["busyCount"]
    s["last_busy_change"] = env.now
    s["busyCount"] += 1
    svc_time = sample(params.get("serviceDistribution", "exponential"),
                      params.get("serviceTimeMean", 1))
    yield env.timeout(svc_time)
    s["busy_seconds"] += (env.now - s["last_busy_change"]) * s["busyCount"]
    s["last_busy_change"] = env.now
    s["busyCount"] -= 1
    s["entitiesOut"] += 1
    return get_next_target(node_id) # caller releases the server first
```

6.7.3 Running Python in the Browser

The worker gets a start message, generates the script from the graph, registers two callbacks in Python's global namespace and runs the script. The callbacks are ordinary JavaScript functions, so the Python code can stream ticks and hand over its result with no network traffic.

Listing 6.5: Executing the generated SimPy script in the Web Worker (pyodideWorker.ts)

```
case "start": {
  const { python } = generateSimPyScript(
    params.graph, params.durationSeconds, params.tickIntervalSeconds);

  pyodide.globals.set("emit_sim_tick", (tickJson: string) => {
    if (!stopped) emit(JSON.parse(tickJson));
  });
  pyodide.globals.set("emit_sim_result", (resultJson: string) => {
    if (!stopped) emit(JSON.parse(resultJson));
  });

  await pyodide.runPythonAsync(python); // SimPy runs in WebAssembly
  break;
}
```

6.7.4 Confidence Interval Computation

Listing 6.6: Monte Carlo 95% confidence interval (monteCarlo.ts, abbreviated)

```
export function computeConfidenceInterval(samples: number[]): RunStatistic {
  const n = samples.length;
  const mean = samples.reduce((a, b) => a + b, 0) / n;
  const variance =
    samples.reduce((a, b) => a + (b - mean) ** 2, 0) / (n - 1);
  const stdDev = Math.sqrt(variance);
  const marginOfError = 1.96 * (stdDev / Math.sqrt(n));
  return {
    mean,
    ciLower: Math.max(0, mean - marginOfError),
    ciUpper: mean + marginOfError,
    stdDev, samples,
  };
}
```

6.7.5 System Health Score

Listing 6.7: Composite system health score (analyticsEngine.ts, abbreviated)

```
const completionScore = Math.min(1, completed / arrived) * 35;

let utilScore = 30; // utilisation balance
if (maxUtil > 0.95) utilScore = 15; // severe bottleneck
else if (maxUtil > 0.88) utilScore = 22;

let stabilityScore = 20; // Little's Law consistency
if (!littlesLaw.isStable) stabilityScore = 5;
else if (littlesLaw.discrepancyPercent > 20) stabilityScore = 12;
```

```
let tailScore = 15;                                // tail latency
if (waitStats.p99 > waitStats.p50 * 5) tailScore = 8;

return clamp(Math.round(completionScore + utilScore +
                        stabilityScore + tailScore), 10, 100);
```

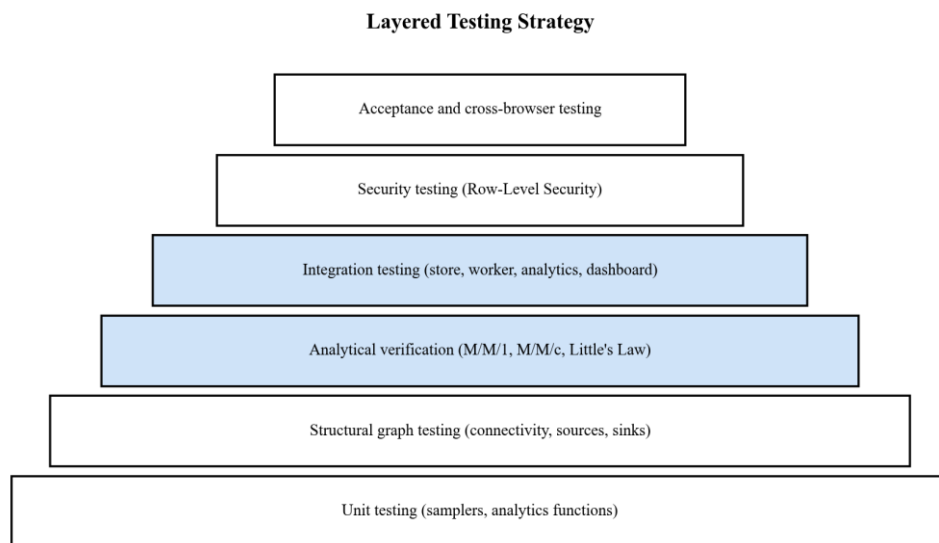
Taken together, the fragments show the shape of the platform: a small, readable SimPy program produced from a diagram, a worker that runs it without touching the network, analytics that turn its output into checkable indicators, and a database that decides who may see what.

7. SYSTEM TESTING

A simulator can be easy to use and still give wrong answers, so testing carries extra weight in this project. Interface tests can show that a button works, but only a comparison with independent mathematics can show that the numbers behind it are right. We therefore tested in two halves: verification of the **engine** against analytical results and invariants, and functional testing of the **application** around it. The chapter describes the strategy, the units under test, the test cases, the approach and the results.

7.1 Testing Strategy

Testing ran beside development and not after it, for the reason given in Chapter 3: a defect in an early module is cheaper to fix while the modules built on it are still being written. The strategy is layered, with fast, narrow tests at the bottom and broader, slower checks above them.



Lower layers are cheap and run constantly; upper layers are broader and run before each release

Figure 7.1: Layered Testing Strategy for JustCmul8

- **Unit testing** runs the pure TypeScript modules, namely the samplers and the analytics functions, in isolation.
- **Structural graph testing** checks that invalid models (no blocks, no connections, unreachable nodes) are refused before execution, with a message that names the fault.
- **Analytical verification** is the most valuable layer. The engine's output for reference queues is compared with closed-form M/M/1 and M/M/c results and with Little's Law.

- **Invariant testing** applies conservation and flow-balance laws to a complex multi-stage model that has no closed-form answer.
- **Integration testing** covers the paths that cross module boundaries: store, code generator, worker, analytics and dashboard.
- **Security testing** attacks authorisation directly at the database and bypasses the interface.
- **Cross-browser and acceptance testing** checks that behaviour is the same across browsers and that a first-time user gets a result without a tutorial.

7.2 Testable Units

Table 7.1: Testable Units and the Techniques Applied to Them

Unit	Location	What is tested	Technique
Distribution samplers	codeGenerator.ts (generated sample), distributions.ts	Mean, spread and truncation of each of the five distributions	Statistical unit test
Code generator	lib/simulation/codeGenerator.ts	Every node type produces valid, executable SimPy	Execution of generated script
Analytics engine	lib/simulation/analyticsEngine.ts	Utilisation, waits, percentiles, Little's Law, health score	Known-answer test; analytical oracle
Monte Carlo aggregation	lib/simulation/monteCarlo.ts	Mean, standard deviation and 95% interval	Known-answer test
Graph validation	NodeCanvas.tsx (validateGraphConnectivity)	Empty, unconnected and partially connected graphs	Boundary and negative testing
Graph operations	lib/ai/graphOps.ts	Validation of AI-generated graphs	Negative testing with malformed input
Model router	lib/ai/modelRouter.ts	Fail-over between models on error classes	Fault injection
Database policies	supabase/schema.sql	Row-Level Security and share functions	Direct-request security testing
Presence channel	lib/realtime/usePresence.ts	Join authorisation, presence, cursor broadcast	Multi-session integration test
Export modules	pdfReportGenerator.ts, jupyterExport.ts	Content of PDF, CSV, JSON and notebook exports	Output inspection

7.3 Test Case Design with Reference to the Project

We designed two families of test cases. **Engine verification cases** take their expected results from queueing theory and produce numerical outcomes. **Functional test cases** for the application state expected behaviours that trace back to the requirements of Section 2.1.

7.3.1 Engine Verification Cases

The verification suite runs the generated SimPy script verbatim, with five independent replications per benchmark over 36,000 simulated seconds, and compares the results with theory. B1 is an M/M/1 queue at utilisation 0.8. B1v repeats it without an explicit queue node. B2 is an M/M/3 queue, and B3 is a deliberately saturated queue. B4 checks how time units are handled, and B5 checks Little's Law. Each deviation is judged against a tolerance stated in advance.

Table 7.2: Engine Verification Test Cases and Results

ID	Benchmark	Metric	Theory	Simulated	Dev.	Tol.	Result
E-01	B1 M/M/1	Utilisation ρ	0.8000	0.7979	-0.27%	$\pm 3.5\%$	Pass
E-02	B1 M/M/1	Queue wait W_q	4.000	4.005	+0.13%	$\pm 3.5\%$	Pass
E-03	B1 M/M/1	System time W	5.000	5.004	+0.08%	$\pm 3.5\%$	Pass
E-04	B1 M/M/1	Queue length L_q	3.200	3.188	-0.39%	$\pm 3.5\%$	Pass
E-05	B1 M/M/1	Percentiles from real samples (4,000 points)	real	real	n/a	n/a	Pass
E-06	B1 M/M/1	WIP conservation (arrived – completed)	3.8	3.8	n/a	n/a	Pass
E-07	B1 M/M/1	Source node arrivals reported	28,584	28,584	n/a	n/a	Pass
E-08	B1v (no queue)	Utilisation ρ	0.8000	0.7979	-0.27%	$\pm 3.5\%$	Pass
E-09	B1v (no queue)	Queue wait W_q	4.000	4.005	+0.13%	$\pm 3.5\%$	Pass
E-10	B1v (no queue)	System time W	5.000	5.004	+0.08%	$\pm 3.5\%$	Pass
E-11	B1v (no queue)	System WIP L	4.000	3.980	-0.51%	$\pm 3.5\%$	Pass
E-12	B2 M/M/3	Utilisation per server	0.8000	0.7983	-0.21%	$\pm 4.0\%$	Pass
E-13	B2 M/M/3	Queue wait W_q (Erlang-C)	1.079	1.101	+2.09%	$\pm 4.0\%$	Pass
E-14	B2 M/M/3	System time W	2.079	2.100	+1.03%	$\pm 4.0\%$	Pass
E-15	B3 saturated	Total arrivals	1,200	1,181	-1.55%	$\pm 5.0\%$	Pass
E-16	B3 saturated	Total completed	600	603.2	+0.53%	$\pm 5.0\%$	Pass
E-17	B3 saturated	Utilisation ρ	1.000	0.9968	-0.32%	$\pm 2.0\%$	Pass
E-18	B3 saturated	Final WIP within 600 ± 70	530–670	578.2	n/a	2σ	Pass
E-19	B3 saturated	Little's Law verdict (accumulating backlog)	backlog	backlog	n/a	n/a	Pass
E-20	B3 saturated	Health score degrades below 70	< 70	45.8	n/a	n/a	Pass

ID	Benchmark	Metric	Theory	Simulated	Dev.	Tol.	Result
E-21	B4 time units	Run horizon identical in seconds, minutes, hours	3,600	3,600 / 3,600 / 3,600	n/a	n/a	Pass
E-22	B4 time units	Tick interval identical across units	3.6	3.6 / 3.6 / 3.6	n/a	n/a	Pass
E-23	B4 time units	Arrival-count spread across units	< 1%	0.000%	n/a	n/a	Pass
E-24	B4 time units	Tick granularity for a 5 s run	≤ 0.1 s	0.05 s (100 ticks)	n/a	n/a	Pass
E-25	B5 Little's Law	Discrepancy between L and λW	< 5%	2.41%	n/a	n/a	Pass
E-26	B5 Little's Law	Time-averaged true WIP	4.000	3.982	-0.46%	$\pm 8.0\%$	Pass
E-27	B5v (no queue)	Little's Law discrepancy	< 5%	2.41%	n/a	n/a	Pass
E-28	B5v (no queue)	Time-averaged true WIP	4.000	3.982	-0.46%	$\pm 8.0\%$	Pass

7.3.2 Invariant Cases for a Complex Multi-Stage Model

Benchmark B6 is a hospital-flow model with a triage station and a probabilistic decision node that sends 70 per cent of patients to minor treatment and 30 per cent to major treatment, ending in two sinks. We ran it with and without patient abandonment (reneging), at an arrival rate of 1.2 per second over 7,200 simulated seconds. No closed-form answer exists for it, so the test asserts **invariants**, which are laws that every correct simulation must obey.

Table 7.3: Invariants Asserted for the Complex Model (Benchmark B6)

ID	Invariant	Meaning
I1	Conservation residual ≥ 0 and below 5% of arrivals	Arrived = completed + reneged + dropped + stuck; nothing is created or lost
I2	Flow balance at every node	Entities in = entities out + current depth
I3	Decision split equals the configured probability	70/30 routing is honoured (observed 0.700 with abandonment on, 0.687 off)
I4	Total completed equals the sum over all sinks	Sink accounting is complete
I5	Throughput clears offered load ($\geq 95\%$)	Stable stations serve what they are offered
I6	Per-station utilisation approximates ρ	Triage ≈ 0.72 , minor ≈ 0.70 , major ≈ 0.72

7.3.3 Functional Test Cases

The functional cases below cover every module, and each one names the requirement it verifies. The **Status** column is for the tester to fill in as each case is run (Pass or Fail, with the date).

Table 7.4: Functional Test Case Design

ID	Module	Test scenario and input	Expected output	Req.	Status
TC-01	Platform	Register with a valid, unused email and a compliant password	Account created, profile row inserted, session started	FR-01	
TC-02	Platform	Register with an already-registered email	Registration refused with a clear message	FR-01	
TC-03	Platform	Register with a weak password	Submission blocked; requirement shown	FR-01	
TC-04	Platform	Log in with correct credentials	Session issued; dashboard loads the user's projects	FR-01	
TC-05	Platform	Log in with a wrong password	Access refused without revealing which field was wrong	FR-01	
TC-06	Platform	Sign in with Google and with GitHub OAuth	Account linked; session established	FR-01	
TC-07	Platform	Open a workspace URL while logged out	Redirected to login before any data is fetched	FR-02	
TC-08	Platform	Request a password reset and set a new password	Email link works; new password accepted	FR-03	
TC-09	Platform	Create a project in a chosen domain	Project appears on the dashboard with the domain	FR-04	
TC-10	Platform	Rename and delete a project	Name changes; deletion removes project and its runs and chats	FR-04	
TC-11	Platform	Move a node, then reload the page	Position persisted without an explicit save	FR-05	
TC-12	Platform	Query another user's project by identifier	Row-Level Security returns no rows	NFR-05	
TC-13	Platform	Delete the account after confirmation	User row and all owned data removed	FR-06	
TC-14	Graph Editor	Place each of the fifteen node types	Every node renders with correct icon, handles and defaults	FR-07	
TC-15	Graph Editor	Connect two nodes and move one	Edge created and re-routes live	FR-08	
TC-16	Graph Editor	Delete a mid-graph node	Node and incident edges removed together	FR-07	
TC-17	Graph Editor	Run an empty canvas / an unconnected graph	Run refused; message explains the fault	FR-09	
TC-18	Graph Editor	Run a graph with a node unreachable from the rest	Run refused; the disconnected node is named	FR-09	
TC-19	Node Config	Enter capacity 0 or a negative value	Value rejected with a corrective message	FR-11	
TC-20	Node Config	Switch the service distribution	Parameter fields change to match	FR-10	

ID	Module	Test scenario and input	Expected output	Req.	Status
TC-21	Node Config	Enter routing probabilities not summing to one	Warning shown before the run	FR-11	
TC-22	Engine	Run a valid single-server model	Run completes; arrivals, completions and KPIs reported	FR-12	
TC-23	Engine	Run the same model with duration in seconds, minutes and hours	Equivalent horizons produce equivalent results	FR-13	
TC-24	Engine	Interact with the canvas during a long run	Interface remains responsive	FR-14	
TC-25	Engine	Stop a run part-way; change playback speed	Run halts; speed changes apply	FR-15	
TC-26	Engine	Open the code inspector and export the notebook	Readable SimPy shown; .ipynb downloads and runs	FR-25	
TC-27	AI Assistant	Describe “a clinic with two doctors and a waiting room”	Valid graph proposed with source, queue, resource, sink	FR-16	
TC-28	AI Assistant	Inject a response with an unknown node type	Candidate rejected by validation; not displayed	FR-17	
TC-29	AI Assistant	Generate, then discard the preview	Project remains unmodified	FR-17	
TC-30	AI Assistant	Make the preferred model unavailable	Router falls back to the next model	FR-17	
TC-31	AI Optimiser	Diagnose a deliberately under-provisioned server	Correct node identified with a capacity recommendation	FR-18	
TC-32	AI Optimiser	Apply the suggested fix	Graph updated as an ordinary edit	FR-18	
TC-33	KPI Dashboard	Three-stage line with one slow stage	Slow stage flagged as bottleneck	FR-20	
TC-34	KPI Dashboard	Open the Flow and Balance Check for a stable model	Little’s Law discrepancy shown with a steady-state verdict	FR-20	
TC-35	KPI Dashboard	Enter cost parameters and open Cost and ROI	Total cost and the return of a change are reported	FR-21	
TC-36	Monte Carlo	Run 20 replications	Mean, standard deviation and 95% interval per metric	FR-22	
TC-37	Monte Carlo	Apply a fix, then compare baseline and scenario	Per-metric difference shown with both intervals	FR-22	
TC-38	Digital Twin	Play back a completed run; scrub and change speed	Entities animate; position changes without re-running	FR-23	
TC-39	Templates	Instantiate one template from each domain	Canvas pre-populated; every template runs unmodified	FR-24	
TC-40	Templates	Instantiate, then modify the new project	Source template unchanged	FR-24	
TC-41	Export / Share	Export graph as JSON and results as CSV	Files download with correct content	FR-25	
TC-42	Export / Share	Generate the executive PDF report	PDF contains summary, canvas snapshot and KPI tables	FR-25	
TC-43	Export / Share	Open a share link in a private window	Project renders read-only with no editing controls	FR-26	
TC-44	Export / Share	Attempt a direct write using only a share token	Refused by the database	FR-26	

ID	Module	Test scenario and input	Expected output	Req.	Status
TC-45	Export / Share	Open a revoked share link	Explanatory notice shown instead of an error	FR-26	
TC-46	Collaboration	Open one project in two sessions	Presence and live cursors appear in both	FR-27	
TC-47	Collaboration	Try to join a project channel without access	Channel not joined	FR-27	
TC-48	Cross-browser	Run one model on Chrome, Edge, Firefox and Safari	Consistent metrics; canvas and playback render correctly	NFR-08	
TC-49	Responsiveness	Load the workspace at a phone viewport	Layout adapts; panels remain reachable	NFR-08	
TC-50	Profile	Change display name and default domain	Values persist and apply to new projects	FR-06	

7.4 Test Approach

- **Oracle-based testing.** For the engine, expected values come from mathematics and not from an earlier run: closed-form M/M/1 and Erlang-C results, Little's Law and conservation laws. This avoids a test that only records whatever the code happens to do today.
- **Replication and confidence.** Every analytical benchmark runs five independent replications and reports a 95% interval, so a pass reflects how the engine behaves and not the luck of one run.
- **Tolerance discipline.** Tolerances were fixed beforehand and differ by benchmark: tighter for utilisation, looser for waiting times, which vary more.
- **Invariant testing.** Where no closed-form answer exists, we assert laws that must always hold.
- **Negative and boundary testing.** Zero and negative parameters, zero duration, empty graphs, unreachable nodes and malformed AI responses were fed in on purpose.
- **Adversarial security testing.** Authorisation was tested with requests sent straight to the database, skipping the interface: reading another user's project, writing with only a share token, and joining a channel without access.
- **Cross-environment testing.** One model is run on several browsers and the metrics compared, and layout is checked at phone widths.
- **User acceptance testing.** People matching the personas of Section 2.1.7 are asked to build and run a model without instructions. The aim is to confirm that a first result is reachable without a tutorial, which is the real test of the no-code claim.

- **Defect management.** Each defect is logged as an issue in the project tracker, fixed, re-tested and linked to the change that closed it.

Entry and exit criteria. A module enters integration testing once its unit tests pass and it has been reviewed. The build leaves testing when every engine verification case passes, no high-severity defect is open, and every functional test case has been run and recorded.

7.5 Test Results and Validation

7.5.1 Engine Verification Results

All **28 of 28** analytical checks passed. In the M/M/1 benchmark the simulated utilisation was 0.7979 against a theoretical 0.8000, the mean queue wait 4.005 s against 4.000 s and the mean system time 5.004 s against 5.000 s. The 95% confidence interval of each estimate contained the theoretical value. In the three-server benchmark the simulated queue wait was 1.101 s against an exact Erlang-C value of 1.079 s, a deviation of 2.09 per cent and within the ± 4 per cent tolerance. In the hospital-flow model all **22 of 22** invariants held, with and without patient abandonment. No deviation from theory exceeded 2.1 per cent.

Two things from this exercise are worth recording. The exact Erlang-C probability of waiting for three servers at an offered load of 2.4 is 0.6472, which gives an exact waiting time of 1.0787 s, so we scored the benchmark against the exact value and not against a rounded figure from a planning brief. And the saturated benchmark showed that the analytics react correctly to overload: the health score fell to 45.8 and the Little's Law verdict became *accumulating backlog*, so a failing system is reported as failing.

7.5.2 Edge-Case Probes

We also ran a set of edge-case probes to see how the engine behaves at the limits of its inputs.

Table 7.5: Edge-Case Probes and Observed Behaviour

Probe	Observed behaviour
Zero service time	Run completes; 39 completed with utilisation 0.000, as expected
Zero duration	Rejected by the engine with a clear error
Zero arrival rate	Recorded for input-validation hardening: the sampler falls back to a default rate
Recurring schedule at time zero	A guard prevents an unbounded zero-delay loop
Very long run (28,714 entities)	Completed in 3.65 s; log retention capped at 20,000 events using an entity reservoir

Probe	Observed behaviour
Floating-point accumulation over 36,000 ticks	Simulated clock reached exactly 36,000.0
Multiple sinks	Both sinks counted (240 + 240 = 480 completed)
Two resources in series ($\rho = 0.5$ each)	Backlog of 2 over 1,768 arrivals; the first stage remains seized while the entity is in the second stage (blocking after service), which the model documents

7.5.3 Validation against Requirements

Validation asks whether we built the right system. The traceability table links each group of requirements to the evidence behind it.

Table 7.6: Requirements Traceability

Requirement group	Evidence
FR-01 – FR-06 Platform	Functional cases TC-01 – TC-13, TC-50; database policies and cascade rules in schema.sql
FR-07 – FR-11 Editor and configuration	Cases TC-14 – TC-21; graph validation module
FR-12 – FR-15 Engine	Engine cases E-01 – E-28; functional cases TC-22 – TC-26
FR-16 – FR-18 AI	Cases TC-27 – TC-32; graph validation and model router
FR-19 – FR-21 Analytics	Engine cases E-01 – E-28; invariants I1 – I6; cases TC-33 – TC-35
FR-22 Monte Carlo	Cases TC-36 – TC-37; confidence-interval unit test
FR-23 – FR-24 Playback and templates	Cases TC-38 – TC-40
FR-25 – FR-26 Export and share	Cases TC-41 – TC-45; security-definer share functions
FR-27 Collaboration	Cases TC-46 – TC-47; channel authorisation policy
NFR-03 Correctness	All 28 analytical checks within tolerance; 22 of 22 invariants
NFR-05 Access control	TC-12, TC-44, TC-47

8. RESULTS AND DISCUSSION

This chapter shows what the system produces and what the results mean. It starts with the outputs a user sees, then analyses the numerical results from the verification exercise, and ends with an evaluation of performance.

8.1 System Output

The delivered interface is documented below screen by screen, in the order a user meets the results of a run. Each entry says what the screen shows, and a placeholder marks where the matching screenshot from the running application should go.

When a run finishes, the **Executive Summary** shows the headline figures (arrivals, completions, throughput, the flagged bottleneck and the 0–100 system health score) with a plain-English diagnosis and suggested actions. A non-specialist can read the answer without having to interpret a chart.

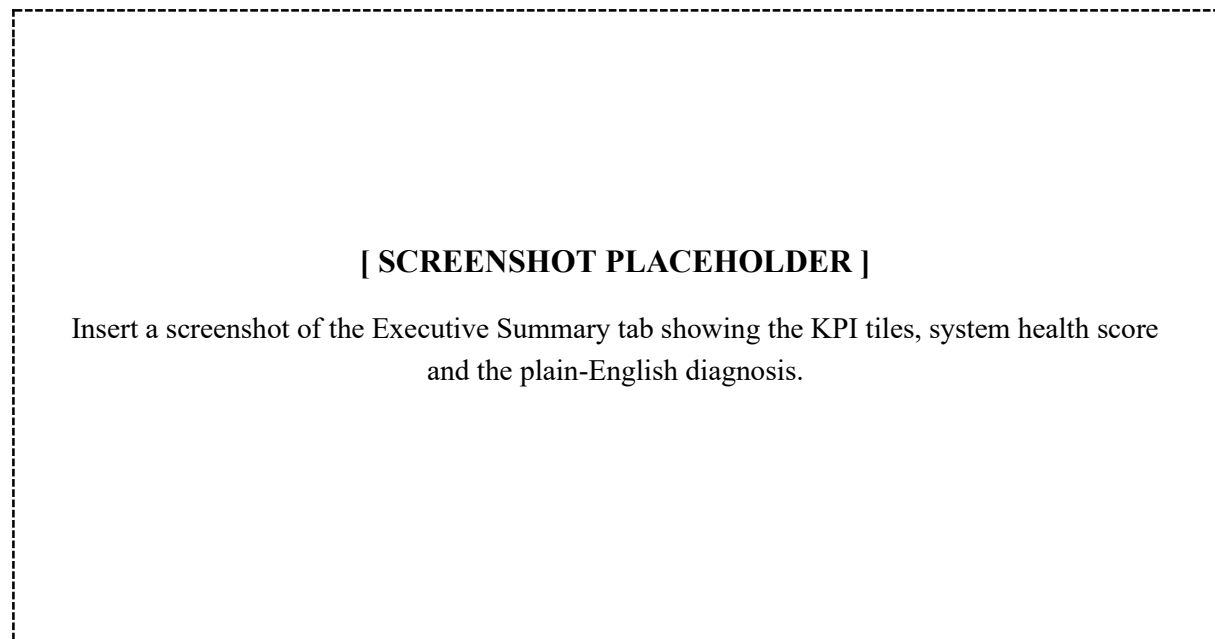


Figure 8.1: Executive Summary Tab of the Results Dashboard

The **Flow and Balance Check** applies Little's Law and a conservation check to see whether the run is consistent with itself. It reports the gap between L and λW and labels the system steady-state, transient or accumulating backlog.

[SCREENSHOT PLACEHOLDER]

Insert a screenshot of the Flow & Balance Check tab showing the Little's Law verification and stability verdict.

Figure 8.2: Flow and Balance Check (Little's Law Verification)

The **Traffic Heatmap** lays the load carried by each node and edge over the model, so congested stations can be spotted at a glance.

[SCREENSHOT PLACEHOLDER]

Insert a screenshot of the Traffic Heatmap tab showing the model with congestion highlighted.

Figure 8.3: Traffic Heatmap

The **Line Buildup Timeline** plots queue length against simulated time and shows whether a queue is stable, cyclical or growing without limit.

[SCREENSHOT PLACEHOLDER]

Insert a screenshot of the Line Buildup Timeline tab showing queue length against simulated time.

Figure 8.4: Line Buildup Timeline

Station Counters list the utilisation, average and percentile waiting times, throughput and queue depth of every node.

[SCREENSHOT PLACEHOLDER]

Insert a screenshot of the Station Counters tab showing per-node utilisation, waiting time and throughput.

Figure 8.5: Station Counters

The **Person / Item Journey** tab follows one entity through the model and lists each node it visited with the time it waited and was served. It is the tool for tracing an odd figure back to the events that produced it.

[SCREENSHOT PLACEHOLDER]

Insert a screenshot of the Person / Item Journey tab tracing one entity through the model.

Figure 8.6: Person / Item Journey (Entity Tracer)

The **Cost and ROI** tab puts the run into money: resource cost over busy time, holding cost weighted by queue length, and the waiting penalty. When a change has been proposed, it says whether the saving repays the cost.

[SCREENSHOT PLACEHOLDER]

Insert a screenshot of the Cost & ROI tab showing resource, holding and waiting cost.

Figure 8.7: Cost and Return-on-Investment Panel

The **Monte Carlo** panel runs 10, 20, 50 or 100 replications and shows progress as each one finishes. Every metric is drawn as a mean with a shaded 95 per cent confidence band, and the baseline-versus-scenario view sets two configurations side by side.

[SCREENSHOT PLACEHOLDER]

Insert a screenshot of the Monte Carlo panel showing confidence-interval bands and the baseline-versus-scenario comparison.

Figure 8.8: Monte Carlo Analysis with 95% Confidence Bands

The **Digital Twin** replays the run as moving entities on a WebGL canvas. Queue congestion is highlighted, and controls allow play, pause, step, scrub and speed changes.

[SCREENSHOT PLACEHOLDER]

Insert a screenshot of the Digital Twin playback with entities moving and the transport controls visible.

Figure 8.9: Digital Twin Animated Playback

The **AI Assistant** takes a description in plain English and returns a candidate graph as a preview. The user has to accept it before the project changes. The same panel answers questions about simulation concepts and about the user's own model.

[SCREENSHOT PLACEHOLDER]

Insert a screenshot of the AI Assistant panel with a plain-English prompt and the generated graph preview.

Figure 8.10: AI Assistant Generating a Model from a Description

The **AI Optimiser** says which node is holding the system back and why, in terms of the computed statistics. Its Apply Fix action carries out the proposed change as an ordinary graph edit.

[SCREENSHOT PLACEHOLDER]

Insert a screenshot of the AI Optimiser showing the diagnosis and the Apply Fix action.

Figure 8.11: AI System Optimiser Recommendation

The **Python Code Inspector** shows the SimPy script generated from the current graph, so any result can be reproduced independently. One action exports it as a Jupyter notebook.

[SCREENSHOT PLACEHOLDER]

Insert a screenshot of the Python Code Inspector showing the generated SimPy script and the notebook export button.

Figure 8.12: Python Code Inspector

In a shared session, each collaborator's **cursor** is drawn in their own colour with their name, and a presence row shows who is in the project.

[SCREENSHOT PLACEHOLDER]

Insert a screenshot of two browser windows in the same project showing live cursors and the presence avatars.

Figure 8.13: Real-Time Presence and Live Cursors

The **Export dialog** offers the graph as JSON, results as CSV, the script as a notebook and the full executive report as PDF. It also manages the public share link, with controls to create, copy and revoke it.

[SCREENSHOT PLACEHOLDER]

Insert a screenshot of the Export and Share dialog and the first page of a generated PDF executive report.

Figure 8.14: Export, Share and Executive PDF Report

A visitor who opens a share link without an account sees the model read-only, with every editing control missing. If the link has been revoked, an explanatory notice replaces the project.

[SCREENSHOT PLACEHOLDER]

Insert a screenshot of the read-only shared view opened from a public link (and, if possible, the revoked-link notice).

Figure 8.15: Read-Only Shared View

The **Settings** screen handles the display name and avatar, password change, the default simulation domain for new projects, and account deletion, which needs confirmation and cascades through all owned data.

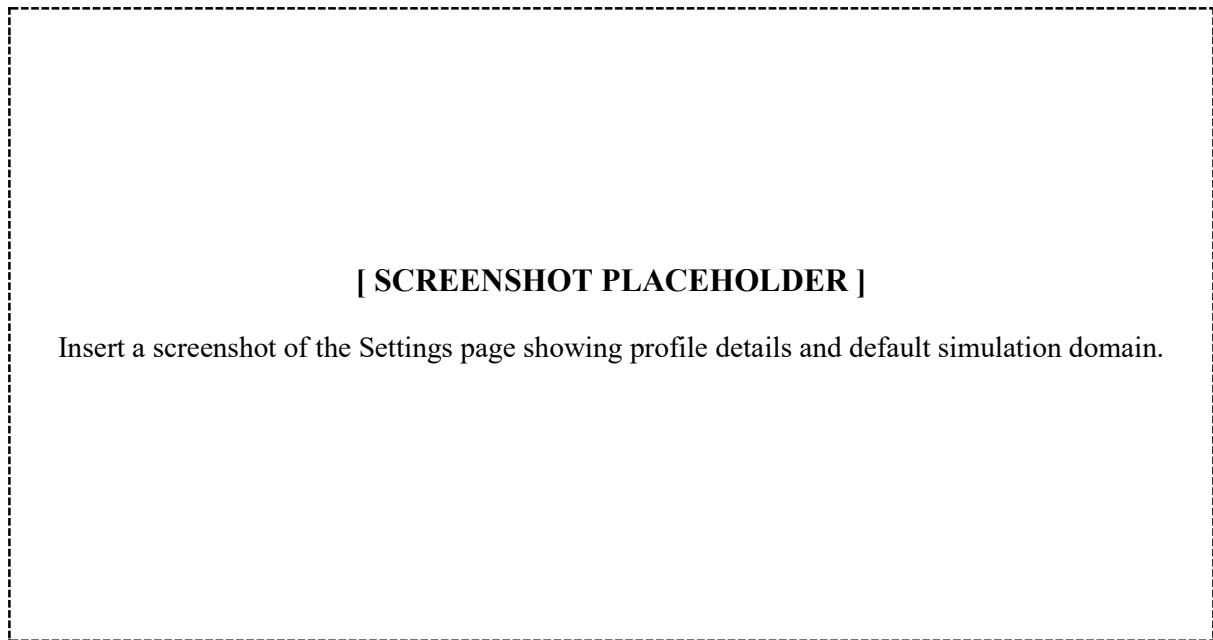


Figure 8.16: Profile and Settings

8.2 Result Analysis

8.2.1 Agreement with Queueing Theory

The key question for any simulator is whether it is right. The figure compares simulated results with the analytical predictions for the M/M/1 and M/M/3 reference queues. In every case the two bars look the same at the scale of the plot.

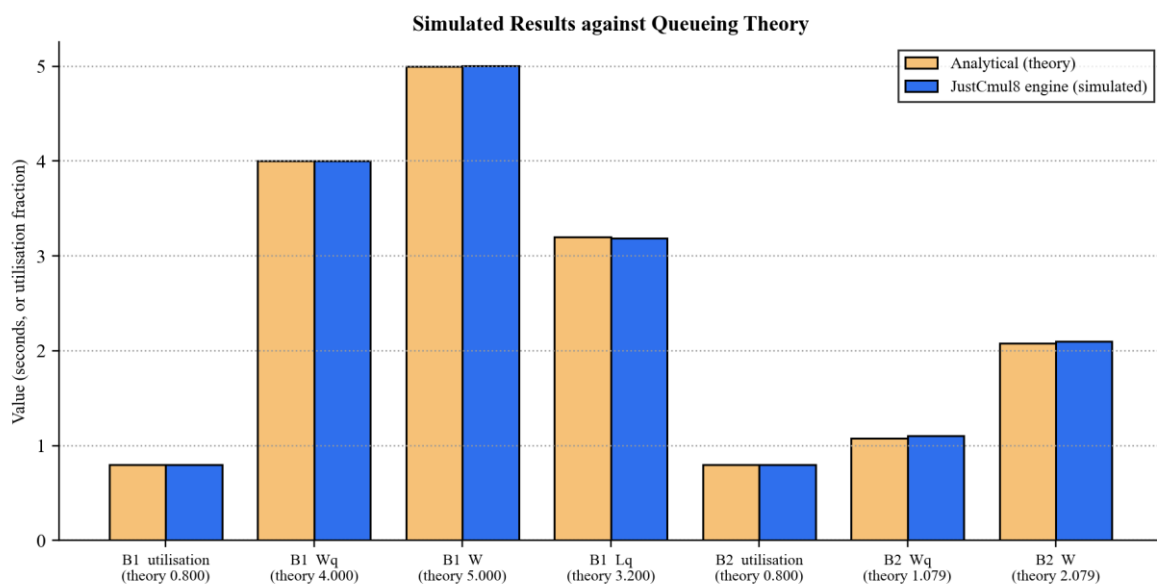


Figure 8.17: Simulated Results against Queueing Theory (M/M/1 and M/M/3 benchmarks)

The second figure gives the deviation of each of the fifteen main checks as a percentage of the theoretical value, next to the tolerance allowed. The biggest deviation, +2.09 per cent for the M/M/3 queue wait, is about half of its tolerance, and the M/M/1 deviations are all below 0.6 per cent.

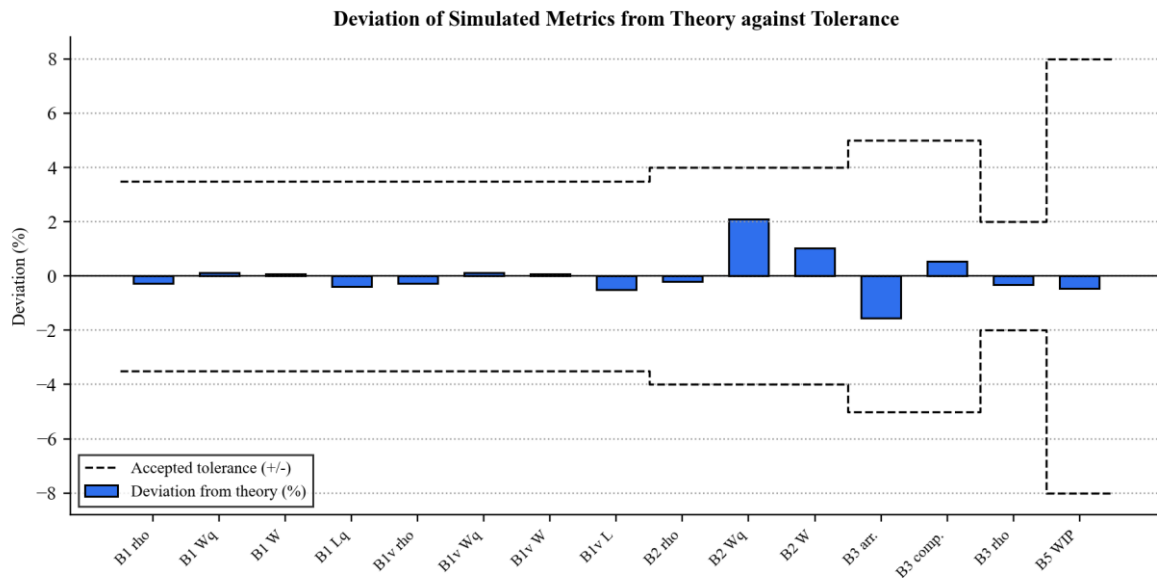


Figure 8.18: Deviation of Simulated Metrics from Theory against Accepted Tolerance

Table 8.1: Worked Example: M/M/1 Queue at $\rho = 0.8$ ($\lambda = 0.8$, $\mu = 1.0$, 36,000 s, 5 replications)

Metric	Formula	Theory	Simulated	95% CI ($\pm$)
Utilisation ρ	λ / μ	0.800	0.798	0.0056
Mean wait in queue W_q	$\rho / (\mu - \lambda)$	4.000 s	4.005 s	0.173
Mean time in system W	$1 / (\mu - \lambda)$	5.000 s	5.004 s	0.174
Mean queue length L_q	$\rho^2 / (1 - \rho)$	3.200	3.188	0.183

8.2.2 A Multi-Stage Model: Hospital Flow

Benchmark B6 puts the platform through a realistic scenario. Patients arrive at 1.2 per second and pass through a triage nurse. A decision node then sends 70 per cent to minor treatment and 30 per cent to major treatment, and each station on its own is stable with utilisation near 0.7. The table sums up the results with and without patient abandonment.

Table 8.2: Results for the Multi-Stage Hospital-Flow Model (7,200 s, 3 seeds)

Measure	Abandonment on	Abandonment off
Patients arrived	8,683	8,541

Measure	Abandonment on	Abandonment off
Patients served	8,301	8,531
Patients who abandoned	374	0
Throughput clearance across seeds	0.956, 0.958, 0.954	0.999, 0.999, 1.000
Utilisation: triage / minor / major	0.705 / 0.690 / 0.697	0.708 / 0.682 / 0.726
Decision split to minor treatment	0.700	0.687
Little's Law discrepancy and verdict	3.2%, steady state	0.2%, steady state
System health score	91	93
Invariants satisfied	11 of 11	11 of 11

This is the kind of result a decision-maker needs. With abandonment on, 4.3 per cent of patients leave unserved, yet the system stays stable, and the health score of 91 describes a well-balanced service that is not perfect. With abandonment off, the service clears almost all of the offered load. The same model can be run again under a proposed change, for instance a third triage nurse, and compared through the Monte Carlo scenario view to see whether the gain is larger than the noise.

8.2.3 The Platform Detects Failure as Well as Success

An analytics layer is only useful if it notices a system in trouble. In the saturated benchmark the arrival rate is twice the service rate. 1,181 entities arrive but only about 603 complete, utilisation is 0.997 and work-in-progress climbs to 578. The Little's Law verdict switches to *accumulating backlog* and the health score drops to 45.8, well under the alert threshold of 70. So the platform separates a healthy system from a failing one without asking the user to interpret raw numbers. A separate smoke test also confirmed that a model using the extended node types (broadcaster, channel, store and container) runs without error and conserves flow, with 1,225 entities in and out at every stage.

8.3 Performance Evaluation

We assessed performance on two levels. The first is the speed of the **engine**, measured by the verification harness, which runs the generated SimPy script in native CPython. The second is how responsive the **application** is in the browser, measured with the browser's developer tools.

8.3.1 Engine Throughput

Table 8.3: Engine Throughput Measured by the Verification Harness

Measurement	Workload	Observed
Analytical benchmark suite (B1–B5)	Five replications of each benchmark at a 36,000 simulated-second horizon; the M/M/1 benchmark generates $\approx 28,600$ entities per replication	159 s wall-clock for the whole suite
Single long run	28,714 entities through one resource	3.65 s wall-clock
Complex model	Hospital flow, 7,200 simulated seconds, 3 seeds, two configurations	Completed within the harness run
Clock accuracy	36,000 accumulated timeouts	Simulated clock exactly 36,000.0

One run with nearly thirty thousand entities finishes in under four seconds, and a working day of a typical service model takes far less. A discrete event simulation jumps from event to event, so run time depends on how many events occur and not on the simulated duration, and the heap-based event queue keeps the cost per event logarithmic. In the browser the same script runs through WebAssembly, which for CPU-bound Python is usually within a small multiple of native speed.

8.3.2 Application Performance in the Browser

The measurements below are to be taken on the target device with Chrome DevTools and Lighthouse and entered in the table.

Table 8.4: Browser Performance Measurements (to be recorded)

Measurement	Target	Measured
Canvas interaction (drag / connect a node)	Immediate; no dropped frames	
First-run Pyodide download and start-up	Cached thereafter	
Run of a 10-node model, 1,000 simulated seconds	Interactive	
Run of a 100-node model, 1,000 simulated seconds	A few seconds	
Dashboard render after a run	Perceptibly immediate	
Peak browser memory for the 100-node run	Well below device limits	
Lighthouse performance score (landing page)	≥ 90	
Presence / cursor update between two sessions	Near real time	

8.3.3 Discussion

The measurements back the main claim of the project, that a genuine discrete event simulator can run acceptably inside a browser. Verification is the more important result. The engine agrees with queueing theory to within a few per cent, conserves entities across a multi-stage model, and classifies both healthy and overloaded systems correctly. That shows the platform is not just fast enough but *correct*, and for a simulator correctness is what counts. Three points follow from the results and feed into the roadmap of Chapter 10. Memory, and not processing speed, sets the ceiling on model size. Monte Carlo replications are independent, so they are good candidates for running in parallel. And the first-run download of the runtime is the first thing a new user notices, which makes it the best target for optimisation.

9. COST AND BENEFIT ANALYSIS AND SOFTWARE PARAMETER ESTIMATION

This chapter estimates the effort, schedule and cost that JustCmul8 would represent if a company built it commercially. We apply two independent models, the Constructive Cost Model (COCOMO), which starts from code size, and Function Point Analysis (FPA), which starts from delivered functionality. Their results are compared with each other and with the effort we actually spent. The chapter ends with a cost and benefit analysis of the platform for its users.

9.1 Effort Estimation

9.1.1 Basic COCOMO

Basic COCOMO estimates effort from size, using coefficients that depend on the development mode. We classify JustCmul8 as **organic**, meaning a small team in a well-understood application domain with stable requirements and flexible constraints.

Table 9.1: COCOMO Coefficients by Development Mode

Mode	a	b	c	d	Applicable
Organic	2.4	1.05	2.5	0.38	Yes: small team, familiar domain, stable requirements
Semi-detached	3.0	1.12	2.5	0.35	No: assumes a medium team of mixed experience
Embedded	3.6	1.20	2.5	0.32	No: assumes tight hardware and regulatory constraints

For a measured size of 22.19 KSLOC (Section 9.2) the model gives:

$$\text{Effort } E = a \times (\text{KSLOC})^b = 2.4 \times (22.19)^{1.05} = 2.4 \times 25.93 = \mathbf{62.19 \text{ person-months}}$$

$$\text{Duration } D = c \times E^d = 2.5 \times (62.19)^{0.38} = 2.5 \times 4.80 = \mathbf{12.01 \text{ months}}$$

$$\text{Staffing } S = E / D = 62.19 / 12.01 = \mathbf{5.18 \text{ persons}}$$

$$\text{Productivity } P = \text{KSLOC} / E = 22.19 / 62.19 = \mathbf{0.357 \text{ KSLOC per person-month}}$$

Table 9.2: COCOMO Estimation Results

Parameter	Estimated value
Effort	62.19 person-months
Development duration	12.01 months

Parameter	Estimated value
Average staffing	5.18 persons
Productivity	0.357 KSLOC per person-month
Cost at ₹40,000 per person-month	₹24,87,600 ($\approx$ ₹24.88 lakh)

The money figure uses a notional forty thousand rupees per person-month, roughly an entry-level Indian software engineering salary with overhead. The rate is only illustrative; effort and duration are the substantive output of the model.

9.2 KSLOC Analysis

We measured size from the delivered source tree, counting TypeScript, TSX and Python files under the application's source directory and leaving out dependencies, build output and generated artefacts.

Table 9.3: Source Lines of Code by Layer

Layer / directory	Contents	SLOC	Share
src/components	Workspace, landing, layout and shared interface components	11,544	52.0%
src/lib/simulation	Code generator, engine, analytics, Monte Carlo, export, registry	4,905	22.1%
src/app	Pages and API routes (auth, dashboard, workspace, share, AI, export)	3,848	17.3%
src/lib/ai	Model router, prompts, graph operations, flow analysis	1,654	7.5%
src/lib/realtime	Presence and broadcast hook	143	0.6%
src/lib/supabase, hooks	Database clients and helper hooks	59	0.3%
src/middleware.ts	Session refresh and route protection	39	0.2%
Total	91 source files (.ts, .tsx)	22,192	100%

Table 9.4: Measured Project Size

Measure	Value
Source files	91
Total lines of source code	22,192
Non-blank lines	20,286
Size in KSLOC	22.19

Measure	Value
Application pages	9 pages and 7 API routes (including the authentication callback)
Node types implemented	15
Simulation domains	6
Delivered modules	12
Database tables (application)	5, plus 2 access functions and 1 trigger

More than half of the code (52 per cent) is interface code, which reflects how much the workspace contains: a canvas editor, eight result tabs, a Monte Carlo panel, a WebGL digital twin and a landing page. The engine, analytics and export logic, which decide whether the numbers are right, take another 22 per cent, and the AI layer 7.5 per cent. The database schema is small on purpose, since the graph is stored as a document and Supabase provides authentication and authorisation.

9.3 Function Point Analysis

Function Point Analysis measures the functionality delivered and not the code written, so it works as a cross-check on a size-based estimate. The IFPUG method counts five categories of component, weights each by complexity and adjusts the total with fourteen general system characteristics.

Table 9.5: Standard IFPUG Complexity Weights

Component type	Simple	Average	Complex
External Input (EI)	3	4	6
External Output (EO)	4	5	7
External Inquiry (EQ)	3	4	6
Internal Logical File (ILF)	7	10	15
External Interface File (EIF)	5	7	10

Table 9.6: Function Point Count by Component Type

Component type	Simple	Average	Complex	Count	Weighted
External Inputs: forms, edits and commands	6	8	4	18	74
External Outputs: dashboards, charts and exports	2	4	4	10	56

Component type	Simple	Average	Complex	Count	Weighted
External Inquiries: listings and retrievals	2	3	2	7	30
Internal Logical Files: application tables	3	0	2	5	51
External Interface Files: Auth, Gemini, Pyodide CDN	1	2	0	3	19
Unadjusted Function Points (UFP)				43	230

The eighteen external inputs are the authentication forms, project creation, renaming and deletion, graph edits, parameter configuration, template loading, run initiation, AI prompts, fix application, share-link creation and revocation, profile update and account deletion. The ten external outputs are the KPI dashboard, the bottleneck flag, the Monte Carlo interval charts, the digital-twin playback, the streamed log, the entity journey trace and the export formats. The five internal logical files are the database tables of Section 5.6. Two of them, projects and runs, count as complex because they hold JSON documents.

Table 9.7: General System Characteristics and Degrees of Influence

No.	General system characteristic	Rating	Justification
1	Data communications	4	Client, server, database and AI service all communicate
2	Distributed data processing	5	Simulation executes on the client, not the server
3	Performance	4	Responsive canvas and immediate dashboard expected
4	Heavily used configuration	2	No constrained deployment hardware
5	Transaction rate	3	Moderate, rising during collaborative editing
6	Online data entry	5	All modelling is interactive and immediate
7	End-user efficiency	5	No-code operation is a primary objective
8	Online update	5	Automatic persistence on every change
9	Complex processing	5	Discrete event simulation, Monte Carlo statistics, AI
10	Reusability	3	Service modules reusable; interface is application-specific
11	Installation ease	5	Zero installation by design
12	Operational ease	4	Managed hosting with automatic deployment
13	Multiple sites	3	Browser-portable across platforms
14	Facilitate change	4	Registry-driven extension of domains and node types
	Total Degree of Influence (TDI)	57	

$$\text{VAF} = 0.65 + (0.01 \times \text{TDI}) = 0.65 + 0.57 = \mathbf{1.22}$$

$$\text{Function Points } \text{FP} = \text{UFP} \times \text{VAF} = 230 \times 1.22 = \mathbf{281 \text{ function points}}$$

$$\text{Effort } E = \text{FP} / \text{productivity} = 281 / 13 = \mathbf{21.6 \text{ person-months}}$$

$$\text{Cost at ₹40,000 per person-month} = 21.6 \times 40,000 = \mathbf{₹8,64,000 (\approx ₹8.64 \text{ lakh})}$$

$$\text{Implied size } 22,192 \text{ SLOC} / 281 \text{ FP} = \mathbf{79 \text{ SLOC per function point}}$$

For a modern web application of this kind, industry productivity is around thirteen function points per person-month. The implied 79 lines per function point is above the usual 50 to 60 for TypeScript. The reason can be seen in Table Table 9.3: a very large share of the code is presentation, and the landing page alone is about 1,900 lines. That code adds polish and usability but few function points.

9.4 Comparison of Estimates and Cost–Benefit Analysis

Table 9.8: Comparison of Estimates against Planned Effort

Basis	Effort	Duration	Team	Indicative cost
COCOMO (basic, organic)	62.19 PM	12.01 months	5.18 persons	₹24.88 lakh
Function Point Analysis	21.6 PM	n/a	n/a	₹8.64 lakh
Actual project (planned)	12 PM	6 months	2 students	Nil cash cost; academic, open-source stack

The three rows differ a lot, and the gap says more than any single number. Both models were calibrated on projects written largely from scratch, and neither allows for the head start that a modern framework stack gives. The simulation engine is the clearest case. A discrete event scheduler with resource contention, preemption, container levels and typed stores would probably account for a third of the COCOMO estimate on its own, yet we got all of it from SimPy, and our effort went into the code generator that targets it. The same holds for React Flow, which gave us the canvas, and for Supabase, which gave us authentication, access control and the realtime transport. The twenty-two thousand lines we wrote sit on top of a very large amount of code we did not write. Function Point Analysis lands nearer to our actual effort than COCOMO does, which fits with its measuring delivered functionality and not code volume.

The estimates therefore give an upper bound on what the platform would cost to build without such foundations, and they show what the open-source ecosystem is worth to a project like this. A commercially built equivalent would probably fall between the two estimates.

9.4.1 Costs

Table 9.9: Development and Operating Costs

Item	Nature	Cost
Development effort	Two students, six months	Academic; no cash cost
Software licences	Entire stack is open source or free-tier	Nil
Hosting (front end)	Vercel free tier	Nil at academic scale
Database, authentication, realtime	Supabase free tier	Nil at academic scale; paid tiers scale with data volume
AI inference	Gemini API, metered per explicit request	Small and controllable
Simulation compute	Runs on the user's own device	Nil; does not grow with usage
Maintenance	Dependency upgrades, registry additions	Student / community effort

9.4.2 Benefits

Table 9.10: Benefits of the Platform

Benefit	Type	Beneficiary
Removes per-seat licence fees that commercial tools charge	Tangible, recurring	Institutions, small businesses
Removes installation and IT-support effort on every machine	Tangible	Laboratories, IT departments
No server compute cost that scales with the number of simulations	Tangible	The platform operator
Decisions tested on a computer instead of on the real system, avoiding costly mistakes	Tangible, decision-dependent	Small-business owners
Time to first result measured in minutes rather than days of training	Tangible (time)	Students, lecturers
Wider access to a decision-support technique	Intangible	Society, education
A transparent, reproducible engine that supports teaching of SimPy and queueing theory	Intangible	Educators, researchers

The balance of cost and benefit is unusually good. The main cost, computation, moves onto hardware the user already owns, while the main benefit, access to trustworthy simulation, arrives with no licence fee. A single commercial seat at a few thousand dollars a year would cost more than the whole annual running cost of the platform at academic scale, and each extra user adds benefit without adding cost.

10. CONCLUSION AND FUTURE SCOPE

10.1 Achievements of the Project

JustCmul8 set out to remove three barriers that keep Discrete Event Simulation away from most of the people who could use it: the price of commercial licences, the programming skill that open-source alternatives demand, and the installation that desktop software needs. What the project achieved is summarised below.

- **A complete no-code simulation platform:** twelve integrated modules, from creating an account and modelling on a canvas to analysis, sharing and collaboration, all running in a web browser.
- **A real engine and not an imitation:** each model is compiled into a genuine Python SimPy program and run by CPython compiled to WebAssembly in a Web Worker, so results come from the same library researchers use directly.
- **Verified correctness:** all 28 analytical checks against M/M/1, M/M/c and saturated-queue theory passed within tolerance, every deviation was below 2.1 per cent, and all 22 conservation and flow-balance invariants held on a multi-stage hospital-flow model.
- **Breadth of modelling:** fifteen node types across six application domains, including preemptive resources, typed stores, containers, channels, broadcasters, event synchronisers and interrupters.
- **Decision support and not only numbers:** bottleneck flagging, a Little's Law consistency check, a composite health score, cost and return-on-investment analysis, and Monte Carlo confidence intervals with baseline-versus-scenario comparison.
- **AI-assisted workflow:** plain-English model generation and one-click, cost-aware fixes, served through a router that watches model health and fails over between models automatically.
- **Output that is easy to understand:** an eight-tab results dashboard, a WebGL digital-twin animation, an entity tracer and a traffic heatmap turn statistics into pictures.
- **Transparency and reproducibility:** a live Python code inspector and one-click Jupyter export let anyone verify a result independently.
- **Security built in:** Row-Level Security on every table, opaque revocable share links served through narrow database functions, secrets kept on the server and cascading deletion of user data.

- **Sustainable economics:** client-side computation and an open-source stack keep the running cost almost independent of usage.

On the academic side, the project put material from across the programme to work and did not just cite it. Priority queues and graph traversal from data structures appear in the event scheduler and the validation pass. Normalisation, referential integrity and access control from database management appear in the schema and its policies. The software development life cycle shows in the scheduling decisions of Chapter 3, including the choice to tackle the riskiest component first. Probability and statistics appear in the distribution samplers, the confidence intervals and the check against queueing theory.

10.2 Conclusion

The decision that shaped the result most was our refusal to approximate the engine. Writing a rough event loop in JavaScript would have been far easier, and the platform would have looked much the same from outside. Instead, the graph is compiled into a genuine SimPy script, and the verification in Chapter 7 turns that from a claim into evidence: agreement with theory to within a few per cent shows that the generated script expresses the intended model and that the statistics are gathered correctly.

In practical terms the platform can be used today. A student can open a template and see queueing behaviour within a minute. A lecturer can demonstrate congestion live and send a share link afterwards. A small-business owner can describe a shop in plain English and read off a confidence interval before committing any money. All of this needs no licence, no installation and no code, which was the outcome we wanted. JustCmul8 shows that serious simulation can be as easy to reach as a web page, and that accessibility does not have to cost accuracy.

10.3 Future Scope

We designed the architecture to be extended: one registry defines each domain, the engine sits behind a worker boundary, and authorisation lives in the database. The roadmap below builds on those decisions and does not replace them, and it answers each limitation recorded in Section 1.4.

Near-term enhancements

- **Parallel Monte Carlo:** spreading replications over several Web Workers, so a hundred-replication analysis takes close to the machine's core count times less, since the replications are independent.
- **Reproducible runs:** user-selectable random seeds, so any run can be repeated exactly and shared together with its seed.
- **Pause and resume:** checkpointing, so a long run can be suspended and continued later.
- **Streaming result aggregation:** dropping per-entity detail once it has been aggregated and keeping full traces only when playback is requested, which raises the ceiling on model size.
- **Role-based collaboration:** invitations by email with owner, editor and viewer permissions enforced by Row-Level Security, and version history with rollback.
- **Convergent conflict resolution:** replacing last-write-wins with a merge based on operational transformation or CRDTs, so two users can edit the same node without either losing work.
- **Input-validation hardening:** turning every boundary condition, such as a zero arrival rate, into a friendly message before a run.
- **Progressive runtime loading:** fetching a reduced Pyodide payload to shorten the first-run start-up.

Medium-term extensions

- **Custom node types through a plugin interface,** run safely inside the existing sandbox, so users can model domains we did not foresee.
- **More simulation paradigms:** an agent-based engine and a true continuous solver next to SimPy, chosen per project.
- **Search-based optimisation:** sweeping a parameter space to find the configuration with the lowest cost that still meets a service-level constraint, so that advice becomes optimisation.
- **Offline and classroom mode:** a progressive web application with local storage for laboratories without dependable internet.
- **A native mobile companion** for viewing dashboards and playback.
- **A community template marketplace** where users publish and rate models for their own domains.

Longer-term directions

- **Optional cloud execution** for models too big for browser memory, with client-side execution staying the default.
- **Parallel and distributed simulation:** splitting very large models across workers or machines with synchronised clocks.
- **Calibration from real data:** fitting arrival and service distributions automatically from historical or sensor data, so measured parameters replace estimated ones.
- **Live digital-twin integration:** linking a running model to IoT feeds so the simulation follows the physical system it represents.
- **Institutional features:** single sign-on, organisation management and classroom cohorts for department-scale adoption.

None of these requires us to revisit what we decided in this project. The layered architecture, the worker boundary, the registry-driven definition of domains and node types and the enforcement of authorisation at the database were each picked to make extension possible, and every item above builds on them.

REFERENCES AND BIBLIOGRAPHY

References follow IEEE citation style. Textbooks and articles that give the theoretical basis come first, then standards, then the technical documentation of the software the platform is built on.

- [1] A. M. Law, *Simulation Modeling and Analysis*, 5th ed. New York, NY, USA: McGraw-Hill Education, 2015.
- [2] J. Banks, J. S. Carson, B. L. Nelson and D. M. Nicol, *Discrete-Event System Simulation*, 5th ed. Upper Saddle River, NJ, USA: Pearson Prentice Hall, 2010.
- [3] B. P. Zeigler, H. Praehofer and T. G. Kim, *Theory of Modeling and Simulation*, 2nd ed. San Diego, CA, USA: Academic Press, 2000.
- [4] S. Robinson, *Simulation: The Practice of Model Development and Use*, 2nd ed. Basingstoke, U.K.: Palgrave Macmillan, 2014.
- [5] D. Gross, J. F. Shortle, J. M. Thompson and C. M. Harris, *Fundamentals of Queueing Theory*, 5th ed. Hoboken, NJ, USA: John Wiley & Sons, 2018.
- [6] L. Kleinrock, *Queueing Systems, Volume 1: Theory*. New York, NY, USA: Wiley-Interscience, 1975.
- [7] J. D. C. Little, “A proof for the queuing formula: $L = \lambda W$,” *Operations Research*, vol. 9, no. 3, pp. 383–387, 1961.
- [8] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.
- [9] G. E. P. Box and M. E. Muller, “A note on the generation of random normal deviates,” *The Annals of Mathematical Statistics*, vol. 29, no. 2, pp. 610–611, Jun. 1958.
- [10] A. B. Owen, *Monte Carlo Theory, Methods and Examples*. Stanford, CA, USA: Stanford University, 2013.
- [11] B. W. Boehm, *Software Engineering Economics*. Englewood Cliffs, NJ, USA: Prentice Hall, 1981.
- [12] A. J. Albrecht and J. E. Gaffney, “Software function, source lines of code, and development effort prediction: A software science validation,” *IEEE Transactions on Software Engineering*, vol. SE-9, no. 6, pp. 639–648, Nov. 1983.
- [13] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner’s Approach*, 9th ed. New York, NY, USA: McGraw-Hill Education, 2020.

- [14] I. Sommerville, Software Engineering, 10th ed. Harlow, U.K.: Pearson Education, 2015.
- [15] IEEE Recommended Practice for Software Requirements Specifications, IEEE Std 830-1998, 1998.
- [16] G. Booch, J. Rumbaugh and I. Jacobson, The Unified Modeling Language User Guide, 2nd ed. Boston, MA, USA: Addison-Wesley, 2005.
- [17] A. Silberschatz, H. F. Korth and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill Education, 2019.
- [18] Team SimPy, “SimPy: Discrete event simulation for Python, documentation.” [Online]. Available: <https://simpy.readthedocs.io/> [Accessed: Sep. 2026].
- [19] Pyodide Development Team, “Pyodide: Python with the scientific stack, compiled to WebAssembly.” [Online]. Available: <https://pyodide.org/> [Accessed: Sep. 2026].
- [20] WebAssembly Community Group, “WebAssembly Core Specification,” W3C Recommendation, 2019. [Online]. Available: <https://www.w3.org/TR/wasm-core-1/> [Accessed: Sep. 2026].
- [21] Vercel Inc., “Next.js Documentation: App Router.” [Online]. Available: <https://nextjs.org/docs> [Accessed: Sep. 2026].
- [22] Meta Open Source, “React Documentation.” [Online]. Available: <https://react.dev/> [Accessed: Sep. 2026].
- [23] xyflow, “React Flow Documentation.” [Online]. Available: <https://reactflow.dev/> [Accessed: Sep. 2026].
- [24] Supabase Inc., “Supabase Documentation: Database, Auth, Realtime and Row Level Security.” [Online]. Available: <https://supabase.com/docs> [Accessed: Sep. 2026].
- [25] PostgreSQL Global Development Group, “PostgreSQL Documentation: Row Security Policies.” [Online]. Available: <https://www.postgresql.org/docs/> [Accessed: Sep. 2026].
- [26] Google, “Gemini API Documentation.” [Online]. Available: <https://ai.google.dev/docs> [Accessed: Sep. 2026].
- [27] Microsoft Corporation, “TypeScript Handbook.” [Online]. Available: <https://www.typescriptlang.org/docs/> [Accessed: Sep. 2026].
- [28] Mozilla Developer Network, “Using Web Workers.” [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API [Accessed: Sep. 2026].
- [29] International Function Point Users Group, Function Point Counting Practices Manual, Release 4.3.1. Princeton Junction, NJ, USA: IFPUG, 2010.